



Octane Security Report

Security Analysis of SailMoney: Protocol
6/29/2026

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

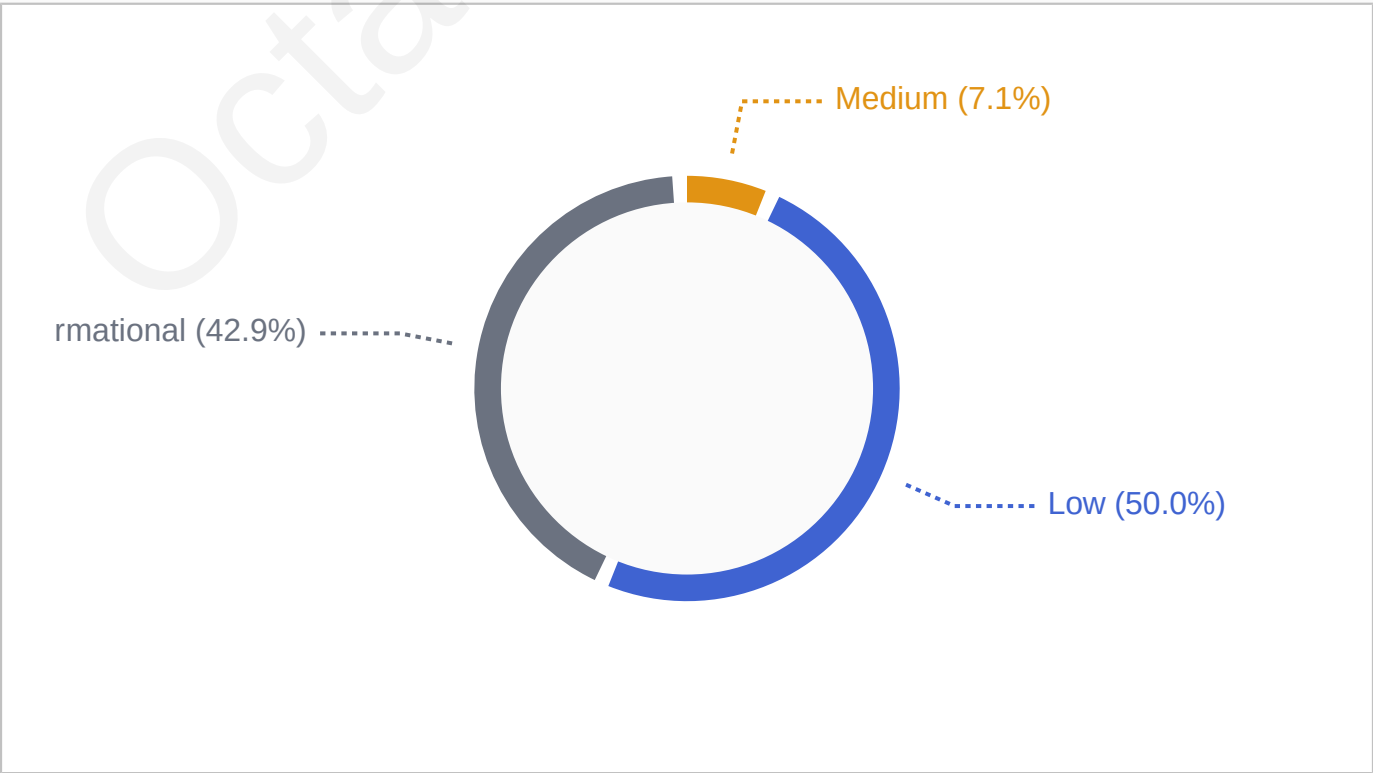
Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This analysis reviewed the SailMoney: Protocol smart contracts using Octane’s automated analysis and included team feedback on findings.

The analysis identified a total of 14 issues. After triage and review, 14 of these issues have been resolved or acknowledged.

Vulnerabilities Confirmed	14/14
Critical	00/00
High	00/00
Medium	01/01
Low	07/07
Informational	06/06



Vulnerabilities & Warnings

Vulnerability #1

Unchecked Safe.setup payment fields in SailKernel.createAccount forwarding initializer causes loss of prefunded assets during account creation

Severity: MEDIUM

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

createAccount now rejects any non-zero Safe.setup payment configuration in the forwarded initializer (paymentToken, payment, paymentReceiver), reverting before deployment. The minimum initializer length was raised so the payment words are always in bounds. The check is a runtime revert on the same initializer bytes and changes no constructor argument, salt, init code, or singleton reference – the derived CREATE2 address is byte-identical across all chains. Honest onboarding does not use Safe's deployment-payment feature and is unaffected; a deployment that would have skimmed a pre-funded counterfactual address now reverts. Fixed and merged.

SailKernel.createAccount forwards the Safe.setup initializer unchanged except for validating the delegatecall target, leaving Safe.setup's payment fields (paymentToken, payment, paymentReceiver) usable at setup time. If the predicted Safe address is pre-funded, setup can immediately transfer ETH or ERC-20 from the new Safe to a receiver (or to tx.origin if paymentReceiver is zero). In a shared-caller relayer model, this enables draining a user's pre-funded assets during account creation.

SailKernel.createAccount validates only the Safe.setup delegatecall "to" address and its runtime codehash, then forwards the entire initializer to SafeProxyFactory.createProxyWithNonce. It does not inspect or constrain Safe.setup's paymentToken, payment, or paymentReceiver fields. After enabling modules, Safe.setup executes a payment when payment > 0: for ETH, it routes funds to paymentReceiver, or to tx.origin if paymentReceiver == 0; for ERC-20, it transfers tokens to paymentReceiver via transferToken. Because the Safe proxy is deployed via CREATE2 with a predictable address bound to msg.sender and the initializer hash, an operator/relayer can disclose the predicted address to a user, the user can pre-fund that address, and then the relayer can deploy using an initializer that both enables the SailKernel module and sets a nonzero payment, causing setup to transfer the pre-funded assets out of the Safe during account creation. If the initializer fails to enable the kernel module or the Safe lacks sufficient funds for the requested payment, createAccount reverts and the entire transaction (including payment) is rolled back. Otherwise, the payment succeeds and the account registers, leaving the Safe empty of the pre-funded assets.

```
560     error InvalidInitializer();
561
562     /// @dev Thrown by `createAccount` when the Safe.setup
563     delegatecall `to` target is not
564     ///         in governance's trusted module-setup allowlist.
565
566     ...
567
578     // setup() ABI layout: selector(4) + owners_offset(32) +
579     // threshold(32) + to(32) + ...
580     // 'to' is at bytes [68:100].
581     - if (safeInitializer.length < 100) revert InvalidInitial-
582       izer();
583     address setupTarget = ad-
584       dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
585     // address(0) means no delegatecall (vanilla Safe.setup)
586     - always safe, no allowlist check needed.
587
588     ...
589
590     revert UntrustedModuleSetupCodehash(setupTarget);
591   }
592
593   ...
594
595   745
596   746     uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
597     msg.sender, permissionSigner, manager, feePolicy)));
598   }
```

```
560     error InvalidInitializer();
561
562 + error NonZeroSetupPayment();
563     /// @dev Thrown by `createAccount` when the Safe.setup
564     delegatecall `to` target is not
565     ///         in governance's trusted module-setup allowlist.
566
567     ...
```

```

731 // setup() ABI layout: selector(4) + owners_offset(32) +
    threshold(32) + to(32) + ...
732 // 'to' is at bytes [68:100].
733 + if (safeInitializer.length < 260) revert InvalidInitial-
    izer();
734   address setupTarget = ad-
    dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
735 // address(0) means no delegatecall (vanilla Safe.setup)
    – always safe, no allowlist check needed.

...

744   revert UntrustedModuleSetupCodehash(setupTarget);
745 }
746 +// Reject any Safe.setup payment during account creation
    (ETH or ERC-20).
747 + address paymentToken = ad-
    dress(uint160(uint256(bytes32(safeInitializer[164:196]))));
748 + uint256 payment      = uint256(bytes32(safeInitializ-
    er[196:228]));
749 + address paymentRecv  = ad-
    dress(uint160(uint256(bytes32(safeInitializer[228:260]))));
750 + if (paymentToken != address(0) || payment != 0 || paymentRecv
    != address(0)) revert NonZeroSetupPayment();
751
752   uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
    msg.sender, permissionSigner, manager, feePolicy)));

```

Severity

Impact Explanation: [High] Direct, non-recoverable loss of principal ETH or ERC-20 from the user's pre-funded Safe address to an external party during setup; value leaves the account beneficiary set.

Likelihood Explanation: [Medium] Exploitation requires an uncommon but realistic operator setup (shared-caller relay) and user pre-funding of the predicted Safe address; both are plausible but not universal defaults.

Exploitation

Exploitation Scenarios:

Scenario 1.

Relayer drains user's pre-funded ETH via tx.origin fallback: a shared-caller relayer submits createAccount with a Safe.setup initializer that enables the kernel module and sets paymentToken=ETH, payment>0, paymentReceiver=0; Safe.setup pays ETH to tx.origin (the relayer) during setup.

Preconditions / Assumptions

- (a). Operator uses a shared-caller relayer/bundler as the top-level caller of createAccount (tx.origin = relayer EOA).
- (b). Relayer provides the predicted Safe address, computed from the relayer's msg.sender, chosen singleton/factory, and initializer.
- (c). User pre-funds the predicted Safe address with ETH before createAccount.
- (d). Initializer enables the SailKernel module via the allowlisted SafeModuleEnabler and matches the pinned codehash.
- (e). Initializer sets paymentToken = address(0), payment > 0, paymentReceiver = address(0).
- (f). Safe has sufficient ETH to satisfy the payment; otherwise setup reverts and no loss occurs.
- (g). Governance allowlists for Safe factory, singleton, setup target, and proxy codehash are correctly configured (normal case).



Scenario 2.

Relayer drains user's pre-funded ERC-20 via explicit paymentReceiver: a shared-caller relayer submits createAccount with a Safe.setup initializer that enables the kernel module and sets paymentToken=, payment>0, paymentReceiver=; Safe.setup transfers tokens to the relayer during setup.

Preconditions / Assumptions

- (a). Operator uses a shared-caller relayer as the top-level caller of createAccount.
- (b). Relayer provides the predicted Safe address to the user.
- (c). User pre-funds the predicted Safe address with an ERC-20 token.
- (d). Initializer enables the SailKernel module via the allowlisted SafeModuleEnabler and matches the pinned code-hash.
- (e). Initializer sets paymentToken = , payment > 0, paymentReceiver = .
- (f). Token adheres to standard ERC-20 behavior; the Safe's balance is sufficient for the payment; otherwise setup reverts.
- (g). Governance allowlists for Safe components are correctly configured.

Scenario 3.

Relayer over-collects a "creation fee" from pre-funded ETH: a shared-caller relayer instructs the user to pre-fund and then uses a Safe.setup initializer that enables the kernel module and sets a large ETH payment to their address (or relies on tx.origin), extracting more than any disclosed fee during setup.

Preconditions / Assumptions

- (a). Operator uses a shared-caller relayer as the top-level caller of createAccount.
- (b). Relayer instructs the user to pre-fund the predicted Safe address to cover creation fees.
- (c). User pre-funds the predicted Safe address with ETH.
- (d). Initializer enables the SailKernel module via the allowlisted SafeModuleEnabler and matches the pinned code-hash.
- (e). Initializer sets paymentToken = address(0), payment = a large amount ($\leq$ pre-funded balance), and paymentReceiver = relayer EOA (or paymentReceiver = 0 to route to tx.origin).
- (f). Safe has sufficient ETH to satisfy the payment; other-



wise setup reverts.

(g). Governance allowlists are correctly configured.

Octane Security

Vulnerability #2

Missing output-token/min-out and default-unpinned recipient in ApproveAndCallBatchPermission allows manager-led value exfiltration or destruction

Severity: LOW

Likelihood: HIGH

Impact: LOW

Status: Resolved

The batch template now pins the output recipient to the account by default – the previously opt-in recipient pin is now the default, with an explicit opt-out preserved for operators who deliberately want an unconstrained recipient. This closes the default-unsafe recipient window (an adversarial manager could otherwise direct output elsewhere when the pin was left off). Note: denying swap selectors entirely and forcing the pin always-on were both evaluated and rejected – each removed legitimate, exercised capability (swaps belong in the dedicated SwapPermission, which constrains tokenOut/min-out; forcing always-on removed operator choice and broke the documented opt-out mode). Fixed and merged.

ApproveAndCallBatchPermission enforces an approve/consume/reset bracket that binds only tokenIn and spender (and optionally amount/recipient). It does not validate tokenOut, V2 path tail/hops, V3 fee tier/pool choice, or amountOut-Min, and by default leaves the output recipient unconstrained. A malicious manager can use allowlisted Uniswap/Aave calls to exfiltrate funds to arbitrary recipients when recipient pinning is off, or destroy value by swapping into low-value tokens even when recipient pinning is on.

The template ApproveAndCallBatchPermission.evaluateBatch strictly requires three calls: (0) approve(spender, amount) on an allowlisted token with pre-batch allowance==0 and amount≤cap, (1) a consuming call hitting an allowlisted (target,selector) pair with target==spender and decoded consumed asset==token from call 0 (optionally amountIn==approveAmount and recipient==account), and (2) approve(spender, 0) to reset the allowance. For swaps and similar consuming calls, the decoder only binds tokenIn (V2 path[0], V3 tokenIn) and spender; it never validates tokenOut, V2 path tail/hops, V3 fee tier/pool choice, or amountOut-Min. The optional recipient pin (requireRecipientIsAccount) is off by default; when off, the output recipient (e.g., V2 'to', V3 'recipient', Aave 'onBehalfOf', ERC-4626 'receiver') is unconstrained. As a result, an adversarial manager can: (a) exfiltrate the full approved amount by swapping to an attacker-controlled recipient via Uniswap V2/V3, or by depositing/supplying onBehalfOf an attacker at Aave/4626, or (b) even with recipient pinning on, route an exact-in swap through an illiquid/manipulated pool to deliver low-value or dust tokens to the account itself. While this behavior is explicitly documented as a design tradeoff

of this bracket template, it still enables manager-led loss within the template's authorized envelope unless safer swap templates or stricter config are used.

file: contracts/templates/ApproveAndCallBatchPermission.sol

```
181     error EmptyAllowlist();
182     error AllowlistTooLong();

183
184     /// @notice Tooling-layer attribution for the template author.
    The kernel never reads this.

...

253     ConsumingPair memory p = cfg.consumingPairs[i];
254     if (p.target == address(0) || p.selector == bytes4(0))
    revert EmptyAllowlist();

255     isConsumingPair[account][_pairKey(p.target, p.selector)]
    = true;
256 }

...

376     if (recipient != account) return false;
377 }

378
379 // calls[2] approve(spender, 0) – strict reset of same
    token & spender
```

```
181     error EmptyAllowlist();
182     error AllowlistTooLong();
183 + error SwapSelectorsNotSupported();
184
185     /// @notice Tooling-layer attribution for the template author.
    The kernel never reads this.

...
```

```

254 ConsumingPair memory p = cfg.consumingPairs[i];
255 if (p.target == address(0) || p.selector == bytes4(0))
    revert EmptyAllowlist();
256 + if (p.selector == SWAP_EXACT_TOKENS_FOR_TOKENS ||
    p.selector == EXACT_INPUT_SINGLE || p.selector == EXACT_INPUT_SINGLE_02) revert SwapSelectorsNotSupported();
257 isConsumingPair[account][_pairKey(p.target, p.selector)]
    = true;
258 }

...

378 if (recipient != account) return false;
379 }
380 + else {
381 + if (sel == AAVE_V3_SUPPLY || sel == AAVE_V2_DEPOSIT
    || sel == ERC4626_DEPOSIT || sel == ERC4626_MINT) {
382 +(bool d, address r) = _decodeRecipient(sel,
    c1.data);
383 + if (!d || r != account) return false;
384 +}
385 +}
386
387 // calls[2] approve(spender, 0) – strict reset of same
    token & spender

```

Severity

Impact Explanation: [Low] Although raw effects include principal loss/exfiltration, the scope severity guidance caps impact when actions remain inside the permission's authorized envelope (recipient unconstrained by design) or when value stays within the beneficiary set. This bracket template explicitly permits unconstrained recipients by default and does not enforce tokenOut/min-out.

Likelihood Explanation: [High] No special constraints beyond typical configuration and an adversarial manager: allowlisting Uniswap/Aave selectors is plausible for this bracket template, and the default recipient mode is unconstrained. The manager controls calldata within the registered permission's bounds.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (Uniswap V2, recipient pinning OFF): Manager signs a batch: [approve(USDC, router, X), router.swapExactTokensForTokens(X, 0, [USDC, WETH], attackerEOA, deadline), approve(USDC, router, 0)]. The template binds tokenIn (USDC) and spender (router) but does not constrain the recipient when requireRecipientIsAccount=false, so proceeds are sent to attackerEOA and the account loses X USDC.

Preconditions / Assumptions

- (a). Safe account registered and holds tokenIn (e.g., USDC)
- (b). ApproveAndCallBatchPermission registered and config current for the account
- (c). tokens include tokenIn with sufficient maxApprovalAmount
- (d). spenders include Uniswap V2 router
- (e). consumingPairs include (router, swapExactTokensForTokens)
- (f). requireRecipientIsAccount=false (default)
- (g). Pre-batch allowance on (tokenIn, router) is zero
- (h). Manager key is adversarial/compromised and signs the



batch

Scenario 2.

Scenario 2 (Aave V2/V3, recipient pinning OFF): Manager signs a batch: [approve(USDC, AavePool, X), AavePool.supply/deposit(USDC, X, onBehalfOf=attackerEOA, ...), approve(USDC, AavePool, 0)]. The template binds tokenIn and spender but does not constrain onBehalfOf when recipient pinning is off, so aTokens/credit go to the attacker and the account loses X USDC.

Preconditions / Assumptions

- (a). Safe account registered and holds tokenIn (e.g., USDC)
- (b). ApproveAndCallBatchPermission registered and config current for the account
- (c). tokens include tokenIn with sufficient maxApprovalAmount
- (d). spenders include Aave V2/V3 pool
- (e). consumingPairs include (AAVE_V2_DEPOSIT or AAVE_V3_SUPPLY on that pool)
- (f). requireRecipientIsAccount=false (default)
- (g). Pre-batch allowance on (tokenIn, AavePool) is zero
- (h). Manager key is adversarial/compromised and signs the batch

Scenario 3.

Scenario 3 (Uniswap V3, recipient pinning ON): Manager signs a batch: [approve(USDC, router, X), router.exactInputSingle({tokenIn:USDC, tokenOut:XYZ, fee:tier, recipient:account, amountIn:X, amountOutMinimum:0, ...}), approve(USDC, router, 0)]. With recipient pinning on, outputs must go to the account, but tokenOut/min-out are still unconstrained; the manager selects an illiquid/manipulated route to convert X USDC into near-worthless XYZ in the account.

Preconditions / Assumptions

- (a). Safe account registered and holds tokenIn (e.g., USDC)
- (b). ApproveAndCallBatchPermission registered and config current for the account
- (c). spenders include Uniswap V3 SwapRouter/SwapRouter02
- (d). consumingPairs include (router, exactInputSingle)
- (e). requireRecipientIsAccount=true (recipient pinned to account)
- (f). tokens include tokenIn with sufficient maxApprovalAmount
- (g). Pre-batch allowance on (tokenIn, router) is zero
- (h). An illiquid or manipulable tokenOut route exists (or



can be selected)

(i). Manager key is adversarial/compromised and signs the batch

Octane Security



Vulnerability #3

Missing self-pair guard and endpoint-only checks in SwapPermission (V2 path) cause bounded value burn via self-route swaps

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Resolved

SwapPermission now rejects same-token routes: `tokenIn == tokenOut` is denied on the V3 branches, and `path[0] == path[last]` is denied on the V2 path (the load-bearing guard, since a same-token V3 `exactInputSingle` reverts at the router regardless). A same-token round-trip is value-destroying by definition and has no legitimate use, so denying it removes no real capability. `SwapPermissionNoOracle` was already immune (its coverage requirement skips self-pairs). Fixed and merged.

SwapPermission does not reject `tokenIn == tokenOut` and checks only endpoints on the Uniswap V2 path. If the configured oracle adapter returns a non-zero, fresh price for `base==quote`, a manager can execute a single-call round-trip (e.g., `A->B->A`) that passes the oracle floor and burns AMM fees/price impact while ending with the same token. Loss is bounded per call by the configured slippage and size cap and remains within the authorized envelope.

SwapPermission's `evaluate()` allows Uniswap V2 `swapExactTokensForTokens` calls when: (1) the router is allowlisted, (2) `path[0]` `tokensIn`, (3) `path[last]` `tokensOut`, (4) `recipient == account`, (5) `amountIn ≤ maxAmountPerTx`, and (6) the oracle floor passes. There is no guard that `tokenIn != tokenOut`. On the V2 path, identical endpoints (`A->...->A`) are syntactically valid and intermediate hops are intentionally unconstrained. `_oracleCheck` consumes `IOracle.getPrice(tokenIn, tokenOut)` and denies only if `price==0`, `decimals>77`, `updatedAt` is stale/0, or the computed floor truncates to zero. The oracle-adapter spec does not forbid `base==quote` pricing; many adapters will return a 1:1 price with a fresh `updatedAt`. Therefore, if the adapter returns a positive, fresh `base==quote` price and the route's output meets `amountOutMin` at or above the oracle-implied floor, a single V2 call with path `[A, B, A]` is allowed. The result is a bounded decrease in the account's A due to AMM fees/price impact, with output pinned to the account. This is not an authorization bypass: it remains within the template's documented envelope (allowlisted router, endpoint-only allowlists, recipient pin, per-call cap, oracle floor). The Uniswap V3 `exactInputSingle` path is not exploitable for self-routes because `tokenIn == tokenOut` reverts at the router, and the template does not support V3 `exactInput` (bytes path).

file: contracts/templates/SwapPermission.sol

```

203 (address, address, uint24, address, uint256,
    uint256, uint256, uint160)
204 );

205 if (!isAllowedTokenIn[ctx.account][tokenIn]) return
    false;
206 if (!isAllowedTokenOut[ctx.account][tokenOut]) return
    false;

...

223 (address, address, uint24, address, uint256,
    uint256, uint160)
224 );

225 if (!isAllowedTokenIn[ctx.account][tokenIn]) return
    false;
226 if (!isAllowedTokenOut[ctx.account][tokenOut]) return
    false;

...

239 ) = abi.decode(txData[4:], (uint256, uint256, address[],
    address, uint256));
240 if (path.length < 2) return
    false;

241 if (!isAllowedTokenIn[ctx.account][path[0]])-
    return false;
242 if (!isAllowedTokenOut[ctx.account][path[path.length
    - 1]]) return false;

```

```

203 (address, address, uint24, address, uint256,
    uint256, uint256, uint160)
204 );
205 + if (tokenIn == tokenOut) return
    false;
206 if (!isAllowedTokenIn[ctx.account][tokenIn]) return
    false;
207 if (!isAllowedTokenOut[ctx.account][tokenOut]) return
    false;

...

```

```

224 (address, address, uint24, address, uint256,
    uint256, uint160)
225 );
226 + if (tokenIn == tokenOut)                return
    false;
227   if (!isAllowedTokenIn[ctx.account][tokenIn]) return
    false;
228   if (!isAllowedTokenOut[ctx.account][tokenOut]) return
    false;

...

241 ) = abi.decode(txData[4:], (uint256, uint256, address[],
    address, uint256));
242 if (path.length < 2)                      return
    false;
243 + if (path[0] == path[path.length - 1])    re-
    turn false;
244   if (!isAllowedTokenIn[ctx.account][path[0]])-
    return false;
245   if (!isAllowedTokenOut[ctx.account][path[path.length
    - 1]]) return false;

```

Severity

Impact Explanation: [Low] Each allowed swap is an authorized, within-envelope call to an allowlisted router with output pinned to the account. Loss per call is bounded by the configured slippage and per-call cap and does not redirect value to an unauthorized recipient. Under the scope's anti-escalation guidance, this caps impact below High/Critical and fits Low.

Likelihood Explanation: [Medium] Viability depends on plausible but not universal conditions: a V2 router being allowlisted, overlapping endpoint allowlists, an adapter that returns a non-zero, fresh base==quote price, and sufficient liquidity to meet the oracle floor. These are significant but realistic constraints outside the attacker's complete control.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (Uniswap V2): Manager executes a single-call A->B->A self-route with amountOutMin set to the oracle floor (e.g., 1% band), passing endpoint allowlists and oracle(A,A) 1:1 pricing; the account ends with fewer A due to AMM fees/price impact.

Preconditions / Assumptions

- (a). Account registered with SailKernel; SwapPermission registered and configured; config epoch current.
- (b). Allowlisted router is a Uniswap V2-like router supporting swapExactTokensForTokens.
- (c). tokensIn includes A; tokensOut includes A (overlap).
- (d). Oracle adapter returns a non-zero, fresh price for getPrice(A, A) with decimals ≤ 77 ; maxPriceAgeSec > 0 .
- (e). Pools along [A, B, A] have sufficient liquidity to return $\geq$ amountOutMin (at or above the oracle floor).
- (f). maxAmountPerTx $\geq$ chosen amountIn; ctx.value == 0; recipient equals account.

Scenario 2.



Scenario 2 (Uniswap V2, tokensOut restricted to A): Even if tokensOut contains only A, the manager uses a self-route A->B->A. Endpoint checks pass (A endpoints only), oracle(A,A) 1:1 pricing passes, and the account's A is degraded while still ending in A.

Preconditions / Assumptions

- (a). Account registered with SailKernel; SwapPermission registered and configured; config epoch current.
- (b). Allowlisted router is a Uniswap V2-like router supporting swapExactTokensForTokens.
- (c). tokensIn includes A; tokensOut contains only A (no B).
- (d). Oracle adapter returns a non-zero, fresh price for getPrice(A, A) with decimals ≤ 77 ; maxPriceAgeSec > 0 .
- (e). Pools along [A, B, A] have sufficient liquidity to return $\geq$ amountOutMin (at or above the oracle floor).
- (f). maxAmountPerTx $\geq$ chosen amountIn; ctx.value == 0; recipient equals account.

Scenario 3.

Scenario 3 (Uniswap V2, extreme slippage): With maxSlippageBps near 9,999 and amount sized so oracleMinOut > 0 , the manager executes A->B->A producing output just above the minimal floor, causing a near-total single-call loss while still within configured bounds.

Preconditions / Assumptions

- (a). Account registered with SailKernel; SwapPermission registered and configured; config epoch current.
- (b). Allowlisted router is a Uniswap V2-like router supporting swapExactTokensForTokens.
- (c). tokensIn includes A; tokensOut includes A; maxSlippageBps configured very high (e.g., 9,999).
- (d). Oracle adapter returns a non-zero, fresh price for getPrice(A, A) with decimals ≤ 77 ; maxPriceAgeSec > 0 .
- (e). AmountIn chosen such that oracleMinOut > 0 ; route [A, B, A] yields $\geq$ the minimal oracle floor.
- (f). maxAmountPerTx $\geq$ chosen amountIn; ctx.value == 0; recipient equals account.

Vulnerability #4

Per-call LTV check without existing-debt accounting in BorrowPermission causes staircase borrowing beyond cumulative LTV ceiling

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

BorrowPermission enforces LTV per borrow at borrow time, not cumulatively, and does not subtract existing debt – so staircased borrows can exceed an intended aggregate LTV. This is a documented, by-design boundary of an oracle-free template: cumulative safety is delegated to the lending venue's own health-factor enforcement, and value never leaves the account. Adding cumulative-LTV accounting would require a debt-oracle subsystem we are deliberately not adding. Documented as a per-transaction (not cumulative) boundary.

BorrowPermission enforces maxLtvBps only per transaction using current collateral and borrow prices, without subtracting existing debt or reading venue state. This allows a manager to staircase multiple borrows, each individually within the per-call ceiling, to exceed any intended cumulative LTV target. With zero oracles configured, only the per-tx cap applies, further enabling repeated borrows. The impact is elevated liquidation risk to the account; no unauthorized value leaves the beneficiary set.

BorrowPermission.evaluate authorizes Aave/Morpho/Compound borrows after checking protocol/asset allowlists, amount $\leq$ maxAmountPerTx, and pinning credit to the account (onBehalfOf/receiver == account). It then calls `_ltvCheck`, which derives a per-call maxAmountAllowed from two oracles: a collateral oracle queried as `getPrice(account, address(0))` and a borrow-asset oracle queried as `getPrice(asset, address(0))`. Freshness, nonzero values, and decimals ≤ 77 are enforced; computation floors (fail-closed). Critically, `_ltvCheck` never reads existing debt from the lending venue and never subtracts prior borrow exposure; it compares only the current requested amount to a per-call bound derived from the current collateral valuation and borrow price. When either oracle is unset, `_ltvCheck` returns true (amount-cap-only mode, no LTV ceiling). As a result, a manager can staircase borrows—each individually $\leq$ per-call bound—until the venue's health-factor limits stop further borrowing. Pairing with DepositPermission, the manager can also loop borrow→deposit to raise the collateral oracle's reported value, increasing the next per-call bound and pushing leverage higher. This behavior is explicitly documented in the template's honest boundaries (per-transaction cap, not cumulative; leverage loops not bounded). Impact is elevated liquidation risk to the account; value does not leave the beneficiary set via an unauthorized permissioned action. Operators can mitigate by using a debt-aware collateral adapter (net headroom), setting a lower maxAmountPerTx,

or adding a monitoring permission to block further borrows above a chosen cumulative threshold.

file: contracts/templates/BorrowPermission.sol

```
212 // When the oracles are set, a freshness bound is mandatory;  
    0 would silently accept  
213 // arbitrarily stale prices and re-open the gap the oracle  
    is meant to close.  
  
214 if (colSet && maxPriceAgeSec == 0) revert MissingPriceAge();  
  
...  
  
286 function _ltvCheck(Slot storage s, address asset, uint256  
    amount, address account)  
  
287 internal  
288 view
```

```
212 // When the oracles are set, a freshness bound is mandatory;  
    0 would silently accept  
213 // arbitrarily stale prices and re-open the gap the oracle  
    is meant to close.  
214 +// TODO(cumulative-LTV): introduce a third oracle  
    `debtOracle` and require all three  
215 +// (collateral, borrow, debt) to be set for strict  
    cumulative-LTV enforcement.  
216 if (colSet && maxPriceAgeSec == 0) revert MissingPriceAge();  
  
...  
  
288 function _ltvCheck(Slot storage s, address asset, uint256  
    amount, address account)  
289 +/* FIXME(cumulative-LTV):  
290 +* - Current logic enforces per-transaction LTV only (does  
    not subtract existing debt).
```

```
291 +* - Add Slot.debtOracle and in this function read
    `debtValue` (same quote/decimals as collateral),
292 +*   apply freshness; require `debtDec == colDec`, then:
293 +*   allowedValue = floor(colValue * maxLtvBps / 10_000);
    if (debtValue >= allowedValue) return false;
294 +*   headroomValue = allowedValue - debtValue;
    maxAmountAllowed = floor(headroomValue * 10^borDec / (10^colDec)
    / borPrice);
295 +*   and require amount <= maxAmountAllowed. Keep fail-closed
    and within PERMISSION_GAS_CAP. */
296   internal
297   view
```


Severity

Impact Explanation: [Low] No direct unauthorized outflow occurs and no core invariant is bypassed; the effect is elevated liquidation risk within the account's own positions. This matches the documented by-design tradeoff (per-transaction, not cumulative LTV) and falls under policy under-enforcement with available operator mitigations.

Likelihood Explanation: [Medium] Staircase borrowing and leverage loops are feasible under normal operation with standard configurations. However, actual loss requires subsequent adverse market movement and depends on operator configuration (oracle choice, caps), so likelihood is not highest.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (staircase without deposits): A registered manager repeatedly calls Aave/Morpho/Compound borrow through BorrowPermission. Each call passes `_ltvCheck` because it uses only current collateral valuation and the requested amount, not existing debt. The manager repeats per-call-sized borrows until the venue's health factor stops further borrowing, exceeding any intended cumulative LTV target.

Preconditions / Assumptions

- (a). Account is registered; BorrowPermission is registered and configured
- (b). Protocol and asset allowlists include the chosen venue and asset
- (c). `maxAmountPerTx` is not the binding limit relative to the per-call LTV-derived bound
- (d). Both oracles configured and fresh; collateral oracle reports aggregate collateral (not net of debt)
- (e). Manager can submit repeated borrow calls; realized loss requires adverse market movement for liquidation

Scenario 2.



Scenario 2 (leverage loop with deposits): The manager alternates BorrowPermission (borrow up to current per-call bound) and DepositPermission (supply the borrowed asset back on behalf of the account). The collateral oracle then reports higher collateral, lifting the next per-call borrow bound. Repeating this loop ratchets leverage upward to the venue's health-factor limit.

Preconditions / Assumptions

- (a). Account is registered; BorrowPermission and DepositPermission are both registered and configured
- (b). Borrow/Deposit targets and tokens are allowlisted; receivers/onBehalfOf pinned to the account
- (c). Both oracles configured and fresh; collateral oracle increases reported value after deposits
- (d). Manager performs repeated borrow→deposit sequences; realized loss requires adverse market movement

Scenario 3.

Scenario 3 (zero-oracle mode): BorrowPermission is configured with both oracles unset (amount-cap-only). `_ltvCheck` returns true without an LTV ceiling, so the manager repeatedly borrows up to `maxAmountPerTx` until the venue's health factor prevents more.

Preconditions / Assumptions

- (a). Account is registered; BorrowPermission is registered with both collateral and borrow oracles unset
- (b). Protocol and asset allowlists include the venue and asset; nonzero `maxAmountPerTx`
- (c). Manager performs repeated borrow calls up to `maxAmountPerTx`; realized loss requires adverse market movement

Vulnerability #5

Single raw perTx cap across heterogeneous units in DepositPermission causes larger than intended deposits (intra account)

Severity: LOW

Likelihood: HIGH

Impact: LOW

Status: Acknowledged

DepositPermission applies one raw maxAmountPerTx uniformly across tokens of differing decimals, and the ERC-4626 mint cap is shares-denominated – so a cap sized for one asset under-constrains another. This is a documented oracle-free tradeoff; all value stays in the account (no theft, no cross-account effect). Operators size the cap per the lowest-decimal asset or use separate instances. Documented rather than adding per-token caps or decimals normalization.

DepositPermission applies one maxAmountPerTx uniformly to ERC4626 deposit/assets, ERC4626 mint/shares, and Aave v2/v3 deposit/supply(amount) without decimals normalization or per token/per vault caps. Mixing assets with different decimals, or using mint (sharesdenominated), can allow a manager to deposit more underlying per transaction than the operator intended. Recipients are pinned to the account, so value stays within the beneficiary set; the effect is policy underenforcement rather than theft.

Per account, DepositPermission stores Slot{targets[], tokens[], maxAmountPerTx} and enforces a single raw integer cap against heterogeneous deposit shapes: (1) ERC4626 deposit(uint256 assets, address) caps underlying asset units; (2) ERC4626 mint(uint256 shares, address) caps shares (not assets); and (3) Aave v2/v3 deposit/supply(address asset, uint256 amount, address onBehalfOf, uint16) caps the asset's raw units. There is no decimals normalization and no per token/per vault cap map. For ERC4626 paths, the vault address must be in the token allowlist; for Aave, the ERC20 asset is allowlisted. evaluate() approves when amount/shares ≤ maxAmountPerTx and receiver/onBehalfOf == account. As a result, if the configuration mixes tokens with different decimals or uses mint (shares) under the same cap, a cap sized for one asset or unit type can implicitly authorize far larger underlying perTx deposits on another path. This is a documented design choice (sharesdenominated cap keeps the template oraclefree). All deposits credit the Safe itself; no value leaves the beneficiary set. The risk is policy underenforcement that can be corrected by splitting configurations (multiple instances) or sizing per instance caps appropriately.

file: contracts/templates/DepositPermission.sol

```

87     address[] tokens;
88     uint256    maxAmountPerTx;

89 }

...

117 (address[] memory targets, address[] memory tokens, uint256
118 maxAmountPerTx) =
119     abi.decode(params, (address[], address[], uint256));

119
120 // Config validation (reference-grade): bound both
    allowlists, reject empty arrays,

...

158 // mint(uint256 shares, address receiver) – ERC-4626
159 if (ctx.selector == MINT) {

160     if (txData.length < LEN_2ARG) return false;
161     (uint256 shares, address receiver) = abi.decode(txData[4:], (uint256, address));

...

171 // deposit(address asset, uint256 amount, address
    onBehalfOf, uint16) – Aave v2
172 if (ctx.selector == DEPOSIT_AAVE) {

173     if (txData.length < LEN_4ARG) return false;
174     (address asset, uint256 amount, address onBehalfOf,) =

...

181 // supply(address asset, uint256 amount, address onBehalfOf,
    uint16) – Aave v3
182 if (ctx.selector == SUPPLY_AAVE) {

183     if (txData.length < LEN_4ARG) return false;
184     (address asset, uint256 amount, address onBehalfOf,) =

```

```

87     address[] tokens;
88     uint256    maxAmountPerTx;
89 +// FIX: Add flags to mitigate unit heterogeneity and shares
    path ambiguity:
90 +// bool allowMint;                // when false, deny
    ERC-4626 mint selector
91 +// bool enforceUniformDecimals; // when true, enforce
    consistent decimals at configure-time
92     }

...

120 (address[] memory targets, address[] memory tokens, uint256
    maxAmountPerTx) =
121     abi.decode(params, (address[], address[], uint256));
122 +// FIX: Extend params to include allowMint and
    enforceUniformDecimals flags;
123 +// when enforceUniformDecimals is true, at configure-time
    read decimals for each allowlisted
124 +// asset (and vault underlying/share for ERC-4626) and
    revert on mixed scales.
125
126 // Config validation (reference-grade): bound both
    allowlists, reject empty arrays,

...

164 // mint(uint256 shares, address receiver) – ERC-4626
165 if (ctx.selector == MINT) {
166 +// FIX: Once stored, gate mint: if (!s.allowMint)
    return false;
167 if (txData.length < LEN_2ARG) return false;
168 (uint256 shares, address receiver) = abi.decode(txData[4:], (uint256, address));

...

178 // deposit(address asset, uint256 amount, address
    onBehalfOf, uint16) – Aave v2
179 if (ctx.selector == DEPOSIT_AAVE) {
180 +// FIX: If enforceUniformDecimals is enabled, ensure
    configured decimals match (or normalize in a V2).
181 if (txData.length < LEN_4ARG) return false;
182 (address asset, uint256 amount, address onBehalfOf,) =

...

```

```
189 // supply(address asset, uint256 amount, address onBehalfOf,  
    uint16) – Aave v3  
190 if (ctx.selector == SUPPLY_AAVE) {  
191 +// FIX: If enforceUniformDecimals is enabled, ensure  
    configured decimals match (or normalize in a V2).  
192 if (txData.length < LEN_4ARG) return false;  
193 (address asset, uint256 amount, address onBehalfOf,) =
```

Severity

Impact Explanation: [Low] Policy underenforcement only: recipients are pinned to the Safe (intraenvelope), no theft or crossenvelope value movement, and the configuration can be tightened by splitting instances or adjusting perinstance caps. The sharesdenominated cap for mint is a documented bydesign tradeoff to keep the template oraclefree.

Likelihood Explanation: [High] Once configured, ERC4626 vaults commonly support mint and appreciate (share price > 1), and accounts often mix assets with different decimals. No rare external preconditions are required beyond balances/allowances; a manager can select the most permissive allowed path.

Exploitation

Exploitation Scenarios:

Scenario 1.

ERC4626 mint(shares) exceeds an assetdenominated intent when share price > 1: A vault V is allowlisted as target and token; maxAmountPerTx = S. Manager calls mint(S, account). evaluate() caps shares (S) not assets, so the call passes. The vault mints S shares and pulls convertToAssets(S) = S*P underlying (P>1), exceeding the Sasset intent for a deposit(assets) path. Proceeds remain in the account.

Preconditions / Assumptions

- (a). Account registered; DepositPermission configured and current (config epoch matches).
- (b). Vault V is allowlisted as target and token (ERC4626 paths).
- (c). Manager is potentially adversarial within bounds.
- (d). Safe holds sufficient underlying and has allowance to V.
- (e). Vault supports mint and share price P>1.

Scenario 2.

Mixeddecimal Aave supply under a single cap: targets include an Aave pool; tokens include WETH (18 decimals) and USDC (6 decimals); maxAmountPerTx = 1e18 (chosen for



WETH). Manager supplies USDC with amount = min(balance, 1e18). Because the cap is raw units, evaluate() allows up to 1e18 USDC units (far beyond 1 whole USDC). aUSDC credits the account; exposure per tx is larger than intended.

Preconditions / Assumptions

- (a). Account registered; DepositPermission configured and current.
- (b). Aave v2/v3 pool is allowlisted as target; USDC is allowlisted as token.
- (c). Manager is potentially adversarial within bounds.
- (d). Safe has substantial USDC balance and allowance.
- (e). Single shared cap sized for 18dec assets is used across mixeddecimal tokens.

Scenario 3.

ERC4626 shareddecimals vs 18dec cap: a vault with 6decimal shares is allowlisted alongside 18dec contexts; cap set to 1e18. Manager calls mint(1e18, account). evaluate() treats 1e18 as shares (no normalization). Depending on vault math/balances/price, this can translate into a very large underlying transfer in a single tx. All value is credited to the account.

Preconditions / Assumptions

- (a). Account registered; DepositPermission configured and current.
- (b). ERC4626 vault with share decimals differing from other assets is allowlisted (as target and token).
- (c). Manager is potentially adversarial within bounds.
- (d). Safe holds sufficient underlying and has allowance.
- (e). Single shared cap set assuming 18dec contexts; no decimals normalization in the template.

Vulnerability #6

Unbound registration signature and pre-registration configs in MandateFactory/SailKernel bundling cause front-run activation of older config or bundle revert

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

A mempool observer can front-run the kernel registration leg of a factory bundle using the revealed registration signature, activating an older same-epoch configuration or causing the bundle to revert. Every outcome is recoverable and stays within the configuration the permission signer already authorized: either a previously-signed same-epoch config activates, or the permission ends registered-but-unconfigured and fails closed until a single `configure()` is applied. The proposed remediation binds the configuration hash into the kernel's registration entrypoints and signing scheme – re-opening the frozen trusted core – and no clean factory-only mitigation exists, since the front-run is submitted directly to the kernel. Given the recoverable, in-envelope impact, we accept it and document that operators use `replacePermission` for atomic tightening and private orderflow for the bundle.

MandateFactory's `configure+register/replace` bundles do not cryptographically bind the kernel-side registration signature to the permission's configuration parameters. Because templates allow `configure()` pre-registration and SailKernel does not bump `registrationEpoch` on `register`, a mempool observer can front-run the `register/replace` using the leaked kernel signature. This can either activate an older same-epoch config or cause the factory bundle to revert (rolling back `configure`). Effects are operator-recoverable and stay within the previously authorized envelope.

MandateFactory.`attach/attachBatch/replace` orchestrate `template.configure(...)` followed by `SailKernel.registerPermission(...)` or `replacePermission(...)`, but each inner call is independently signature-authenticated. The kernel-side signatures (`REGISTER_PERMISSION` / `REPLACE_PERMISSION`) bind only account, permission(s), signer nonce, and deadline, not configuration params. `IConfigurablePermission.configure` can be called while the template is unregistered and stamps its own `configuredEpoch` using `kernel.registrationEpoch(account, permission)`. SailKernel deliberately does not bump `registrationEpoch` on `register` (only when a permission leaves the registry), and all reference templates gate `evaluate()` with `_configCurrent(account, ctxEpoch)` which passes if a same-epoch config exists. A mempool observer can front-run by calling `registerPermission/replacePermission` directly with the revealed `kernelSig`. If a same-epoch config already exists on the template, registration immediately activates the

old params; the victim's factory bundle then reverts at the register/replace step, rolling back the intended fresh configure. If no prior config exists, the attacker still causes the register/replace to happen first, the victim's bundle reverts, and the new permission remains unconfigured (evaluation fails closed) until the operator sends configure. The attacker must pay the registration fee and cannot dispatch (only the manager can). Any movements remain within the signer-approved envelope enforced by the templates. The behavior is documented (in-place configure is front-runnable; use replacePermission for atomic tightening) and yields recoverable, operator-level impact.

file: contracts/core/SailKernel.sol

```
287 mapping(address account => uint256) public signerNonces;
288

289 /// @notice Per-(account, permission) registration epoch. A
    plain monotonic counter, bumped
290 /// every time a permission LEAVES an account's registry
    (revoke, the removed side of a
```

```
287 mapping(address account => uint256) public signerNonces;
288
289 +/// FIX: Bind registration to a config hash to prevent front-run
    activation of stale same-epoch configs.
290 +/// - Add: mapping(address account => mapping(address permission
    => bytes32)) registrationConfigHash;
291 +/// - Add bound + idempotent kernel entrypoints (ensureRegis-
    trationBound / ensureReplacePermissionBound)
292 +/// that verify signatures including paramsHash, set/check
    registrationConfigHash, and refund on no-op.
293 +/// - Clear registrationConfigHash on revoke/clear; keep epoch
    semantics unchanged.
294 +
295 /// @notice Per-(account, permission) registration epoch. A
    plain monotonic counter, bumped
296 /// every time a permission LEAVES an account's registry
    (revoke, the removed side of a
```

Severity

Impact Explanation: [Low] Effects are limited to operator-level policy under-enforcement or bundling-path denial: either an older, signer-approved same-epoch config becomes active or the bundle reverts and the new configure is rolled back. No unauthorized party can dispatch, and all movements remain within the signer-approved envelope enforced by the templates. Recovery is a single authenticated configure (or using a private relay/replace flow).

Likelihood Explanation: [Medium] Exploitation requires a mempool race and a specific but plausible operational state: a prior same-epoch config (Scenario 1) or a replace flow in flight (Scenario 3). These are uncommon but realistic; attackers may also be the manager (rational) rather than a third party (griefing). Scenario 2 requires prior leakage of a valid configure signature and is rarer.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (early register activates older config): Account A is registered; template P is not registered on A but already has a same-epoch config (`isConfigured=true`; `configuredEpoch==registrationEpoch`). Operator submits `MandateFactory.attach(A, P, paramsNew, configureSigNew, kernelSig)`. A mempool observer reuses `kernelSig` to call `SailKernel.registerPermission(A, P, kernelSig)` first. Registration does not bump epoch, so P becomes active under the old config. The victim's attach then reverts at `registerPermission`, rolling back `configureNew`. The manager can use P under old bounds until the operator reconfigures in a separate call.

Preconditions / Assumptions

- (a). Account A is registered in `SailKernel`
- (b). Template P is not currently registered for A
- (c). P has a prior configuration for A stamped at the current `registrationEpoch` (same-epoch)
- (d). Operator prepared valid `configureSigNew` and `kernel-`



Sig for A and P

(e). Attacker can observe mempool and is willing to pay the registration fee

Scenario 2.

Scenario 2 (use previously obtained configureSigOld, then register): Same setup as Scenario 1, but the attacker already holds a valid same-epoch configureSigOld for the current config nonce. The attacker first calls P.configure(A, paramsOld, configureSigOld), then front-runs SailKernel.registerPermission(A, P, kernelSig). The victim's attach reverts at the register step, undoing its configureNew. P remains active under paramsOld.

Preconditions / Assumptions

(a). All preconditions from Scenario 1

(b). Attacker already possesses a valid same-epoch configureSigOld for P at the current config nonce and unexpired deadline

Scenario 3.

Scenario 3 (replace front-run causes temporary DoS): Account A has OldPerm registered. Operator submits MandateFactory.replace(A, OldPerm, NewPerm, newParams, configureSigNew, kernelReplaceSig). An observer reuses kernelReplaceSig to call SailKernel.replacePermission(A, OldPerm, NewPerm, kernelReplaceSig) first; OldPerm is removed (its epoch bumped), NewPerm is added but unconfigured. The victim's replace then reverts at the kernel step (state already changed), rolling back its configure. A temporarily has no working permission until the operator sends a standalone configure() for NewPerm or reruns a replacement flow.

Preconditions / Assumptions

(a). Account A currently has OldPerm registered

(b). Operator submits a replace bundle for NewPerm (bundle is visible in mempool)

(c). Attacker can observe kernelReplaceSig and is willing to pay the replacement fee

(d). NewPerm is not yet configured for A in the current epoch (or the configure in the bundle will be rolled back by the revert)



Vulnerability #7

Missing min-shares bound in ERC-4626 deposit authorization (DepositPermission/ApproveAndCallBatchPermission) enables negative-EV griefing to reduce minted shares to near zero

Severity: LOW

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

The ERC-4626 deposit/assets paths lack a min-shares guard, so a donation attack can mint near-zero shares to the depositor. This is negative-EV griefing – the attacker must donate more than the deposit's value – and it is account-pinned (no theft). The mint/shares path pins the share outcome and is the donation-safe alternative; both templates support it. Rather than remove the canonical deposit/assets entrypoint to defend a negative-EV griefing path, we document that mint/shares is the safe path and deposit/assets carries the donation risk.

The ERC-4626 deposit paths in DepositPermission and ApproveAndCallBatchPermission authorize deposits based only on allowlists, a raw per-tx asset cap, and receiver==account, without any min-shares or preview-based guard. An attacker can donate underlying to the vault before a permitted deposit so that, under OZ v4.9+ ERC-4626 math, the victim's deposit mints very few or even zero shares. This is a negative-EV griefing vector (not rationally profitable for the attacker) but can materially harm the victim; per-tx caps bound loss per call.

DepositPermission's ERC-4626 deposit(uint256 assets, address receiver) branch authorizes a call when: (i) ctx.value==0, (ii) the target vault is allowlisted, (iii) the vault address is on the token allowlist (since ERC-4626 deposit does not carry the underlying in calldata), (iv) amount <= maxAmountPerTx, and (v) receiver==account. There is no post-condition check on shares minted and no previewDeposit-based bound. ApproveAndCallBatchPermission similarly authorizes ERC-4626 deposit within its approve/consume/reset bracket, binding spender==target and the consumed asset, and optionally pinning receiver==account, but it also omits any min-shares check. The kernel treats Safe module success as sufficient and does not inspect return values from the vault. With OZ v4.9+ ERC-4626 (vendored), $\text{sharesOut} = \text{floor}(\text{assets} \times (S+1)/(A+1))$. An attacker can seed a small share position, donate D underlying to inflate A, then let/induce the manager to deposit X via the authorized path; the victim mints very small or even zero shares. Under v4.9+ math, forcing near-zero shares requires $D \leq S \times X$, which makes the attacker's net profit negative; thus, this is negative-EV griefing rather than rational theft. Nevertheless, the victim's deposit can be materially harmed. The per-transaction cap bounds loss per call. Operators can mitigate by using the mint/shares path (already supported) or by wrapping deposit with a min-shares preview.

file: contracts/templates/DepositPermission.sol

```
148 // deposit(uint256 assets, address receiver) – ERC-4626
149 if (ctx.selector == DEPOSIT_SIMPLE) {
150 - if (txData.length < LEN_2ARG) return false;
151 - (uint256 amount, address receiver) = abi.decode(txData[4:], (uint256, address));
152 - // Asset is not in calldata; require the vault itself
    to be token-allowlisted.
153 - if (!isAllowedToken[ctx.account][ctx.target]) return
    false;
154 - if (amount > s.maxAmountPerTx) return false;
155 - return receiver == ctx.account;
156 }
```

```
148 // deposit(uint256 assets, address receiver) – ERC-4626
149 if (ctx.selector == DEPOSIT_SIMPLE) {
150 + // ERC-4626 deposit(assets) disabled: use mint(shares)
    to pin outcome and avoid donation/griefing vectors.
151 + return false;

152 }
```

Severity

Impact Explanation: [High] From the victim's perspective, a permitted deposit of X assets can mint very few or zero shares when an attacker donates beforehand, producing a direct, material loss of principal value on that operation; however, this stems from a by-design omission (no post-condition/min-shares) in an oracle-free template.

Likelihood Explanation: [Low] To suppress shares materially under OZ v4.9+ math, the attacker must donate $D \geq S \times X$, making their net profit negative; it also requires timing relative to a manager-triggered deposit. This is negative-EV griefing with additional timing and capital constraints.

Exploitation

Exploitation Scenarios:

Scenario 1.

Modern ERC-4626 griefing: (1) An account registers `DepositPermission` (or `ApproveAndCallBatchPermission`) for a specific ERC-4626 vault V , with V allowlisted as target and as the 'token' on the ERC-4626 path; per-tx cap $\geq X$; the account `Safe` holds X units of underlying U . (2) An attacker seeds a small number of shares μ in V , then donates D units of U directly to V , inflating `totalAssets` without minting shares. (3) The manager dispatches `deposit(X, receiver=account)` through the authorized permission. The permission approves based on allowlists/cap/receiver; no min-shares is enforced. (4) The vault mints $\text{sharesOut} = \text{floor}(X \times (\mu + 1) / (\mu + D + 1))$; for large D this can be very small or even 0. (5) Under OZ v4.9+ math, the attacker's net profit is negative for D large enough to suppress `sharesOut` toward 0, so this is a negative-EV griefing attack that can still materially harm the victim's deposit outcome.

Preconditions / Assumptions

(a). Account registered `DepositPermission` or `ApproveAndCallBatchPermission` for an ERC-4626 vault V ; `isAllowedTargetaccount=true` and `isAllowedTokenaccount=true` (vault address for ERC-4626 path).



- (b). Per-transaction cap `maxAmountPerTx` for this account in the permission config is $\geq X$ (the deposit amount).
- (c). The account's Safe holds at least X units of the vault's underlying token U and, for the single-call `DepositPermission` path, has an allowance set for V (or uses the batch permission bracket to approve temporarily).
- (d). Vault V implements OZ v4.9+ ERC-4626 semantics with virtual shares/assets ($\text{sharesOut} = \text{floor}(\text{assets} \times (S+1)/(A+1))$).
- (e). An attacker can donate D units of U to V and time this relative to the manager's deposit.
- (f). The manager (potentially adversarial within permissions) triggers `deposit(X, receiver=account)` through the kernel.

Vulnerability #8

Double-rounding in V3 expectedOut computation in SwapPermission-NoOracle loosens tolerance floor, allowing lower amountOutMin

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Resolved

The V3 reference projection in SwapPermissionNoOracle now rounds up (`Math.Rounding.Ceil`) instead of flooring twice, so the tolerance floor can only tighten or stay equal, never loosen. This removes the additive slack that previously allowed a marginally lower `amountOutMin` to pass. The existing zero-output and zero-floor guards are unaffected. Fixed and merged.

In SwapPermissionNoOracle's Uniswap V3 reference-pool branch, `expectedOut` is computed by applying `sqrtPriceX96/Q96` twice with two `Math.mulDiv` calls, each flooring the result. This double-flooring underestimates the spot-implied output by an additive amount in raw `tokenOut` units, so the tolerance-adjusted floor (`poolFloor`) is looser than it would be under a single full-precision evaluation. Managers can set a lower `amountOutMin` that still passes the on-chain check. The effect is self-directed (within the account's beneficiary set) and bounded; V2 path is unaffected.

SwapPermissionNoOracle enforces a 'sanity band' by comparing the caller's `amountOutMin` against a tolerance-adjusted output implied by an operator-named reference pool. In the V3 branch of `_readExpectedOut`, the code computes `expectedOut` via two sequential `Math.mulDiv` operations using `sqrtPriceX96` and `Q96`: for `token0→token1`, `tmp = floor(amountIn * sqrtP / Q96)`, then `expectedOut = floor(tmp * sqrtP / Q96)`; for `token1→token0`, `tmp = floor(amountIn * Q96 / sqrtP)`, then `expectedOut = floor(tmp * Q96 / sqrtP)`. This is equivalent to `floor(floor(amountIn * r) * r)` or `floor(floor(amountIn / r) / r)`, where $r = \text{sqrtPriceX96} / 2^{96} = \text{sqrt}(\text{priceRaw})$. Sequential flooring introduces an additive underestimation relative to a single full-precision evaluation of $\text{amountIn} * r^2$ (or $/ r^2$). The additive error is bounded by roughly $r + 1$ raw units for `token0→token1` and $1/r + 1$ for `token1→token0`. `_sanityCheck` then computes `poolFloor = floor(expectedOut * (1 - toleranceBps/10_000))`. Because `expectedOut` is underestimated, `poolFloor` is additively looser than a full-precision tolerance floor, permitting a lower `amountOutMin` to pass. Guards deny if `expectedOut == 0` or if the adjusted floor would round to 0, preventing degenerate acceptance, but they do not remove the additive slack. The V2 branch uses a single `mulDiv` and is unaffected. All other structural checks (router/tokens allowlists, recipient pinned to the account, per-tx cap, epoch freshness, `ctx.value == 0`) remain enforced. Practically, for common 18/6/8-decimal tokens, the additive slack is a handful of raw units (wei or micro-units), often collapsing to zero.



after tolerance and flooring (Informational-level effect). The largest absolute differences occur with uncommon 0-decimal pairs and large price ratios, where the looser floor can be several whole units; still, the effect is self-directed within the beneficiary set and does not enable value to leave the account. Mitigations include computing the V3 projection in a single full-precision step (e.g., squaring sqrtP with 512-bit intermediates) or biasing the first step to round up to avoid looser acceptance at the cost of occasional false rejects.

file: contracts/templates/SwapPermissionNoOracle.sol

```
395  if (rp.tokenInIsToken0) {
396  // price = (sqrtP / 2^96)^2 token1 per
    token0; out = in * price
397 - uint256 tmp = Math.mulDiv(amountIn, sqrtP,
    Q96);
398 - return (true, Math.mulDiv(tmp, sqrtP,
    Q96));
399  } else {
400  // out = in / price
401 - uint256 tmp = Math.mulDiv(amountIn, Q96,
    sqrtP);
402 - return (true, Math.mulDiv(tmp, Q96, sqrtP));
403  }
404  } catch {
```

```
395  if (rp.tokenInIsToken0) {
396  // price = (sqrtP / 2^96)^2 token1 per
    token0; out = in * price
397 + uint256 tmp = Math.mulDiv(amountIn, sqrtP,
    Q96, Math.Rounding.Ceil);
398 + return (true, Math.mulDiv(tmp, sqrtP, Q96,
    Math.Rounding.Ceil));
399  } else {
400  // out = in / price
401 + uint256 tmp = Math.mulDiv(amountIn, Q96,
    sqrtP, Math.Rounding.Ceil);
402 + return (true, Math.mulDiv(tmp, Q96, sqrtP,
    Math.Rounding.Ceil));
403  }
404  } catch {
```

Severity

Impact Explanation: [Low] Policy under-enforcement on the sanity band allows a lower amountOutMin than a full-precision tolerance floor. The effect is self-directed within the account beneficiary set, moves no value to external parties, and does not break core invariants. Under uncommon 0-decimal conditions the difference is several whole units (not dust); under typical decimals it is dust-level.

Likelihood Explanation: [Low] Largest absolute impact requires uncommon preconditions (0-decimal tokens on a V3 pool plus large price ratio); the behavior is present for common decimals but typically reduces to dust-level differences. All other checks (allowlists, recipient pin, per-tx cap, epoch freshness) must also hold.

Exploitation

Exploitation Scenarios:

Scenario 1.

Uncommon 0-decimal tokens with high human price on a V3 reference pool: tokenIn→tokenOut, priceRaw ≈ 2000 , $r \approx 44.72$, tolerance 50%. For amountIn = 1, code computes expectedOut = floor(floor(1 * r) * r) = 1967, while full-precision is 2000. poolFloor_code = floor(1967 * 0.5) = 983 vs poolFloor_true = 1000. Manager sets amountOutMin = 983; the template approves though a full-precision 50% band would require at least 1000.

Preconditions / Assumptions

- (a). Account is registered; session active; SwapPermission-NoOracle configured with V3 reference pool for (tokenIn, tokenOut)
- (b). Router, tokenIn, tokenOut are allowlisted; recipient pinned to the account; ctx.value == 0
- (c). tokenIn and tokenOut both have 0 decimals (uncommon) and the V3 pool is live and liquid (slot0.sqrtPriceX96 != 0, liquidity() != 0)
- (d). High human price ratio (e.g., ≈ 2000) and toleranceBps up to 5000 (50%)
- (e). Manager selects amountOutMin that equals the



code-computed tolerance floor but is below the full-precision floor

Scenario 2.

Equal-decimal pair on V3 (e.g., 18/18) at high price: tokenIn→tokenOut, priceRaw ≈ 2000 , $r \approx 44.72$, tolerance 50%. For realistic amounts, additive slack is at most a few raw units (wei). poolFloor_code may be a few wei below poolFloor_true, allowing a slightly lower amountOutMin that still passes. Economically negligible but mechanically valid.

Preconditions / Assumptions

- (a). Account is registered; session active; V3 reference pool configured and liquid
- (b). Router, tokenIn, tokenOut are allowlisted; recipient pinned; ctx.value == 0
- (c). Equal or typical decimals (e.g., 18/18 or 18/6), high human price
- (d). Manager sets amountOutMin marginally below the full-precision tolerance floor by a few raw units

Scenario 3.

Operational compounding with many tiny V3 swaps (under the 0-decimal, high-price setup): splitting a larger trade into many 1-unit swaps repeats the per-op additive slack (e.g., accepting 983 where a full-precision 50% floor would be 1000). Each individual swap passes for the same reason, but linear aggregation does not raise severity.

Preconditions / Assumptions

- (a). Same preconditions as Scenario 1 (0-decimal, high-price V3 setup)
- (b). Manager deliberately splits a larger trade into many small V3 swaps to realize the per-op additive slack repeatedly

Vulnerability #9

Unbindable ERC-4626 mint amount in ApproveAndCallBatchPermission (requireAmountMatch=false) causes bounded self-directed overspend on price drift

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Acknowledged

With requireAmountMatch enabled, ERC-4626 mint is already denied (the consumed amount is not decodable), so mint is usable only with amount-match off, where the underlying pulled equals previewMint(shares) and any drift is bounded by the approved amount and pinned to the account. The proposed previewMint binding would relax this currently fail-closed state and import an operator-supplied vault's view-call gas into evaluation, to address a bounded, account-internal, informational issue. We decline the change; the existing fail-closed behavior under amount-match is the safer posture.

When using ERC-4626 mint(shares, receiver) through ApproveAndCallBatchPermission, the template cannot bind the consumed assets to the approved amount; with requireAmountMatch disabled (mandatory for mint), a rise in previewMint(shares) between signing and execution can consume more underlying than expected, but never more than the approved allowance and per-token cap, and shares are minted to the account.

ApproveAndCallBatchPermission evaluateBatch enforces a strict approve->consume->reset bracket. For ERC-4626 mint(shares,receiver): (1) _decodeConsumedAmount returns decodable=false, so with requireAmountMatch=true the batch is denied; operators must set requireAmountMatch=false to use mint. (2) With amount-match disabled, the template still binds the consumed asset by reading target.asset() (the vault's underlying), requires c1.target==spender from c0, ensures the pre-batch allowance on (token,spender) is zero, and requires c2 to reset that same allowance to zero. The actual assets pulled are determined at execution by previewMint(shares), which may increase due to price/state changes between signing and inclusion. Consequently, the vault can pull more underlying than the signer initially expected, but only up to approveAmount (bounded by maxApprovalAmounts[token]). If previewMint exceeds approveAmount, mint reverts and the entire batch reverts atomically. When requireRecipientIsAccount=true, minted shares are pinned to the account, so no value leaves the beneficiary set.

file: contracts/templates/ApproveAndCallBatchPermission.sol

```

359   if (_cfg[account].requireAmountMatch) {
360 -// The "consumed amount" lives at a different calldata
    word per selector, so it must be
361 -// decoded selector-aware (a fixed word-0 read
    would compare the approved amount against a
362 -// token ADDRESS for the V3/Aave shapes, or against
    a SHARE count for ERC-4626 mint).
363 -// A selector whose consumed amount cannot be
    located in calldata (e.g. mint, where the
364 -// pulled assets are previewMint(shares),
    off-chain) is reported not-decodable 'deny.
365 -(bool amtDecodable, uint256 consumedAmount) =
    _decodeConsumedAmount(sel, c1.data);
366 - if (!amtDecodable) return false;
367 - if (consumedAmount != approveAmount) return false;

368 }

```

```

359   if (_cfg[account].requireAmountMatch) {
360 + if (sel == ERC4626_MINT) {
361 +// For ERC-4626 mint(shares,receiver), bind the
    consumed assets via previewMint(shares).
362 + uint256 shares = uint256(bytes32(c1.data[4:36]));
363 +(bool ok, bytes memory ret) = c1.target.staticcall(
364 + abi.encodeWithSignature("previewMint(uint256)",
    shares)
365 +);
366 + if (!ok || ret.length < 32 || abi.decode(ret,
    (uint256)) != approveAmount) return false;
367 +} else {
368 +(bool amtDecodable, uint256 consumedAmount) =
    _decodeConsumedAmount(sel, c1.data);
369 + if (!amtDecodable || consumedAmount != approveAmount)
    return false;
370 +}
371 }

```

Severity

Impact Explanation: [Informational] Value remains within the account beneficiary set (shares minted to the account); no external theft or broken invariant. Assets consumed are capped by approveAmount and per-token cap; batch reverts if previewMint exceeds the bracket.

Likelihood Explanation: [Low] Requires an intentional configuration (requireAmountMatch=false) to use mint and operator-chosen headroom in approveAmount; any adversary gains nothing if recipient is pinned. Material variance further depends on price drift at execution time.

Exploitation

Exploitation Scenarios:

Scenario 1.

Operator configures the template for vault V (underlying U) with consuming pair (V, ERC-4626 mint), requireAmountMatch=false, and requireRecipientIsAccount=true. The manager signs a batch: (1) approve(U->V, approveAmount) set to expected previewMint(shares) plus small headroom; (2) V.mint(shares, receiver=account); (3) approve(U->V, 0). Before inclusion, V's assets/share increases, so previewMint(shares) rises. At execution, V pulls more U via transferFrom(account,V,previewMint(shares)) but not more than approveAmount, mints shares to the account, then the allowance is reset to zero. The account spends more underlying than expected but within the approved bracket; no external value loss occurs.

Preconditions / Assumptions

- (a). ApproveAndCallBatchPermission is registered and configured for the account with: tokens[] including U (vault underlying), spenders[] including V (ERC-4626 vault), consumingPairs[] including (V, ERC4626_MINT)
- (b). requireAmountMatch is false (required for mint in this template)
- (c). requireRecipientIsAccount is true (to pin minted shares to the account)
- (d). maxApprovalAmounts[U] set; approveAmount chosen <=



- cap; pre-batch allowance for (U,V) is zero
- (e). ERC-20 standard behavior (no FoT/rebasing; approve(0) succeeds) per scope assumptions
 - (f). Normal vault share price can drift upward between signing and execution (or be influenced by market activity/MEV)
 - (g). Manager signs a valid batch and submits via SailKernel.dispatchBatch with a valid deadline and nonce

Octane Security

Vulnerability #10

Missing code-existence and post-call validation in `MandateFactory.reconfigure` causes spoofable Reconfigured events with no state change

Severity: INFORMATIONAL

Likelihood: HIGH

Impact: INFORMATIONAL

Status: Resolved

`reconfigure` now reverts on a zero-address template, a code-less template address, and a template not registered for the account, before performing the configuration call. This prevents emitting a Reconfigured event for a no-op call to a code-less address, so the event can be trusted by indexers and tooling. The factory holds no protocol privileges and no funds path changed. Fixed and merged.

`MandateFactory.reconfigure` emits a Reconfigured event even when called with a code-less template address (EOA/undeployed), because the external call succeeds at the EVM level and there are no checks for code presence or post-call success. This enables misleading event emission without applying any configuration.

`MandateFactory.reconfigure` directly calls `IConfigurablePermission(template).configure(account, params, deadline, configureSig)` and then emits `Reconfigured(account, template, keccak256(params))` without validating that `template != address(0)`, `template.code.length > 0`, or that `configure` actually executed and verified a valid signature. At the EVM level, `CALL` to a code-less address returns success with empty returndata (no revert), so the factory emits the event after a silent no-op. This creates a misleading success-sounding event while no permission state changes and no kernel state is affected. The included kernel paths (`register/replace`) enforce code-length and signature checks, but the standalone `reconfigure` path does not. The impact is limited to off-chain observability/UX: on-chain authorization and funds remain unaffected.

file: `contracts/factory/MandateFactory.sol`

```
33     }
34
35     /// @notice Orchestrator that bundles (configure 'register) into
36     /// a single transaction.
```

```

...

72     error ZeroAddress();
73     error InitDataTooShort();

74     error CloneInitFailed();

...

157     bytes calldata configureSig
158 ) external {

159     IConfigurablePermission(template).configure(account,
params, deadline, configureSig);
160     emit Reconfigured(account, template, keccak256(params));

```

```

33     }
34
35 +interface ISailKernelRead { function isPermissionRegistered(ad-
dress, address) external view returns (bool); }
36     /// @notice Orchestrator that bundles (configure 'register) into
a single transaction.
37     ///

...

73     error ZeroAddress();
74     error InitDataTooShort();
75 + error NotAContract();
76 + error TemplateNotRegistered();
77     error CloneInitFailed();

...

160     bytes calldata configureSig
161 ) external {
162 + if (template == address(0)) revert ZeroAddress();
163 + if (template.code.length == 0) revert NotAContract();
164 + if (!ISailKernelRead(address(kernel)).isPermissionRegis-
tered(account, template)) revert TemplateNotRegistered();
165     IConfigurablePermission(template).configure(account,
params, deadline, configureSig);

```

```
166 emit Reconfigured(account, template, keccak256(params));
```

Octane Security

Severity

Impact Explanation: [Informational] This is a view-only/observability discrepancy: no on-chain permission or kernel state changes and no funds move. It can mislead off-chain systems but does not bypass on-chain constraints.

Likelihood Explanation: [High] Scenarios 1 and 3 require no special constraints—any user can call the public function with a code-less address. Scenario 2 depends on relayer malice, but is not needed for exploitation. Overall likelihood is high.

Exploitation

Exploitation Scenarios:

Scenario 1.

Public spoofing: An attacker calls `MandateFactory.reconfigure` for a real account with template set to an EOA (no code) and arbitrary params/sig. The external call returns success, the factory emits `Reconfigured`, but no template config is applied. Off-chain systems that treat the event as authoritative may be misled.

Preconditions / Assumptions

- (a). `MandateFactory.reconfigure` is publicly callable and deployed
- (b). A target account address exists (e.g., a registered Safe)
- (c). Attacker controls or selects a code-less address (EOA/undeployed/self-destructed) as template
- (d). Some off-chain monitors/dashboards treat the `Reconfigured` event as authoritative without cross-checking template state

Scenario 2.

Malicious relayer substitution: An operator produces a valid `configureSig` for a real permission template T, but an untrusted relayer instead submits `reconfigure` with a code-less address for template while keeping the same params/deadline/sig. The factory emits `Reconfigured` for the code-less address; the real template T remains un-



changed. Off-chain systems or operators who rely on the factory event without verifying template state may act under incorrect assumptions.

Preconditions / Assumptions

- (a). Operator uses a relayer to submit a genuine configure for a real permission template T
- (b). Relayer acts maliciously and substitutes template with a code-less address
- (c). Operator or off-chain systems rely on the factory's Reconfigured event without verifying template state (e.g., template's Configured event or configNonces increment)

Scenario 3.

Log spam: An attacker repeatedly calls reconfigure with various EOAs as template for popular accounts, generating many Reconfigured events. Monitoring and alerting tools that do not cross-check template state are flooded with misleading signals, degrading observability.

Preconditions / Assumptions

- (a). MandateFactory.reconfigure is publicly callable
- (b). Attacker can spend gas to issue repeated calls
- (c). Off-chain monitoring or alerting relies on the factory's Reconfigured event and does not cross-validate template state

Vulnerability #11

Lack of returndata interpretation in SailKernel.dispatch with Borrow-Permission on Compound V2 borrow causes nonce burn and misleading Dispatched event

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Acknowledged

Compound V2 borrow returns a non-zero status code without reverting on soft-fail (borrow cap reached, insufficient cash, comptroller rejection). The kernel treats any non-reverting module call as success – a deliberate venue-agnostic design property we are not changing, as decoding per-venue return data would break the kernel's generality. The consequence is a recoverable nonce-burn (a pre-signed manager dispatch is consumed with no borrow); no funds are at risk. Aave and Morpho revert on failure and are unaffected. We keep Compound V2 borrow available and document the soft-fail nonce-burn so operators register it as an informed choice.

SailKernel's single-dispatch path treats any non-reverting Safe module call as success and does not decode target returndata. BorrowPermission authorizes Compound V2 `cToken.borrow(uint256)`, which often returns non-zero error codes without reverting. A failing borrow is thus recorded as a successful dispatch and consumes the manager nonce even though no underlying is borrowed. Any party with the pre-signed manager message can time submission to such a soft-fail state and grief by burning the nonce.

SailKernel.dispatch verifies the manager's EIP-712 signature, evaluates the named permission, and then calls Safe's `execTransactionFromModule`. The kernel only checks the module boolean (true if the inner call did not revert) and never decodes the called contract's return data. BorrowPermission explicitly supports Compound V2 `borrow(uint256)`, resolving the underlying via `cToken.underlying()` and enforcing protocol/asset allowlists, per-tx caps, and optional LTV, but it does not and cannot verify that the borrow actually succeeds. Compound V2 `cToken.borrow` uses status codes: it returns 0 on success and non-zero error codes for many failures without reverting (e.g., borrow cap reached, insufficient cash, comptroller rejections). When submitted under these soft-fail conditions, the Safe module call does not revert, so SailKernel treats the dispatch as successful, emits Dispatched, and the manager nonce is consumed; however, no underlying is borrowed. Because SailKernel allows open-submit (any caller can submit a valid manager signature), a third party can time submission during a soft-fail window to burn the pre-signed nonce. This yields a recoverable operational DoS on that single pre-signed message and a potentially misleading success event; no funds

move and no invariants are broken. Targets without underlying() (e.g., cETH) are denied by BorrowPermission and cannot be used for this path.

file: contracts/templates/BorrowPermission.sol

```
264  if (ctx.selector == COMPOUND_BORROW) {

265  if (txData.length < LEN_COMPOUND) return false;
266  uint256 amount = abi.decode(txData[4:], (uint256));

264  if (ctx.selector == COMPOUND_BORROW) {
265  +// Deny Compound V2 borrow in single-dispatch: kernel
    dispatch treats any non-reverting
266  +// call as success and Compound borrow is a status-code
    API. Disallow here to avoid
267  +// nonce burns and misleading success events in this
    hardened template.
268  + return false;
269  if (txData.length < LEN_COMPOUND) return false;
270  uint256 amount = abi.decode(txData[4:], (uint256));
```

Severity

Impact Explanation: [Informational] Only a manager nonce is burned and a misleading Dispatched event is emitted; no funds move and the state is recoverable by re-signing. This matches the explicit scope carve-out for 'a manager nonce burned with no fund effect.'

Likelihood Explanation: [Low] Exploitation requires access to a valid pre-signed manager message and timing submission to a Compound V2 soft-fail window (e.g., borrow cap or insufficient cash). The attacker bears gas cost and gains no direct profit.

Exploitation

Exploitation Scenarios:

Scenario 1.

Borrow cap reached: The manager pre-signs a dispatch calling `cToken.borrow(amount)`. During a period when the market's borrow cap is hit, `cToken.borrow` returns a non-zero error code without reverting. `BorrowPermission` passes static checks, `Safe` module call does not revert, `SailKernel` emits `Dispatched` and consumes the nonce, but no underlying is borrowed.

Preconditions / Assumptions

- (a). Account registered with `SailKernel`; `BorrowPermission` configured with the `cToken` (protocol allowlist) and its underlying (asset allowlist).
- (b). Per-tx cap and optional LTV checks pass for the intended amount.
- (c). Valid EIP-712 manager signature for dispatch is available to third parties (open-submit relay or public mem-pool); deadline not expired.
- (d). Comptroller enforces a borrow cap such that `borrowAllowed()` rejects non-revertingly at submission time.

Scenario 2.

Insufficient cash window: The manager pre-signs a dispatch for `cToken.borrow(amount)`. When pool liquidity (`getCash`) drops below the amount, `cToken.borrow` returns a



non-zero code without revert. The kernel records success and burns the nonce while no borrow occurs.

Preconditions / Assumptions

- (a). Same account and BorrowPermission configuration with cERC20 (not cETH).
- (b). Valid manager pre-signed dispatch is accessible; deadline not expired.
- (c). Pool liquidity temporarily insufficient (`getCash < amount`) so `cToken.borrow` returns a non-zero error without revert.

Scenario 3.

Controller rejection window (e.g., market not entered or parameter change): The manager pre-signs a borrow. During a transient controller rejection (non-reverting error via `borrowAllowed`), `cToken.borrow` returns a non-zero code. The dispatch is recorded as successful; the nonce is consumed; no borrow is created.

Preconditions / Assumptions

- (a). Same account and BorrowPermission configuration with cERC20.
- (b). Valid manager pre-signed dispatch is accessible; deadline not expired.
- (c). A transient comptroller rejection (e.g., market not entered or parameter changes) occurs, causing `cToken.borrow` to return a non-zero error without revert.

Vulnerability #12

Missing fallback-relay length guard in `SailKernel.registerAccount` with `Safe fallbackHandler==kernel` causes recoverable DoS of owners' `Safe` transaction

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Out of scope

The precondition is an operator setting their own `Safe`'s `fallbackHandler` to the kernel – a self-inflicted misconfiguration the protocol cannot prevent and which honest onboarding never produces (the kernel is a `Safe` module, never a fallback handler). Even when present, the relayed call can only complete the exact registration the owners already authorized (the signature commits to the principals); it cannot hijack registration. The sole effect is the owners' own subsequent transaction reverting `AccountAlreadyRegistered` – recoverable, no funds at risk, no authorization bypassed. This is outside the protocol's enforceable threat model; it is documented as an operator requirement (do not set a `Sail`-related `Safe`'s fallback handler to the kernel).

If a `Safe` sets its fallback handler to the `SailKernel`, an external party can reuse a valid `ownerSig` (e.g., seen in mempool) to invoke `registerAccount` via the `Safe`'s fallback before the owners' `Safe.execTransaction` executes, registering legitimately but causing the owners' later tx/batch to revert with `AccountAlreadyRegistered`. No funds move and authorization is not bypassed; impact is a recoverable UX/DoS.

`Safe v1.4.1`'s `FallbackManager` forwards calls to the configured fallback handler and appends 20 bytes of the original caller. `SailKernel.registerAccount` omits an exact-calldata-length guard (unlike `collectFees` and `setManager`) and accepts a dynamic bytes `ownerSig`, so trailing bytes from the `Safe` fallback are ignored by Solidity decoding. The function relies on a robust owner-signature gate and enforces: trusted `Safe` proxy codehash, trusted masterCopy, module enabled, `Safe.nonce()>0` (blocking setup-time), deadline not expired, and rejecting `v==1` approved-hash entries before delegating signature validation to `Safe.checkSignatures`. Therefore, a third party who observes or obtains a valid `ownerSig` can call the `Safe` directly (with zero value) to relay via fallback and register early with exactly the signed principals. The owners' subsequent `Safe.execTransaction` that includes `registerAccount` then reverts with `AccountAlreadyRegistered`. No principals are altered, no funds move, and setup-time front-running remains blocked by the `nonce()==0` guard. This yields a recoverable, minor DoS/UX effect in an uncommon operator configuration (`fallbackHandler==kernel`).

```
824     bytes calldata ownerSig
825 ) external {
826 -// No exact-length calldata guard here (cf. setManager/collectFees): `ownerSig` is dynamic,
827 -// so an exact length is undefined and a minimum-length
828   check would not catch a +20-byte
829 -// fallback relay. The owner-signature requirement closes
830   the fallback-relay vector for this function
831 -// directly – a relay cannot produce the owners'
832   signature over the digest.

833
834 // 1. Codehash (cheap/static; uses extcodehash, not an
835   ISafe method call). Pins genuine
```

```
824     bytes calldata ownerSig
825 ) external {
826 +// Fallback-relay guard for dynamic-ABI: enforce exact
827   canonical calldata length to reject Safe fallback (+20 bytes).
828 +// expected length = 4 (selector) + 6*32 (heads) + 32
829   (ownerSig length word) + ceil32(ownerSig.length)
830 +{
831 + uint256 __padded = (ownerSig.length + 31) & ~uint256(31);
832 + uint256 __expected = 4 + 32 * 7 + __padded;
833 + if (msg.data.length != __expected) revert
834   UnexpectedCalldataLength();
835 +}
836
837 // 1. Codehash (cheap/static; uses extcodehash, not an
838   ISafe method call). Pins genuine
```

Severity

Impact Explanation: [Informational] Registration completes legitimately with the owners' signed principals; the only effect is a recoverable denial where the owners' subsequent Safe transaction/batch reverts with `AccountAlreadyRegistered`. No funds move, no custody changes, and setup-time abuse is blocked.

Likelihood Explanation: [Low] Requires an uncommon operator configuration (`Safe.fallbackHandler == SailKernel`), Safe already enabled with the module and `nonce > 0`, plus access to a valid `ownerSig` (mempool visibility or off-chain leak). The attack is pure grieving with no profit.

Exploitation

Exploitation Scenarios:

Scenario 1.

S1: Mempool front-run via Safe fallback. Owners broadcast a `Safe.execTransaction` to call `SailKernel.registerAccount` with a valid `ownerSig`. An attacker copies that calldata and calls the Safe directly (`value=0`). The Safe forwards to the kernel via fallback, `msg.sender` becomes the Safe, and `registerAccount` passes all checks (trusted codehash/singleton, module enabled, `nonce > 0`, deadline, approved-hash rejected, `Safe.checkSignatures` succeeds). Registration completes early; the owners' later Safe transaction reverts with `AccountAlreadyRegistered`.

Preconditions / Assumptions

- (a). `Safe.nonce() > 0` (blocks setup-time front-running)
- (b). Safe has `SailKernel` enabled as a module (`isModuleEnabled(address(this)) == true`)
- (c). `Safe.fallbackHandler == SailKernel` (uncommon operator configuration)
- (d). A valid, unexpired `ownerSig` over `RegisterAccount` (including exact principals) is visible in the mempool
- (e). Governance allowlists include the Safe proxy runtime codehash and the singleton returned by `masterCopy()`
- (f). Caller sends zero ETH (FallbackManager only forwards data with no value; `registerAccount` is non-payable)



- (g). Block timestamp $\leq$ deadline
- (b). Safe has SailKernel enabled as a module
- (c). Safe.fallbackHandler == SailKernel
- (d). Attacker possesses a valid, unexpired ownerSig for the intended principals (off-chain leak)
- (e). Governance allowlists include the Safe proxy runtime codehash and the singleton
- (f). Caller sends zero ETH
- (g). Block timestamp $\leq$ deadline
- (b). Safe has SailKernel enabled as a module before the planned batch
- (c). Safe.fallbackHandler == SailKernel
- (d). A valid, unexpired ownerSig for registerAccount is visible (e.g., in a planned batch)
- (e). Governance allowlists include the Safe proxy runtime codehash and the singleton
- (f). Caller sends zero ETH
- (g). Block timestamp $\leq$ deadline

Vulnerability #13

Non-atomic configure→register sequencing in MandateFactory causes recoverable permission denial

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Acknowledged

A front-run of the kernel leg of a factory configure→register bundle reverts the bundle and rolls back the configure, leaving the permission registered-but-unconfigured – in which state it fails closed (all dispatches deny) until a single configure() is reapplied. No funds move and no permission is bypassed; the state is deny-by-default and operator-recoverable. As with the related bundling finding, a robust fix re-opens the trusted core and there is no clean factory-only mitigation, so we accept it and document the recovery (reapply configure, or use private orderflow / replacePermission).

MandateFactory performs template.configure followed by SailKernel register/replace as separate external calls; since kernel signatures are not bound to the submitter, a mempool observer can front-run only the kernel leg, causing the factory tx to revert and rolling back configure, leaving permissions registered but unconfigured so all evaluations fail closed until reconfiguration.

MandateFactory.attach/attachBatch/replace invoke IConfigurablePermission(template).configure(...) first and then call SailKernel.registerPermission/registerPermissions/replacePermission. The kernel operations are authenticated solely by an EIP-712 permissionSigner signature and are not bound to msg.sender, enabling any relayer to submit a valid signature. An attacker who observes the factory call in the public mempool can front-run only the kernel leg, successfully registering or replacing the permission(s) and paying the required fee(s). When the original factory transaction executes, its subsequent kernel call reverts due to already-registered/already-replaced state, which rolls back the earlier configure call performed in the same factory transaction. The final state is that the permission(s) are present in the kernel but remain unconfigured in the template, and all included reference templates deny evaluates via the _configCurrent check (isConfigured false or epoch mismatch). No funds move and kernel invariants hold; the operator can recover by re-running configure (or MandateFactory.reconfigure), or by re-signing configure if deadlines have expired.

file: contracts/factory/MandateFactory.sol



```
33     }
34
35     /// @notice Orchestrator that bundles (configure 'register) into
36     a single transaction.
37     ///
38     ...
39
90     //
91     -----
92
93     function attach(
94         address account,
95         ...
96     )
97     {
110         //
111         -----
112
113         function attachBatch(
114             address account,
115             ...
116         )
117         {
165             //
166             -----
167
168             function replace(
169                 address account,
```

```

33  }
34
35  +// SECURITY NOTE:
36  +// To make attach/attachBatch/replace race-safe against front-runs
    of the kernel leg,
37  +// wrap the kernel calls in try/catch and proceed only if
    post-revert state confirms
38  +// "already-registered/replaced" via SailKernel views (see func-
    tion-level TODOs below).
39  +// @notice Orchestrator that bundles (configure 'register) into
    a single transaction.
40  +//
...
94  //
    -----
95
96  +// TODO(security): Make this idempotent and race-safe:
97  +// - Perform the kernel.registerPermission leg first, wrapped
    in try/catch.
98  +// - On catch, proceed only if kernel.isPermissionRegis-
    tered(account, template) == true.
99  +// - Then call template.configure (accept already-config-
    ured-for-current-epoch as success).
100  function attach(
101  address account,
...
118  //
    -----
119
120 +// TODO(security): Wrap kernel.registerPermissions in try/catch
    and proceed only if
121 +// every templates[i] is already registered (iterate and check
    via isPermissionRegistered).
122 +// Then call _batchConfigure with per-item idempotence (accept
    already-configured-for-current-epoch).
123 +// This prevents a revert-after-configure from rolling back
    config and avoids configuring
124 +// any template that is not yet registered.
125  function attachBatch(
126  address account,

```


...

178 //

179

180 +// TODO(security): Wrap kernel.replacePermission in try/catch
and proceed only if

181 +// (newTemplate registered AND oldTemplate not registered).
Then call newTemplate.configure

182 +// (accept already-configured-for-current-epoch). This removes
the revert-after-configure

183 +// rollback window in the presence of a front-run replace.

184 function replace(
185 address account,



Severity

Impact Explanation: [Informational] No funds move, no permission is bypassed, and the state remains deny-by-default; the effect is a recoverable denial of a bundling/orchestration path (registered but unconfigured) that resolves by a single configure call (or re-sign plus configure).

Likelihood Explanation: [Low] Attackers must observe public mempool transactions, win ordering, and pay SailKernel registration/replace fees with no profit motive (pay-to-grief); operators can also mitigate via private orderflow.

Exploitation

Exploitation Scenarios:

Scenario 1.

Single attach: Attacker front-runs `SailKernel.registerPermission(A, T, kernelSig)` from a mempool-observed `MandateFactory.attach`; the kernel accepts the valid signature and registers T. The factory then reverts on `PermissionAlreadyRegistered`, rolling back its configure. Final state: T registered but unconfigured; evaluates deny; operator calls configure once to recover.

Preconditions / Assumptions

- (a). Account A is already registered in SailKernel
- (b). `permissionSigner` has produced a valid `configureSig` for template T and a valid `kernelSig` for `registerPermission`
- (c). Operator submits `MandateFactory.attach` via public mempool with sufficient fee value
- (d). Attacker can observe the mempool and front-run ordering
- (e). Attacker pays the SailKernel registration fee
- (f). Deadlines for signatures have not expired at execution

Scenario 2.

Replace: Attacker front-runs `SailKernel.replacePermission(A, O, N, kernelReplaceSig)` before `MandateFactory.replace`; the kernel atomically removes O and inserts N. The



factory later reverts on `PermissionAlreadyRegistered(new-Permission)`, rolling back `N.configure`. Final state: `O` removed; `N` registered but unconfigured; operator configures `N` once to recover.

Preconditions / Assumptions

- (a). Account `A` has oldTemplate `O` registered
- (b). `permissionSigner` has produced a valid `configureSig` for newTemplate `N` and `kernelReplaceSig` for `replacePermission`
- (c). Operator submits `MandateFactory.replace` via public mempool with sufficient fee value
- (d). Attacker can observe the mempool and front-run ordering
- (e). Attacker pays the `SailKernel` replace fee
- (f). Deadlines for signatures have not expired at execution

Scenario 3.

Batch attach: Attacker front-runs `SailKernel.registerPermissions(A, T0..M-1, kernelBatchSig)`. The factory later reverts on duplicates, rolling back all `configure` calls. Final state: all `T[i]` registered but unconfigured (and if some `configure` deadlines have passed, fresh signatures are needed); operator reconfigures each to recover.

Preconditions / Assumptions

- (a). Account `A` intends to attach a batch of permissions `T[0..M-1]`. `permissionSigner` has produced valid `configureSig[i]` for each `T[i]` and a valid `kernelBatchSig`
- (c). Operator submits `MandateFactory.attachBatch` via public mempool with sufficient aggregate fee value
- (d). Attacker can observe the mempool and front-run ordering
- (e). Attacker pays $M \times$ the `SailKernel` registration fee
- (f). `Configure` deadlines may be close to expiry (optional additional friction), but not required for the base effect

Vulnerability #14

AccessControl admin requirement in SailGovernance.rotateTimelockRoles with self-admin TimelockController causes governance-handoff delay

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Resolved

Under a self-administered TimelockController, rotateTimelockRoles could not function – its inner grantRole/revokeRole calls execute as the governance contract, not the timelock, and revert. The helper has been neutralized (it now reverts with a clear error) and its documentation corrected to describe the working handoff procedure: scheduling timelock self-calls to grant/revoke the roles, then accepting governance. This removes a footgun that would have reverted during a real governance handoff. Fixed and merged.

The helper rotateTimelockRoles in SailGovernance cannot grant/revoke timelock roles under the intended self-admin TimelockController because grantRole/revokeRole require the caller to hold DEFAULT_ADMIN_ROLE. The nested calls are made by SailGovernance (not the timelock), so they revert. As a result, the documented governance handoff flow fails until operators schedule proper timelock self-calls, introducing a recoverable governance delay.

Under the intended configuration, the TimelockController is self-administered: only the timelock holds DEFAULT_ADMIN_ROLE. SailGovernance.rotateTimelockRoles is restricted by onlyTimelock and, when executed via the timelock, makes external calls to timelock.grantRole/revokeRole. In OpenZeppelin AccessControl, grantRole/revokeRole enforce onlyRole(getRoleAdmin(role)) and therefore require the caller to have DEFAULT_ADMIN_ROLE. From TimelockController's perspective, those calls come from the SailGovernance contract (msg.sender == SailGovernance), not from the timelock, so they revert. Consequently, rotateTimelockRoles cannot transfer PROPOSER/EXECUTOR/CANCELLER to the incoming governor. acceptGovernance() requires the candidate to already hold all three roles, so it also reverts. Operators must instead schedule proper timelock self-calls (target = timelock) to grant the roles to the candidate and revoke them from the old governor, then have the candidate call acceptGovernance(). This creates a recoverable operational delay in governance handoff but does not risk funds or break core invariants.

file: contracts/governance/SailGovernance.sol



```

359  ///      can complete, eliminating the window where old
governance retains timelock keys.
360      error RolesNotYetRotated();

361
362  /// @dev Thrown when `pause()` is called before PAUSE_COOLDOWN
has elapsed since the last pause.

...

587  function rotateTimelockRoles(address oldGov, address newGov)
external onlyTimelock {
588      if (newGov == address(0)) revert ZeroAddress();

589      bytes32 proposer  = timelock.PROPOSER_ROLE();
590      bytes32 executor   = timelock.EXECUTOR_ROLE();

```

```

359  ///      can complete, eliminating the window where old
governance retains timelock keys.
360      error RolesNotYetRotated();
361 + error TimelockMustSelfGrantRoles();
362
363  /// @dev Thrown when `pause()` is called before PAUSE_COOLDOWN
has elapsed since the last pause.

...

588  function rotateTimelockRoles(address oldGov, address newGov)
external onlyTimelock {
589      if (newGov == address(0)) revert ZeroAddress();
590 + revert TimelockMustSelfGrantRoles();
591      bytes32 proposer  = timelock.PROPOSER_ROLE();
592      bytes32 executor   = timelock.EXECUTOR_ROLE();

```

Severity

Impact Explanation: [Low] The issue causes a recoverable operational denial/delay of governance handoff and time-locked updates but does not threaten funds, user operations, or core invariants. It is fixed by re-running authenticated steps (scheduling proper timelock self-calls).

Likelihood Explanation: [Low] It requires the uncommon governance-handoff phase and that operators follow the helper path without pretesting, after which they can recover by scheduling correct self-calls. The scenario is not attacker-driven and has multiple preconditions.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (single-chain): Operators follow the documented sequence during a governance handoff. They schedule `proposeGovernance(candidate)` and `rotateTimelockRoles(old-Gov, candidate)` via the timelock. After the delay, `proposeGovernance` executes successfully. When the timelock executes `rotateTimelockRoles`, the nested `grantRole/revokeRole` calls `revert` (caller is `SailGovernance`, not an admin). `acceptGovernance()` by the candidate then reverts because the candidate lacks the required roles. Operators must schedule timelock self-calls to grant/revoke roles, wait the delay, and then complete acceptance, causing a material but recoverable governance delay.

Preconditions / Assumptions

- (a). `TimelockController` is self-admin (only the timelock holds `DEFAULT_ADMIN_ROLE`).
- (b). `SailGovernance` is not granted `DEFAULT_ADMIN_ROLE` on the timelock.
- (c). A governance handoff is in progress (`proposeGovernance` scheduled via timelock).
- (d). Operators, following the contract comments, schedule and execute `rotateTimelockRoles` via the timelock before `acceptGovernance`.
- (e). No separate key loss or admin compromise; operators



can schedule fallback timelock self-calls.

Scenario 2.

Scenario 2 (multi-chain): The same handoff procedure is run across multiple chains. On each chain, `rotateTimelockRoles` reverts for the same reason, and `acceptGovernance` fails until operators schedule the correct timelock self-calls per chain. This creates multi-chain coordination overhead and extended delays, but remains recoverable and does not affect user funds.

Preconditions / Assumptions

- (a). Same as Scenario 1, but occurring across multiple chains with identical deployments.
- (b). Operators schedule the same (faulty) `rotateTimelockRoles` operation on each chain before `acceptGovernance`.
- (c). Operators retain ability to schedule fallback timelock self-calls on each chain.

Warning #1

Direct-auth without fallback-relay guard in ConfigurablePermission with Safe permissionSigner fallback-handler misconfiguration causes unauthorized config and fund theft

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

The precondition is a feeManager that is a Safe whose fallback handler is set to this policy contract – a self-inflicted operator misconfiguration the protocol does not produce and cannot prevent (a default EOA feeManager, or a Safe using the standard fallback handler, has no exposure). When present, an external caller can relay onlyFeeManager calls through the Safe; the realistic impact is fee-recipient takeover via proposeFeeManager (the relayed call decodes the appended caller as the new manager). We are not adding an in-contract guard, for two reasons. First, the obvious cheap guard does not work: an exact-calldata-length check is bypassable on the single-address-argument setters (proposeFeeManager, setDistributor) – the fallback relay appends the caller's 20 bytes, which for a one-address-argument function land exactly in the decoded address slot, so a length-matched relayed call both passes the check and decodes the attacker as the argument. Second, the only effective in-contract defense is a fallback-handler storage check, which cannot use an EOA-only restriction here because a feeManager is legitimately often a Safe. Given the precondition is an attacker-uninducible misconfiguration and the fee policy is a replaceable, non-core contract, we accept the finding and document the operator requirement: never set a feeManager Safe's fallback handler to the policy contract.

ConfigurablePermission's direct-authorized functions (configureDirect, setAgentIdentityDirect) authorize solely by `msg.sender == permissionSigner` and lack a Safe fallback-relay defense. If the permissionSigner is a Gnosis Safe whose fallback handler is (mis)configured to the permission contract (or a DELEGATECALL-style forwarder), any external caller can invoke these direct functions via the Safe. The permission contract observes `msg.sender` as the Safe and accepts the call without an EIP712 signature, enabling unauthorized per-account configuration updates. For templates that constrain value recipients by configuration (TransferPermission, WithdrawPermission, ApproveAndCallBatchPermission with recipient not pinned), this allows an attacker to widen bounds and, with an adversarial manager, exfiltrate funds.

Gnosis Safe v1.4.1 forwards unknown calls to its configured fallback handler by CALL, setting `msg.sender` to the Safe and appending the original caller's 20-byte address to `calldata`. Solidity's ABI decoder ignores trailing bytes.



In ConfigurablePermission, configureDirect and setAgentIdentityDirect only check that msg.sender equals the account's permissionSigner; they do not verify signatures or enforce an exact calldata-length relay guard (params is dynamic). If the permissionSigner is a Safe whose fallback handler is set to the permission contract (or a DELEGATECALL-style forwarder into it), any external caller can send configureDirect/setAgentIdentityDirect calldata to the Safe. The Safe forwards to the permission, which sees msg.sender == Safe and accepts the call, applying new configuration and stamping the current registration epoch. For templates that use configuration to pin or constrain recipients or approvals (e.g., TransferPermission's recipient allowlist, WithdrawPermission's single allowedRecipient, ApproveAndCallBatchPermission with requireRecipientIsAccount disabled), an attacker can reconfigure the account to allow payouts to the attacker. The manager, treated as potentially adversarial within permissions, can then execute within these widened bounds to move funds to the attacker. The default Safe CompatibilityFallbackHandler would not forward arbitrary selectors to the template, so this requires a non-default, uncommon operator misconfiguration of the permissionSigner Safe's fallback handler. Signature-based configuration paths (configure/setAgentIdentity) remain safe.

file: contracts/templates/ConfigurablePermission.sol

```
83     error ZeroAddress();
84     /// @notice Thrown when a price oracle is configured but no
    freshness bound is set.

85     error MissingPriceAge();

...

140     address permSigner = kernel.configs(account);
141     if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);

142
143     uint256 nonce = configNonces[account]++;

...

204     address permSigner = kernel.configs(account);
205     if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);

206
207     configNonces[account]++;
```

```

83     error ZeroAddress();
84     /// @notice Thrown when a price oracle is configured but no
    freshness bound is set.
85 + error DirectAuthEOAOnly();
86     error MissingPriceAge();

...

141     address permSigner = kernel.configs(account);
142     if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);
143 + if (msg.sender.code.length != 0) revert DirectAuthEOAOnly();
144
145     uint256 nonce = configNonces[account]++;

...

206     address permSigner = kernel.configs(account);
207     if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);
208 + if (msg.sender.code.length != 0) revert DirectAuthEOAOnly();
209
210     configNonces[account]++;

```

Severity

Impact Explanation: [High] All valid scenarios enable direct, material loss of principal (ERC-20 transfers or swap outputs sent to the attacker), which is high impact.

Likelihood Explanation: [Low] Exploitation relies on an uncommon operator misconfiguration: the permissionSigner Safe's fallback handler must be set to the permission contract (or a DELEGATECALL-style forwarder). Managers are already modeled as potentially adversarial; the rare operator setup drives low likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.

TransferPermission: An attacker calls the permission-Signer Safe with calldata for T.configureDirect(account, params) where params adds the attacker's address to the allowedRecipients and sets a large maxAmountPerTx. Due to the Safe fallback relay, msg.sender inside T is the Safe, so the direct-authorized update is accepted and epoch-stamped. The adversarial manager then dispatches ERC-20 transfer calls to the attacker within the cap, draining tokens from the account.

Preconditions / Assumptions

- (a). Account A is registered in SailKernel.
- (b). PermissionSigner for A is a Gnosis Safe S.
- (c). Safe S's fallback handler is set to the TransferPermission contract T (or to a handler that DELEGATECALLs into T).
- (d). T is registered for A.
- (e). A holds ERC-20 balances in allowed tokens.
- (f). Manager M is potentially adversarial within permissions (per scope).

Scenario 2.

ApproveAndCallBatchPermission (requireRecipientIsAccount off): An attacker configures B via the permission-Signer Safe to allow a (router, swapExactTokensForTokens) consuming pair, large approval caps, and keep requireRe-



recipientIsAccount=false. The adversarial manager submits an approve/swap/reset batch where the swap's 'to' is the attacker. B.evaluateBatch approves the batch, and swap output is sent to the attacker.

Preconditions / Assumptions

- (a). Account A is registered in SailKernel.
- (b). PermissionSigner for A is a Gnosis Safe S.
- (c). Safe S's fallback handler is set to the ApproveAndCall-BatchPermission contract B (or to a handler that DELEGATECALLs into B).
- (d). B is registered for A.
- (e). A holds ERC-20 balances.
- (f). requireRecipientIsAccount is configured as false post-unauthorized update.
- (g). Consuming pair (router, swapExactTokensForTokens) is allowlisted post-unauthorized update.
- (h). Manager M is potentially adversarial within permissions (per scope).

Scenario 3.

WithdrawPermission: An attacker calls the permission-Signer Safe with W.configureDirect(account, params) setting allowedRecipient to the attacker and allowing relevant tokens. The adversarial manager dispatches transfer or transferFrom to the attacker within maxAmountPerTx. W.evaluate authorizes the call, and funds are transferred to the attacker.

Preconditions / Assumptions

- (a). Account A is registered in SailKernel.
- (b). PermissionSigner for A is a Gnosis Safe S.
- (c). Safe S's fallback handler is set to the WithdrawPermission contract W (or to a handler that DELEGATECALLs into W).
- (d). W is registered for A.
- (e). A holds ERC-20 balances in allowed tokens.
- (f). Manager M is potentially adversarial within permissions (per scope).

Warning #2

Missing contract-code check for nonzero fee asset in SailKernel.collectFees allows advancing fee state without transfers

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Acknowledged

The condition requires a trusted role (permissionSigner/owners) to configure a non-zero, code-less fee asset; there is no unprivileged path, and no funds leave the account – the only effect is that a collection advances fee-policy state (high-water mark/timestamp) against a transfer that moved nothing, foregoing that window's fee. This is self-inflicted by a trusted party and moves no principal. We are keeping the trusted core lean and documenting the requirement that a configured fee asset must be a deployed ERC-20, rather than adding a code-existence check to the kernel.

If an account's canonical fee asset is set to a nonzero address with no code, collectFees takes the ERC20 path and performs Safe module CALLs to IERC20.transfer on that address. Gnosis Safe's module uses a raw CALL with no code check, which succeeds and returns empty returndata to a code-less address. _safeTransferERC20 accepts empty returndata as success, so collectFees records the collection (advancing HWM/timestamp) and emits FeesCollected even though no funds are transferred. This results in missed protocol/distributor payouts for that period while funds remain in the Safe.

SailKernel stores a per-account canonical fee asset (configs[account].feeAsset), with address(0) meaning native ETH. The kernel never enforces that a nonzero fee asset has code. In collectFees, a nonzero feeToken triggers the ERC20 branch, which calls _safeTransferERC20 for each payout share. _safeTransferERC20 uses Safe.execTransactionFromModuleReturnData to call IERC20.transfer on feeToken. Gnosis Safe v1.4.1 executes a raw EVM CALL without checking code; a CALL to a code-less address (EOA/empty) returns success with empty returndata. _safeTransferERC20 explicitly treats empty returndata as success. Additionally, collectFees invokes IFeePolicy.recordCollection before any transfers (CEI), so policy state (e.g., high-water mark and lastCollectionTimestamp) advances and the kernel emits FeesCollected even if no tokens moved. As a result, the protocol treasury and distributor miss their fee revenue for that window, while the value remains in the account. This requires a trusted operator (permissionSigner/owners) to set a nonzero, code-less fee asset and the manager (or the Safe) to call collectFees; there is no unprivileged attack path.

file: contracts/core/SailKernel.sol



```

909     if (permissionSigner == address(0) || manager == address(0))
        revert ZeroAddress();
910     if (feePolicy != address(0) && !governance.trustedFeePol-
        icy(feePolicy)) revert UntrustedFeePolicy(feePolicy);

911     registered[account] = true;
912     configs[account] = AccountConfig({

...

1231     if (governance.isPaused() && newFeePolicy != address(0))
        revert ProtocolPaused();
1232     if (newFeePolicy != address(0) && !governance.trustedFeeP-
        olicy(newFeePolicy)) revert UntrustedFeePolicy(newFeePolicy);

1233     if (block.timestamp > deadline) revert DeadlineExpired(dead-
        line, block.timestamp);
1234     uint256 nonce = signerNonces[account];

...

1802     if (grossFee == 0) revert ZeroFee();
1803     if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(fee-
        Token, cfg.feeAsset);

1804
1805     // Recipient is always pulled from the policy – prevents
        manager from redirecting fees.

```

```

909     if (permissionSigner == address(0) || manager == address(0))
        revert ZeroAddress();
910     if (feePolicy != address(0) && !governance.trustedFeePol-
        icy(feePolicy)) revert UntrustedFeePolicy(feePolicy);
911 +   if (feePolicy != address(0) && feeAsset != address(0) &&
        feeAsset.code.length == 0) revert NotAContract(feeAsset);
912     registered[account] = true;
913     configs[account] = AccountConfig({

...

1232     if (governance.isPaused() && newFeePolicy != address(0))
        revert ProtocolPaused();
1233

```

```
        if (newFeePolicy != address(0) && !governance.trustedFeePolicy(newFeePolicy)) revert UntrustedFeePolicy(newFeePolicy);
1234+ if (newFeePolicy != address(0) && feeAsset != address(0)
    && feeAsset.code.length == 0) revert NotAContract(feeAsset);
1235 if (block.timestamp > deadline) revert DeadlineExpired(deadline, block.timestamp);
1236 uint256 nonce = signerNonces[account];

...

1804 if (grossFee == 0) revert ZeroFee();
1805 if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(feeToken, cfg.feeAsset);
1806+ if (feeToken != address(0) && feeToken.code.length == 0)
    revert NotAContract(feeToken);
1807
1808 // Recipient is always pulled from the policy – prevents
    manager from redirecting fees.
```

Severity

Impact Explanation: [Low] Missed protocol/distributor fee revenue for a bounded collection window while funds remain within the account beneficiary set; policy state advances but no principal leaves the account. The adverse effect is limited to a window and does not involve external theft.

Likelihood Explanation: [Low] Requires trusted-role control (permissionSigner/owners) to set a code-less nonzero fee asset and the manager (or Safe) to perform collectFees; no unprivileged attacker path exists.

Exploitation

Exploitation Scenarios:

Scenario 1.

An adversarial manager in a self-managed account (who also controls the permissionSigner) sets configs[account].feeAsset to a nonzero address without code (e.g., a legacy ETH sentinel). After the policy's HWM is seeded and the collection interval elapses, the manager calls collectFees with feeToken equal to that address and a grossFee within the policy's max. The kernel calls recordCollection first (advancing lastCollectionTimestamp and HWM), then attempts ERC20 transfers via Safe module calls to the code-less address. Each CALL returns success with empty returndata; _safeTransferERC20 accepts them, and FeesCollected is emitted. No funds actually move to treasury/distributor/feeRecipient, permanently burning that period's accrual while funds remain in the Safe.

Preconditions / Assumptions

- (a). Account is registered and sessionActive is true
- (b). A trusted fee policy (e.g., StandardFeePolicy) is set on the account
- (c). The fee policy's HWM has been seeded and MIN_COLLECTION_INTERVAL elapsed
- (d). The permissionSigner/owners configured configs[account].feeAsset to a nonzero, code-less address
- (e). Manager is authorized (or the Safe itself will call



collectFees)

(f). Gnosis Safe v1.4.1 module execution performs raw CALL without code checks

(g). `_safeTransferERC20` accepts empty returndata as success

(h). `grossFee > 0` and `<= policy.computeFee(account, currentNav).grossFee`

Octane Security



Warning #3

Missing onAttach and asset binding in SailKernel._registerAccount causes first-window fee over-collection from stale state

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Acknowledged

Registering with a pre-seeded fee policy can over-charge the first collection because the policy's per-account anchor is not reset at registration. The effect is a one-time, bounded first-window fee difference that stays within the authorized recipients (treasury/distributor/fee recipient) – no theft, no invariant break – and the operator fully avoids it by calling `setFeePolicy` with the same policy once after registration, which re-anchors the policy. Because a simple operator procedure resolves it and most onboarding uses no fee policy, we are documenting the operator step rather than adding lifecycle logic to a trusted-core onboarding path.

On initial account registration with a nonzero fee policy, `SailKernel._registerAccount` stores `feePolicy/feeAsset` but does not bind the policy to the asset or invoke `IFeePolicy.onAttach(account)`. With `StandardFeePolicy`, any pre-existing per-account state (`hwmSeeded`, `lastCollectionTimestamp`, `HWM`) remains active, so the first `collectFees` can charge management fees over the pre-attach interval and performance fees on pre-attach gains. Operators can avoid this by calling `setFeePolicy` with the same policy/asset post-registration to trigger `onAttach`.

`SailKernel._registerAccount` sets `configs[account].feePolicy` and `configs[account].feeAsset` on first registration but omits two protections that `setFeePolicy` applies: (1) pinning the (account, policy) to a single fee asset via `_policyAssetBinding`, and (2) calling `IFeePolicy.onAttach(account)` to re-anchor per-account state for stateful policies. `StandardFeePolicy` keeps per-account state (`highWaterMark`, `lastCollectionTimestamp`, `hwmSeeded`, `applied rates`, `pendingReanchor`). Its `onAttach(account)` resets `lastCollectionTimestamp` to now and sets `pendingReanchor`, ensuring the first post-attach collection charges no performance fee and re-bases the HWM, and preventing management fee accrual over the dormant/pre-attach interval. Because `_registerAccount` does not call `onAttach`, if the policy already has state for the account (e.g., `feeManager` previously called `seedHighWaterMark`), then the first `collectFees` will compute fees from that earlier timestamp and HWM, including management over $t_0 \rightarrow \text{now}$ and performance on gains above the prior HWM. Asset-binding omission at registration is less impactful because `collectFees` enforces `feeToken == configs[account].feeAsset` and `setFeePolicy` later lazily binds and enforces the asset for the policy instance. Operators can fully avoid the effect by calling `setFeePolicy(account, samePolicy,`

sameAsset, ...) immediately after registration and before the first collect to invoke onAttach.

file: contracts/core/SailKernel.sol

```
917     sessionActive:    true
918   });
```

```
919     emit AccountRegistered(account, permissionSigner, manager);
920   }
```

```
917     sessionActive:    true
918   });
919 +// Bind the initial policy to the configured fee asset
    and invoke the lifecycle hook
920 +// to re-anchor any persisted per-account accounting on
    stateful policies.
921 + if (feePolicy != address(0)) {
922 +   PolicyAssetBinding storage binding = _policyAssetBind-
    ing[account][feePolicy];
923 +   if (!binding.bound) { binding.bound = true; binding.asset
    = feeAsset; }
924 +   else if (binding.asset != feeAsset) revert
    FeePolicyAssetMismatch(feePolicy, binding.asset, feeAsset);
925 +   IFeePolicy(feePolicy).onAttach(account);
926 +}
927     emit AccountRegistered(account, permissionSigner, manager);
928   }
```

Severity

Impact Explanation: [Low] One-time bounded first-window fee difference (management over pre-attach interval and performance on pre-attach gains) that remains within the account beneficiary set (fee recipient/distributor/treasury). Operator can avoid it by a single `setFeePolicy(samePolicy)` call pre-collect, so it is operator-recoverable and capped per the scope's carve-outs.

Likelihood Explanation: [Low] Requires privileged/operator sequencing (pre-seeding before registration) and an omission (not calling `setFeePolicy(samePolicy)` to `onAttach` before first collect). Relies on trusted role behavior, not a permissionless trigger.

Exploitation

Exploitation Scenarios:

Scenario 1.

Pre-seeded before registration: `FeeManager` seeds `StandardFeePolicy` for Safe S at time t_0 . Owner registers S in `SailKernel` with `feePolicy` set to that instance. Because `_registerAccount` does not call `onAttach`, the first `collectFees` later charges management fees over $t_0 \rightarrow \text{now}$ and performance fees on gains above the prior HWM.

Preconditions / Assumptions

- (a). `StandardFeePolicy` is trusted and configured in governance
- (b). `FeeManager` has pre-seeded S via `seedHighWaterMark(S, initialNav)` at time t_0
- (c). Owner registers S with `SailKernel` with a nonzero `feePolicy` and `feeAsset`
- (d). Operator does not call `setFeePolicy(account, samePolicy, sameAsset, ...)` post-registration to re-anchor
- (e). Manager (or S) calls `collectFees` with `feeToken` matching `feeAsset` and honest `currentNav`

Scenario 2.

Existing Safe self-registers after pre-seeding: A pre-existing Safe S is seeded at t_0 by the policy's `feeManager`. The



owner self-registers S with SailKernel using that policy. The first collectFees prices from t0 (no onAttach at registration), charging the pre-attach interval and gains.

Preconditions / Assumptions

- (a). Safe S exists off-kernel and is pre-seeded at t0 by StandardFeePolicy.feeManager
- (b). Owner self-registers S in SailKernel with feePolicy set to StandardFeePolicy and a feeAsset
- (c). Operator omits a post-registration setFeePolicy(samePolicy) re-anchor
- (d). Manager (or S) calls collectFees with feeToken == feeAsset and honest currentNav

Scenario 3.

Dormant historical state reused at initial registration: StandardFeePolicy holds historical state for S (hwmSeeded true, lastCollectionTimestamp = t0). Owner registers S with feePolicy = that instance. Without onAttach at registration, the first collectFees includes the dormant period and may include performance fees on gains since the prior HWM.

Preconditions / Assumptions

- (a). StandardFeePolicy already holds historical per-account state for S (hwmSeeded true, lastCollectionTimestamp = t0)
- (b). Owner registers S with SailKernel using that same policy and sets feeAsset
- (c). Operator does not invoke setFeePolicy(samePolicy) to trigger onAttach before the first collect
- (d). Manager (or S) calls collectFees with feeToken == feeAsset and honest currentNav

Warning #4

Single raw-unit per-tx cap across mixed-decimal tokens in SwapPermission causes larger-than-intended trades and slippage exposure

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Unresolved

SwapPermission enforces one `maxAmountPerTx` in raw token units for all allowed tokenIn assets. With mixed-decimal tokens (e.g., 18-dec WETH and 6-dec USDC), the same numeric cap permits vastly different economic sizes. A malicious manager can execute a much larger one-shot trade for a low-decimal token while still passing the cap and oracle checks. Outputs remain pinned to the account; impact is policy under-enforcement within the account envelope.

SwapPermission stores a single `uint256 maxAmountPerTx` per account and checks it directly against the `calldata amountIn` across all supported swap shapes (Uniswap V2 `swapExactTokensForTokens` and Uniswap V3 `exactInputSingle`). There is no per-token cap and no normalization by token decimals. If the `permissionSigner` configures `tokensIn` with heterogeneous decimals (e.g., USDC(6) and WETH(18)) and chooses a cap sized with an 18-dec token in mind (e.g., `1e18`), the same numeric cap allows orders of magnitude larger movement for the 6-dec token. The template also requires the router and tokens to be allowlisted, denies `ctx.value != 0`, pins the recipient to the account, and enforces an oracle-based minimum-out floor via `IOracle` (freshness-checked). Thus, a manager acting adversarially within the registered permission can select the low-decimal token and set `amountOutMin` to the oracle-implied floor, passing all checks and executing a larger-than-intended single trade. The proceeds always return to the account (`recipient == account`), so no external theft occurs. This is a policy/design limitation: a single shared raw-unit cap weakly constrains per-tx size across mixed-decimal tokens. Operators can mitigate by using separate `SwapPermission` instances per decimals group or by using `ApproveAndCallBatchPermission` with per-token caps (and `requireRecipientIsAccount=true`), or selecting a cap sized to the smallest-decimal token.

file: contracts/templates/SwapPermission.sol

```
150     if (maxSlippageBps > 9_999) revert SlippageBpsTooLarge(maxSlippageBps);
151     // The oracle is mandatory by design: without it the
        template would have no on-chain price
152
```

```

    // reference and could only fall back to trusting the
    manager's quote. That reference-free
153 // mode lives in SwapPermissionNoOracle, never here – so
    reject a missing oracle outright.

    ...

188 if (!isAllowedRouter[ctx.account][ctx.target]) return
    false;
189 Slot storage s = _slots[ctx.account];

190
191 if (ctx.selector == EXACT_INPUT_SINGLE_V1) {

```

```

150 if (maxSlippageBps > 9_999) revert SlippageBpsTooLarge(maxS-
    lippageBps);
151 // The oracle is mandatory by design: without it the
    template would have no on-chain price
152 +// Harden: extend params to carry per-token caps
    index-parallel with tokensIn and write into a maxInPerToken
    mapping.
153 // reference and could only fall back to trusting the
    manager's quote. That reference-free
154 // mode lives in SwapPermissionNoOracle, never here – so
    reject a missing oracle outright.

    ...

189 if (!isAllowedRouter[ctx.account][ctx.target]) return
    false;
190 Slot storage s = _slots[ctx.account];
191 +// Harden: the single shared maxAmountPerTx below is
    sensitive to mixed-decimal tokensIn.
192 +// Recommended fix: store per-token caps (maxInPerToken[ac-
    count][tokenIn]) and compare amountIn <= that cap,
193 +// instead of comparing against one shared s.maxAmountPerTx
    across all tokenIn.
194
195 if (ctx.selector == EXACT_INPUT_SINGLE_V1) {

```

Severity

Impact Explanation: [Low] Policy under-enforcement/design limitation: enables larger-than-intended single-trade size for low-decimal tokens but all outputs are pinned to the account and an oracle floor is enforced; no external theft or broken core invariant.

Likelihood Explanation: [Low] Exploitability depends on operator configuration choices (mixed-decimal tokens under one instance, cap sized for an 18-dec token) alongside the account holding and approving a large low-decimal balance; while plausible, it remains configuration-dependent and within operator control.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (Uniswap V2): The account's SwapPermission is configured with routers including a V2 router, tokensIn = [USDC(6), WETH(18)], tokensOut = [WETH], maxAmountPerTx = 1e18, and a valid oracle. The Safe holds a large USDC balance and has allowance set to the router. The manager crafts swapExactTokensForTokens with amountIn equal to the entire USDC balance (e.g., 1e13 raw units for 10m USDC), path = [USDC, WETH], to = account, and amountOutMin ≥ the oracle-implied floor. In evaluate: allowed router/tokens and recipient pin pass; amountIn ≤ maxAmountPerTx passes due to raw-unit comparison; oracle check passes. The router spends USDC and returns WETH to the account, enabling a much larger one-shot USDC trade than intended per tx.

Preconditions / Assumptions

- (a). Account is registered; SwapPermission is registered and configured; configuredEpoch matches ctx.configEpoch
- (b). Allowed routers include the target router (V2); tokensIn include USDC(6) and WETH(18); tokensOut include WETH
- (c). maxAmountPerTx is a single shared raw-unit cap (e.g., 1e18) with no per-token override or normalization
- (d). A valid price oracle and maxPriceAgeSec are set; oracle returns fresh price and valid decimals



- (e). The Safe holds a large USDC balance and has ERC-20 allowance to the router
- (f). Manager holds the delegated key and submits `SailKernel.dispatch` with `value=0` and a valid signature
- (g). Manager sets `amountOutMin` $\geq$ oracle-implied floor to pass the oracle band

Scenario 2.

Scenario 2 (Uniswap V3): The account's `SwapPermission` is configured with a V3 router, `tokensIn` = [USDC(6), WETH(18)], `tokensOut` = [WETH], `maxAmountPerTx` = `1e18`, and a valid oracle. The Safe holds a large USDC balance and has allowance to the router. The manager crafts `exactInputSingle(tokenIn=USDC, tokenOut=WETH, recipient=account, amountIn=entire USDC balance, amountOutMinimum  $\geq$  oracle floor)`. In `evaluate`: allowed router/tokens and recipient pin pass; `amountIn` $\leq$ `maxAmountPerTx` passes due to raw-unit comparison; oracle check passes. The router spends USDC and returns WETH to the account, again allowing a larger-than-intended single trade for the low-decimal token.

Preconditions / Assumptions

- (a). Account is registered; `SwapPermission` is registered and configured; `configuredEpoch` matches `ctx.configEpoch`
- (b). Allowed routers include a Uniswap V3 router; `tokensIn` include USDC(6) and WETH(18); `tokensOut` include WETH
- (c). `maxAmountPerTx` is a single shared raw-unit cap (e.g., `1e18`) with no per-token override or normalization
- (d). A valid price oracle and `maxPriceAgeSec` are set; oracle returns fresh price and valid decimals
- (e). The Safe holds a large USDC balance and has ERC-20 allowance to the V3 router
- (f). Manager holds the delegated key and submits `SailKernel.dispatch` with `value=0` and a valid signature
- (g). Manager sets `amountOutMinimum` $\geq$ oracle-implied floor to pass the oracle band

Warning #5

Calldata-length-only fallback relay guard in SailKernel.setManager with Safe fallbackHandler==kernel causes unauthorized manager rotation and permission clearing

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Out of scope

The precondition is the account's own Safe having its fallbackHandler set to the kernel – a self-inflicted misconfiguration the protocol does not produce and cannot prevent (the kernel is a module, never a fallback handler). When present, the effect is a recoverable griefing rotation: permissions are cleared and epochs bump, but the attacker cannot dispatch (no permissions remain) and cannot move funds. No principal is at risk; the exact-length guard already present blocks the trivial case, and the residual 36-byte coincidence requires the out-of-model fallback misconfiguration. Documented as an operator requirement rather than defended with additional trusted-core logic.

If a registered Safe sets its fallback handler to the SailKernel, an external caller can bypass the calldata-length relay guard in setManager by leveraging Safe's 20-byte caller-append behavior. This rotates the account's manager to the attacker, clears all permissions, and bumps nonce epochs, causing a recoverable DoS of kernel-based dispatch. The attacker cannot dispatch (permissions are cleared) and any fee collection they trigger pays only policy-authorized recipients, not the attacker.

Safe v1.4.1's FallbackManager appends the original caller's 20-byte address to calldata and CALLs the fallback handler. SailKernel.setManager authorizes by requiring msg.sender be the Safe and by checking msg.data.length == 36 (4-byte selector + 32-byte address). An attacker can call the Safe directly with only 16 bytes (selector + 12 bytes filler). After Safe appends 20 bytes, the kernel sees exactly 36 bytes. Solidity decodes the address from the low 20 bytes of the 32-byte slot, which are the appended caller address, so newManager becomes the attacker. Because FallbackManager uses CALL, msg.sender in the kernel is the Safe, so _requireRegistered(msg.sender) passes. The kernel then sets the new manager, clears all permissions, and bumps nonce epochs. Preconditions are narrow: the Safe's fallbackHandler must be the kernel itself (not a separate relay), which is an uncommon operator misconfiguration the attacker cannot induce. Impact is a recoverable operational denial for kernel-based execution: permissions are wiped and must be re-registered; existing signatures are invalidated. As the new manager, the attacker can call collectFees directly, but fee recipients are defined by the fee policy (treasury/distributor/policy feeRecipient), so value remains within authorized recipients; there is no direct theft to the attacker.



Bypassing collectFees via fallback alone is ineffective due to FeeTokenMismatch when the last word decodes to the appended caller address. Severity is Low due to low likelihood (rare misconfiguration) and bounded, recoverable impacts.

file: contracts/core/SailKernel.sol

```
953     address account = msg.sender;  
954     _requireRegistered(account);
```

```
955     if (newManager == address(0)) revert ZeroAddress();  
956     address oldManager = configs[account].manager;
```

```
953     address account = msg.sender;  
954     _requireRegistered(account);  
955 +// Robust guard: reject when the Safe's fallback handler  
    is the kernel itself.  
956 +// Read Safe's FALLBACK_HANDLER_STORAGE_SLOT via  
    StorageAccessible.getStorageAt(slot, 1).  
957 +(bool ok, bytes memory out) = account.staticcall(  
958 + abi.encodeWithSignature(  
959 +"getStorageAt(uint256,uint256)",  
960 +  
    0-  
    x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5,  
961 +1  
962 +)  
963 +);  
964 + if (ok) { bytes memory w = abi.decode(out, (bytes));  
    if (w.length >= 32) { bytes32 sw; assembly { sw := mload(add(w,  
    32)) } if (address(uint160(uint256(sw))) == address(this)) revert  
    UnexpectedCalldataLength(); } }  
965     if (newManager == address(0)) revert ZeroAddress();  
966     address oldManager = configs[account].manager;
```

Severity

Impact Explanation: [Medium] Significant but temporary availability loss/DoS of core per-account kernel dispatch due to permission clearing and epoch bumps; fee-triggering stays within authorized recipients and does not constitute principal theft.

Likelihood Explanation: [Low] Exploit requires a rare/uncommon operator misconfiguration (Safe fallbackHandler set to the kernel) that the attacker cannot induce; additional policy-state constraints apply for fee collection.

Exploitation

Exploitation Scenarios:

Scenario 1.

Unauthorized manager rotation via Safe fallback misconfiguration: attacker calls the Safe with 16-byte calldata so that, after the Safe appends 20 bytes, setManager sees 36 bytes and decodes newManager as the attacker; kernel sets the attacker as manager, clears all permissions, and bumps nonce epochs, causing a recoverable DoS of kernel dispatch.

Preconditions / Assumptions

- (a). The Safe is already registered with the SailKernel.
- (b). The Safe's fallbackHandler is set to the SailKernel contract (CALL-based forwarding ensures msg.sender == Safe in the kernel).
- (c). The attacker can send arbitrary calldata to the Safe.
- (d). The attacker provides 16-byte calldata (selector + 12 filler) so the Safe's 20-byte append yields exactly 36 bytes to the kernel.
- (e). configs[account].manager != attacker and newManager != address(0).

Scenario 2.

Forced fee collection within policy bounds after takeover of manager role: post-rotation, attacker calls collectFees directly as manager with valid parameters; kernel pays treasury, distributor, and the fee policy's feeRecipient; no



funds go to the attacker, and amounts are bounded by the policy.

Preconditions / Assumptions

- (a). All preconditions from Scenario 1 hold (attacker is now the account's manager).
- (b). The account has a nonzero, governance-trusted feePolicy configured and a feeAsset set.
- (c). The fee policy is initialized (e.g., HWM seeded for StandardFeePolicy) and sufficient time has elapsed to allow a positive computed maxFee.
- (d). The Safe has sufficient balance of the fee asset for transfers to succeed.

Scenario 3.

Repeated rotation grieving until fallback handler is fixed: attacker repeats the short-calldata call (using fresh addresses to avoid ManagerUnchanged) to continuously clear newly re-registered permissions, causing ongoing operational disruption and re-registration costs until the fallback handler is corrected.

Preconditions / Assumptions

- (a). All preconditions from Scenario 1 hold and the fallbackHandler remains pointed to the kernel.
- (b). The operator re-registers permissions before fixing fallback, allowing the attacker to re-trigger manager rotation (with a fresh address) and clear permissions again.

Warning #6

Hard-coded 1-unit minimal fee in StandardFeePolicy reattach window causes one-time over-collection for 0-decimal fee assets

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Acknowledged

In the first post-reattach window, when the management leg rounds to zero, the policy returns a minimal 1-base-unit fee to avoid a permanent zero-fee re-anchor deadlock. For a 0-decimal fee asset this is one whole token – a bounded, one-time amount paid to the normal authorized recipients, not an unauthorized party. Removing it would require a coupled kernel change to accept a zero-fee re-anchor, re-opening the trusted core we have frozen pre-launch. Given the impact is bounded and account-internal and 0-decimal fee assets are exotic, we are documenting the behavior rather than making a coupled kernel change.

In the first post-reattach collection window, StandardFeePolicy.computeFee can return a hard-coded maxFee of 1 base unit when pendingReanchor is true and the management leg rounds to zero. SailKernel accepts any grossFee 0 and grossFee <= maxFee and does not normalize by token decimals. Therefore, with a 0-decimal fee asset, the authorized one-time collection can be a full token. This is an intentional liveness tradeoff documented in the policy, but it can be economically non-dust for coarse/0-decimal assets.

file: contracts/policies/StandardFeePolicy.sol

```
289 // balance of the configured fee asset, even this 1-unit
    // transfer cannot settle and the account
290 // stays stuck until it is funded.

291 if (pendingReanchor[account] && managementFee == 0 &&
    elapsed >= MIN_COLLECTION_INTERVAL) {
292     return (1, _distributor, _distributorBps);
```

289

```
    // balance of the configured fee asset, even this 1-unit
    transfer cannot settle and the account
290 // stays stuck until it is funded.
291 +// FIXME(safety): Replace the 1-unit minimal fee with a
    zero-fee reanchor. Return (0, ...) here
292 +// and update the kernel to accept a zero-fee settlement
    ONLY when pendingReanchor(account)==true
293 +// and computeFee returns maxFee==0, then have
    recordCollection clear the latch and ratchet HWM.
294 +// This avoids a one-time 1-unit charge for coarse/0-decimal
    fee assets.
295   if (pendingReanchor[account] && managementFee == 0 &&
    elapsed >= MIN_COLLECTION_INTERVAL) {
296   return (1, _distributor, _distributorBps);
```

Severity

Impact Explanation: [Low] Bounded, one-time over-collection of exactly 1 base unit within the authorized fee envelope during a specific lifecycle window; no unauthorized recipient and no core invariant violation.

Likelihood Explanation: [Low] Requires privileged lifecycle actions (permissionSigner reattach and manager/Safe collect), elapsed ≥ 1 day, HWM pre-seeded, and a 0-decimal fee asset; multiple specific preconditions reduce likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1: The account uses StandardFeePolicy with a 0-decimal ERC-20 fee asset and managementFeeBps = 0. The permissionSigner reattaches the same policy instance (setFeePolicy), which sets pendingReanchor = true. After at least 1 day, the manager (or the Safe) calls collectFees with grossFee = 1 and the configured feeToken. computeFee returns maxFee = 1 due to the reattach window and zero management leg; Kernel accepts the collection and transfers 1 token to the authorized recipients. pendingReanchor is cleared.

Preconditions / Assumptions

- (a). Account is registered with SailKernel and configured to use StandardFeePolicy
- (b). StandardFeePolicy.hwmSeeded[account] is true (feeManager seeded HWM)
- (c). configs[account].feeAsset is a 0-decimal ERC-20 token; the Safe holds at least 1 unit
- (d). configs[account].sessionActive is true and governance is not paused
- (e). permissionSigner calls setFeePolicy with the same policy instance and feeAsset, invoking onAttach
- (f). StandardFeePolicy.onAttach sets pendingReanchor[account] = true and resets lastCollectionTimestamp
- (g). At least MIN_COLLECTION_INTERVAL (1 day) elapses
- (h). managementFeeBps = 0 so the management leg is 0



- (i). Manager or the Safe calls collectFees(account, grossFee = 1, currentNav = any, feeToken = configured asset)
- (j). SailKernel enforces grossFee > 0, feeToken matches, and grossFee 0 but small enough that the first window's management leg floors to 0 at the current NAV
- (i). Manager or the Safe calls collectFees(account, grossFee = 1, currentNav consistent with flooring, feeToken = configured asset)
- (j). SailKernel enforces grossFee > 0, feeToken matches, and grossFee <= maxFee (=1), then calls recordCollection and performs transfers

Warning #7

Confused-deputy via Safe fallback relay in SailGovernance.pause/unpause causes unauthorized pause/unpause and temporary protocol DoS

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Resolved

pause() and unpause() now reject any call whose calldata is not exactly the 4-byte selector. Because a Safe fallback relay appends the caller's 20 bytes, and these functions take no arguments, a relayed call is always ≥ 24 bytes and can never equal 4 – the guard is unbypassable. This closes the relay path (relevant because the emergency admin is a multisig), including the unpause-then-cooldown-lock-out scenario, with no change to the deliberate cooldown behavior. Fixed and merged.

If the emergencyAdmin is a Safe whose fallback handler is (mis)configured to point at the SailGovernance contract (or an equivalent forwarder that preserves `msg.sender == Safe`), any external caller can invoke pause/unpause through the Safe's fallback. Because pause/unpause only check `msg.sender == emergencyAdmin` and have no calldata-length guards, unauthorized outsiders can pause or unpause. Since unpause() does not reset `lastPauseTimestamp`, an attacker can immediately lift a legitimate pause and prevent re-pausing for the remainder of the 72-hour cooldown.

SailGovernance.authorizes pause/unpause using only `msg.sender == emergencyAdmin`. Gnosis Safe's FallbackManager forwards unmatched calls to its configured fallback handler with `msg.sender` set to the Safe and appends 20 bytes to calldata. If the emergencyAdmin Safe is misconfigured to use SailGovernance as its fallback handler (or an equivalent handler that forwards into SailGovernance such that `msg.sender` remains the Safe), any external user can send a call to the Safe with the pause() or unpause() selector. The Safe forwards to SailGovernance, which accepts the call because onlyEmergencyAdmin passes (`msg.sender == the Safe`). Solidity accepts the selector despite the extra 20 bytes because pause/unpause have no calldata-length guards. This enables unauthorized pause (up to 72h temporary DoS) or unauthorized unpause right after a legitimate pause. unpause() intentionally does not reset `lastPauseTimestamp`, so an attacker can immediately unpause and lock the real admin out of re-pausing until the cooldown expires, undermining incident response. Rotating emergencyAdmin requires the 48-hour timelock and cannot fix the misconfiguration instantly. The attack relies on an uncommon operator configuration of the admin Safe; once present, exploitation is trivial.

```

390     error TimelockNotSelfAdministered();
391
392     //
-----
393     // Modifiers
...
644     ///         Subject to a PAUSE_COOLDOWN between consecutive
calls to prevent spam.
645     function pause() external onlyEmergencyAdmin {
646         if (lastPauseTimestamp != 0 && block.timestamp <
lastPauseTimestamp + PAUSE_COOLDOWN)
647             revert PauseCooldown(lastPauseTimestamp +
PAUSE_COOLDOWN);
...
659     ///         revoke* family) are pause-exempt regardless, so
this never blocks owner recovery.
660     function unpause() external onlyEmergencyAdmin {
661         pauseExpiry = 0;
662         emit Unpaused();

```

```

390     error TimelockNotSelfAdministered();
391
392 + /// @dev Thrown by `pause`/`unpause` when calldata length is
not exactly 4 bytes (selector only).
393 + error UnexpectedCalldataLength();
394 +
395     //
-----
396     // Modifiers
...
647

```

```
    ///          Subject to a PAUSE_COOLDOWN between consecutive
    calls to prevent spam.
648  function pause() external onlyEmergencyAdmin {
649 + if (msg.data.length != 4) revert UnexpectedCalldataLength();
650  if (lastPauseTimestamp != 0 && block.timestamp <
    lastPauseTimestamp + PAUSE_COOLDOWN)
651  revert PauseCooldown(lastPauseTimestamp +
    PAUSE_COOLDOWN);

    ...

663  ///          revoke* family) are pause-exempt regardless, so
    this never blocks owner recovery.
664  function unpause() external onlyEmergencyAdmin {
665 + if (msg.data.length != 4) revert UnexpectedCalldataLength();
666  pauseExpiry = 0;
667  emit Unpaused();
```

Severity

Impact Explanation: [Medium] Attackers can cause significant but temporary availability loss of core protocol functionality (pause for up to 72h) or negate the emergency pause lever and prevent re-pausing for the remainder of the cooldown. No permanent loss of funds or broken invariants is shown.

Likelihood Explanation: [Low] Exploitation depends on an uncommon privileged-wallet configuration: the emergencyAdmin must be a Safe whose fallback handler points to SailGovernance (or an equivalent forwarder). Once present, exploitation is trivial, but the precondition is a rare operator mistake.

Exploitation

Exploitation Scenarios:

Scenario 1.

Unauthorized unpause and cooldown lockout: After the real admin pauses, an attacker calls the emergencyAdmin Safe with the unpause() selector. The Safe's fallback forwards to SailGovernance with msg.sender == the Safe; unpause() runs, sets pauseExpiry = 0, and leaves lastPauseTimestamp unchanged, preventing re-pausing for up to 72 hours.

Preconditions / Assumptions

- (a). emergencyAdmin is a Safe multisig (S).
- (b). S's fallback handler is set to the SailGovernance contract (or an equivalent handler that forwards into SailGovernance with msg.sender == S).
- (c). SailGovernance.pause/unpause have no calldata-length guards.
- (d). The protocol is currently paused (lastPauseTimestamp set; pauseExpiry > 0).
- (e). Attacker can send a transaction to the Safe with the unpause() selector.

Scenario 2.

Unauthorized pause causing up to 72-hour protocol-wide DoS: An attacker calls the emergencyAdmin Safe with the



pause() selector. The Safe forwards to SailGovernance; pause() runs and sets pauseExpiry = now + 72h, closing whenNotPaused flows until the admin intervenes or expiry elapses.

Preconditions / Assumptions

- (a). emergencyAdmin is a Safe multisig (S).
- (b). S's fallback handler is set to the SailGovernance contract (or an equivalent handler that forwards into SailGovernance with msg.sender == S).
- (c). SailGovernance.pause/unpause have no calldata-length guards.
- (d). Attacker can send a transaction to the Safe with the pause() selector.

Scenario 3.

Same-block sabotage of a legitimate pause: The real admin's pause() is broadcast; the attacker back-runs in the same block with unpause() via the Safe fallback. Both execute in sequence (Paused then Unpaused). End state is unpaused and the admin cannot re-pause until the cooldown window ends.

Preconditions / Assumptions

- (a). emergencyAdmin is a Safe multisig (S).
- (b). S's fallback handler is set to the SailGovernance contract (or an equivalent handler that forwards into SailGovernance with msg.sender == S).
- (c). SailGovernance.pause/unpause have no calldata-length guards.
- (d). The admin's legitimate pause() is pending in mempool.
- (e). MEV or block builder ordering places attacker's unpause() (via the Safe fallback) after the admin's pause() in the same block.

Warning #8

Missing allowlist bounds/validation in SwapPermission/SwapPermissionNoOracle configuration causes recoverable reconfiguration DoS

Severity: LOW

Likelihood: LOW

Impact: LOW

Status: Resolved

Both swap templates now cap the router/tokenIn/tokenOut allowlists at the same MAX_ALLOWLIST_LENGTH (50) already enforced by the other launch templates, reverting on oversize configs – this is a parity fix bringing the two templates in line with the established pattern. No separate referencePools cap was added: the allowlist cap already bounds the cross-product (and thus the reconfiguration gas), and a separate cap would have rejected valid dense configurations below the coverage requirement. Fixed and merged.

SwapPermission and SwapPermissionNoOracle accept arbitrary-length routers/tokensIn/tokensOut arrays with no caps, empty/zero checks, or de-duplication. Large per-account configs can make later reconfiguration exceed gas. After a registrationEpoch bump (e.g., revoke/replace/manager rotation), evaluate() denies until a fresh configure succeeds; if reconfigure reverts due to gas, the operator must migrate to a fresh permission address and replace the old one. Funds remain safe; impact is recoverable and localized.

The swap templates (SwapPermission and SwapPermissionNoOracle) store per-account allowlists in template storage and apply configurations via `_applyConfig` without MAX_ALLOWLIST_LENGTH, zero-address, empty-array, or duplicate filtering. Each reconfigure first iterates and clears all prior mapping entries by walking the stored arrays, then writes the new mappings and arrays. With very large unique entries, a subsequent configure can exceed the block gas limit (clear-then-write cost), reverting and leaving the old config in place. Because this config is stamped against `kernel.registrationEpoch`, any later epoch bump (triggered by revoke/replace/manager rotation) makes evaluate() deny until a fresh configure succeeds. If that configure repeatedly reverts due to gas, the permission address becomes unusable until the operator registers a fresh deployment of the template, configures it with reasonable arrays, and atomically replaces the old permission via `replacePermission`. Only the permissionSigner can configure; there is no unprivileged attack path. Zero/empty entries do not create unsafe execution due to kernel and router semantics; funds remain safe. Gas limits naturally cap worst-case sizes, and the system provides a practical migration path, keeping the effect operator-recoverable and localized.

file: contracts/templates/SwapPermission.sol



```

85     uint256 private constant LEN_V3_V2  = 228;
86     uint256 private constant LEN_V2_MIN = 196;

87
88     struct Slot {

...

109     ///             deliberately want no on-chain band must use
SwapPermissionNoOracle instead.
110     error OracleRequired();

111
112     constructor(address _kernel, address _author)

...

156     // arbitrarily stale prices and re-open the staleness gap
the oracle is meant to close.
157     if (maxPriceAgeSec == 0) revert MissingPriceAge();

158
159     // Clear previous allowlists for this account

```

```

85     uint256 private constant LEN_V3_V2  = 228;
86     uint256 private constant LEN_V2_MIN = 196;
87 + uint256 private constant MAX_ALLOWLIST_LENGTH = 50;
88
89     struct Slot {

...

110     ///             deliberately want no on-chain band must use
SwapPermissionNoOracle instead.
111     error OracleRequired();
112 + error AllowlistTooLong();
113
114     constructor(address _kernel, address _author)

...

158     // arbitrarily stale prices and re-open the staleness gap
the oracle is meant to close.

```



```
159     if (maxPriceAgeSec == 0) revert MissingPriceAge();
160 +   if (routers.length > MAX_ALLOWLIST_LENGTH || to-
        kensIn.length > MAX_ALLOWLIST_LENGTH || tokensOut.length > MAX_AL-
        LOWLIST_LENGTH) revert AllowlistTooLong();
161
162 // Clear previous allowlists for this account
```

Severity

Impact Explanation: [Low] Recoverable liveness loss/DoS for a single permission address (and UX friction); no fund loss; confined to the account's beneficiary set; resolved by signer/operator migration (register new permission and replace).

Likelihood Explanation: [Low] Requires trusted-role (permissionSigner) configuration with abnormally large arrays and a later epoch bump; gas limits naturally constrain worst-case sizes; no external attacker path.

Exploitation

Exploitation Scenarios:

Scenario 1.

Operator configures SwapPermission with very large routers/tokensIn/tokensOut arrays that narrowly fit block gas. Later they attempt to update the config: the clear-then-write loops exceed gas and the transaction reverts. Subsequently, a revoke/replace or manager rotation bumps registrationEpoch; evaluate() now denies until a fresh configure succeeds, which still reverts due to gas. The operator restores liveness by registering a fresh permission deployment, configuring it with reasonable arrays, and calling replacePermission to swap out the old address.

Preconditions / Assumptions

- (a). Account is registered with SailKernel
- (b). SwapPermission or SwapPermissionNoOracle is registered on the account
- (c). permissionSigner controls configuration and submits very large allowlists that narrowly fit gas limits
- (d). A subsequent revoke/replace or manager rotation bumps registrationEpoch for the permission

Scenario 2.

With large arrays already stored, off-chain calls to getConfig become heavy and unreliable (time-outs/UX friction). After a later epoch bump, any attempt to configure the same permission address reverts due to the cost of clearing the



large arrays; evaluate() denies. The operator resolves this by registering a fresh permission address, configuring it, and replacing the old permission.

Preconditions / Assumptions

- (a). Account is registered with SailKernel
- (b). SwapPermission or SwapPermissionNoOracle is registered and configured with very large arrays
- (c). Off-chain tools call getConfig (large return arrays)
- (d). A subsequent revoke/replace or manager rotation bumps registrationEpoch for the permission

Octane Security

Warning #9

Single shared raw `maxAmountPerTx` across mixed-decimal assets in `BorrowPermission` causes under-enforced per-borrow limits for low-decimal tokens

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

The borrow-side analogue of the `DepositPermission` cap finding: one raw per-transaction cap across mixed-decimal borrow assets under-constrains low-decimal assets. Same disposition – a documented oracle-free boundary, value account-internal, mitigated by operator cap-sizing or separate instances. No per-token caps added.

`BorrowPermission` applies one raw-unit `maxAmountPerTx` to all allowlisted assets without per-asset normalization. When mixed-decimal tokens are allowlisted together, a cap sized for an 18-decimal asset unintentionally permits much larger single-step borrows of 6-decimal assets. With oracles disabled, this becomes the main template-side quantitative bound (venue HF still applies). With oracles enabled, the LTV ceiling holds, but the independent per-step cap is effectively looser for low-decimals.

`BorrowPermission` stores a single `uint256` `maxAmountPerTx` per account in `Slot` and compares the decoded borrow amount directly against it across all supported protocol branches (Aave, Morpho, Compound V2). There is no per-asset cap or normalization by token decimals. In the amount-cap-only mode (both oracles unset), `_ltvCheck` returns true, so the shared raw cap is the only template-side quantitative limiter (besides the venue's own health-factor/liquidity). If `assets[]` contains heterogeneous decimals (e.g., WETH 18-decimals and USDC 6-decimals) and the operator sets `maxAmountPerTx` to `1e18` intending to allow "1 token" per step, the template will also allow up to `1e18` units of USDC per step (i.e., `1e12` USDC tokens) subject to the lending venue's HF. With both oracles enabled, the LTV ceiling is correctly and conservatively enforced, but the independent raw per-step cap remains looser for low-decimal assets, undermining the expectation of a uniform per-borrow size bound. The manager is modeled as potentially adversarial within registered permissions; funds are credited to the account (`onBehalfOf`/receiver checks), and any outflow still requires another permission. Thus, this is a policy under-enforcement design weakness in the template's amount-cap semantics, not an authorization bypass.

file: `contracts/templates/BorrowPermission.sol`



```

1  // SPDX-License-Identifier: GPL-2.0-or-later
2  pragma solidity 0.8.26;
3
4  import {Context} from "../interfaces/IPermission.sol";
5  import {IOracle} from "../interfaces/IOracle.sol";
6  import {IPermissionIntrospection} from "../interfaces/IPermissionIntrospection.sol";
7  import {SailCapabilities} from "../interfaces/SailCapabilities.sol";
8  import {ConfigurablePermission} from "../ConfigurablePermission.sol";
9  import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
10
11  /// @dev Minimal Compound V2 cToken surface: the underlying ERC-20
12  a cToken market wraps.
13  ///      Used to resolve the cToken (the borrow call target) to
14  its underlying borrow asset.
15  interface ICerc20 {
16      function underlying() external view returns (address);
17  }
18
19  /// @title BorrowPermission – bounded borrow with optional LTV
20  ceiling
21  /// @notice REFERENCE LAUNCH TEMPLATE – part of the hardened
22  reference set, NOT part of the
23  ///      trusted core. This is one of the seven hardened launch
24  templates.
25  ///      It is documented with the honest boundaries below
26  ///      ("what this cannot protect against"). It sits OUTSIDE
27  the trusted core
28  ///      (SailKernel, SailGovernance, MandateFactory,
29  StandardFeePolicy, SafeModuleEnabler):
30  ///      a bug here cannot reach the kernel or accounts that
31  have not registered it. The
32  ///      kernel evaluates any permission safely under staticcall
33  + a gas cap + fail-closed
34  ///      semantics, but it does NOT verify that this permission's
35  logic correctly enforces
36  ///      what its NatSpec claims, so registrants remain
37  responsible for reviewing it. The
38  ///      loud "UNAUDITED – EXPERIMENTAL" banner is reserved
39  for the future experimental template
40  ///      set (currently empty), not this hardened launch set.
41  See docs/SECURITY.md for the
42  ///      reference-template documentation.
43  ///

```

```

31  ///      WHAT IT IS. A reference borrow permission. One deployment
    serves any number of accounts;
32  ///      each stores its own protocol and asset allowlists, a
    per-tx amount cap, an LTV ceiling,
33  ///      and a pair of price oracles. Supports Aave V3, Morpho,
    and Compound V2 borrow selectors.
34  ///
35  ///      WHAT IT ENFORCES. For every borrow: the protocol (call
    target) and asset are
36  ///      allowlisted; the amount is within the per-tx cap; the
    position is credited to the
37  ///      account itself (onBehalfOf / receiver == account);
    and, when oracles are configured, the
38  ///      resulting loan-to-value is within maxLtvBps.
39  ///
40  ///      ORACLE MODES. Oracles are configured in matched pairs:
41  ///      - ZERO oracles 'amount-cap-only borrowing. NO LTV
    ceiling is applied; only the
42  ///      per-tx cap and the allowlists
    bound the borrow.
43  ///      - BOTH oracles 'the LTV ceiling is enforced against
    the collateral and borrow feeds.
44  ///      Exactly ONE oracle is rejected at configure()
    (OracleConfigInconsistent): loan-to-value
45  ///      is a ratio of borrow value to collateral value, and a
    single feed can price only one
46  ///      side, so a one-oracle config cannot compute a ratio
    and is a configuration error.
47  ///
48  ///      HONEST BOUNDARY – what it does NOT do. With zero
    oracles, ONLY the size cap applies –
49  ///      there is no LTV ceiling, despite maxLtvBps being stored.
    When oracles are set, the LTV
50  ///      ceiling is only as good as the configured oracles'
    honesty and freshness; it does NOT
51  ///      protect against a manipulated or compromised feed.
    The collateral oracle is trusted to
52  ///      report the account's aggregate collateral value (it
    is queried by account address). LTV
53  ///      is checked at borrow time only – it does NOT monitor
    ongoing position health after the
54  ///      borrow, and the cap is per-transaction, not cumulative.
    Likewise maxLtvBps bounds each
55  ///      borrow step's size against the collateral valuation
    at the moment of that call; it does
56  ///      NOT bound the cumulative LTV of a position built across
    multiple borrows (e.g. a leverage

```

```

57  ///          loop). For cumulative-position safety, rely on the
    lending protocol's own health factor
58  ///          and/or a separate position-monitoring permission.
59  ///
60  ///          LTV CEILING (fail-closed, amount-based). When both
    oracles are configured the ceiling is
61  ///          enforced as an amount-based bound: the maximum borrow
    amount is derived by applying
62  ///          maxLtvBps to the full-precision collateral value and
    collapsing the decimal scale last,
63  ///          then the borrow is required to be at or under that
    amount. A borrow worth less than one
64  ///          numeraire unit no longer rounds its value to zero and
    slips an LTV ceiling – the prior
65  ///          fail-open behaviour is closed. Missing/zero prices,
    out-of-range decimals, or
66  ///          a stale feed all deny. Because every step rounds in
    the borrower's disfavour, a borrow
67  ///          that is marginally WITHIN the true LTV ceiling may be
    rejected; this is the conservative,
68  ///          fail-closed direction (it never permits an over-LTV
    borrow). Recourse is to borrow
69  ///          slightly less.
70  ///
71  ///          ASSET RESOLUTION. The operator allowlists the UNDERLYING
    borrow asset. On the Compound
72  ///          path the call target is the cToken, but the amount
    and the allowlist/LTV are
73  ///          underlying-denominated, so the template resolves
    cToken.underlying() and keys both on the
74  ///          underlying. Targets with no underlying() (e.g. cETH)
    resolve nothing and are denied –
75  ///          fail-closed by design.
76  ///
77  ///          INTEREST-RATE MODE (Aave). Aave borrows are constrained
    to VARIABLE rate mode; a
78  ///          stable-rate borrow is rejected. This is the reference
    template's opinionated default –
79  ///          stable rate imposes an ongoing cost choice on the user
    that the manager should not make
80  ///          on the template's behalf – NOT a claim that variable
    is universally safer (in a rising
81  ///          rate environment a strategy may legitimately prefer a
    locked stable rate). A strategy
82  ///          that genuinely wants stable-rate debt should use a
    dedicated permission, not this one.
83

```

```

    ///      Trade-off: if a market's variable-rate borrowing is
    disabled/paused while stable is
84    ///      available, this template cannot borrow there at all –
    a bounded availability limitation,
85    ///      not a loss of funds (the block is fail-closed). Morpho
    and Compound borrows carry no
86    ///      Aave-style rate-mode argument and are unaffected.
87    ///
88    ///      REFERRAL CODE (Aave). The template does NOT constrain
    the Aave referral code; it is
89    ///      off-chain attribution on the account's own position
    and has no effect on funds,
90    ///      principal, or the resulting position. Whether to
    participate in a venue referral program
91    ///      is the manager's choice.
92    ///
93    ///      GAS BUDGET (operator note). This template's own evaluate
    cost is light, but the whole
94    ///      evaluation runs under the kernel's 150k PERMIS-
    SION_GAS_CAP. A heavy operator-supplied
95    ///      oracle adapter can push the evaluation over that cap,
    which fails closed (deny). Operators
96    ///      must budget their adapter's gas.
97    ///
98    ///      CONFIG FRESHNESS (fail-closed). Evaluation denies
    unless this account is configured AND
99    ///      its stored config epoch equals the kernel's current
    registration epoch for this
100    ///      (account, permission). A configuration left over from
    a prior registration – e.g. after a
101    ///      revoke / re-register cycle – is never honoured.
102    ///
103    /// @dev      Config blob:
104    ///          abi.encode(
105    ///              address[] protocols,
106    ///              address[] assets,
107    ///              uint256    maxAmountPerTx,
108    ///              uint256    maxLtvBps,
109    ///              address    collateralOracle,
110    ///              address    borrowOracle,
111    ///              uint256    maxPriceAgeSec
112    ///          )
113    contract BorrowPermission is ConfigurablePermission, IPermission-
    Introspection {
114        bytes4 private constant AAVE_BORROW      = bytes4(keccak256("bor-
        row(address,uint256,uint256,uint16,address)"));
115

```



```

    /// @dev `borrow(address,uint256,address,address)` – the Morpho
    Optimizer / Morpho-Aave style
116    ///      borrow ABI, NOT Morpho Blue (whose `borrow((Market-
    Params),uint256,uint256,address,address)`
117    ///      has a different selector and will simply not match
    here, i.e. fail closed). Operators
118    ///      targeting Morpho Blue need a Blue-specific permission,
    not this template.
119    bytes4 private constant MORPHO_BORROW    = bytes4(keccak256("bor-
    row(address,uint256,address,address)"));
120    bytes4 private constant COMPOUND_BORROW = bytes4(kec-
    cak256("borrow(uint256)"));
121
122    /// @dev Upper bound on each configured allowlist, mirroring
    the sibling templates.
123    uint256 private constant MAX_ALLOWLIST_LENGTH = 50;
124
125    /// @dev Aave interest-rate mode for variable-rate debt
    (stable-rate is 1). The Aave borrow
126    ///      branch constrains the decoded rate mode to this value
    – see the INTEREST-RATE MODE note.
127    uint256 private constant AAVE_VARIABLE_RATE = 2;
128
129    uint256 private constant LEN_AAVE        = 164;
130    uint256 private constant LEN_MORPHO     = 132;
131    uint256 private constant LEN_COMPOUND = 36;
132
133    struct Slot {
134        address[] protocols;
135        address[] assets;
136        uint256    maxAmountPerTx;
137        uint256    maxLtvBps;
138        address    collateralOracle;
139        address    borrowOracle;
140        uint256    maxPriceAgeSec;
141    }
142
143    mapping(address account => Slot) private _slots;
144    mapping(address account => mapping(address => bool)) public
    isAllowedProtocol;
145    mapping(address account => mapping(address => bool)) public
    isAllowedAsset;
146
147    /// @notice Tooling-layer attribution for the template author.
    The kernel never reads this.
148    address public immutable author;
149

```

```

150     error LtvBpsTooLarge(uint256 bps);
151     /// @notice Thrown when exactly one oracle is configured. LTV
    is a ratio of borrow value to
152     ///         collateral value; a single feed prices only one
    side and cannot form the ratio.
153     ///         Configure either zero oracles (amount-cap-only)
    or both.
154     error OracleConfigInconsistent();
155     /// @notice Thrown when a configured allowlist exceeds
    MAX_ALLOWLIST_LENGTH.
156     error AllowlistTooLong();
157     /// @notice Thrown when the protocols or assets allowlist is
    empty (would deny all borrows).
158     error EmptyAllowlist();
159
160     constructor(address _kernel, address _author)
161     ConfigurablePermission(_kernel, "BorrowPermission", "2")
162     {
163         author = _author;
164     }
165
166     function getConfig(address account)
167     external
168     view
169     returns (
170         address[] memory protocols,
171         address[] memory assets,
172         uint256 maxAmountPerTx,
173         uint256 maxLtvBps,
174         address collateralOracle,
175         address borrowOracle,
176         uint256 maxPriceAgeSec
177     )
178     {
179         Slot storage s = _slots[account];
180         return (s.protocols, s.assets, s.maxAmount-
            PerTx, s.maxLtvBps, s.collateralOracle, s.borrowOracle, s.max-
            PriceAgeSec);
181     }
182
183     function _applyConfig(address account, bytes calldata params)
    internal override {
184     (
185         address[] memory protocols,
186         address[] memory assets,
187         uint256 maxAmountPerTx,
188         uint256 maxLtvBps,

```

```

189     address collateralOracle,
190     address borrowOracle,
191     uint256 maxPriceAgeSec
192 ) = abi.decode(params, (address[], address[], uint256,
uint256, address, address, uint256));
193
194 // Config validation (reference-grade, matching the sibling
templates): bound both allowlists,
195 // reject empty arrays, and reject zero-address entries.
maxAmountPerTx == 0 stays allowed
196 // (fail-closed: it simply denies every borrow). A
zero-address asset combined with a
197 // non-reverting target whose underlying() returns
address(0) would otherwise pass the asset
198 // check, so this also closes that defense-in-depth gap.
199 if (protocols.length > MAX_ALLOWLIST_LENGTH || assets.length
> MAX_ALLOWLIST_LENGTH) revert AllowlistTooLong();
200 if (protocols.length == 0 || assets.length == 0) revert
EmptyAllowlist();
201 for (uint256 i; i < protocols.length; i++) if (protocols[i]
== address(0)) revert ZeroAddress();
202 for (uint256 i; i < assets.length; i++) if (assets[i] ==
address(0)) revert ZeroAddress();
203
204 if (maxLtvBps > 10_000) revert LtvBpsTooLarge(maxLtvBps);
205 // Oracles must come as a matched pair. LTV is a ratio of
borrow value to collateral value;
206 // a single feed can price only one side, so exactly one
oracle cannot form the ratio and is
207 // a meaningless config. Require zero (amount-cap-only)
or both – rejecting the in-between at
208 // config time, where it is debuggable, rather than silently
disabling the ceiling later.
209 bool colSet = collateralOracle != address(0);
210 bool borSet = borrowOracle != address(0);
211 if (colSet != borSet) revert OracleConfigInconsistent();
212 // When the oracles are set, a freshness bound is mandatory;
0 would silently accept
213 // arbitrarily stale prices and re-open the gap the oracle
is meant to close.
214 if (colSet && maxPriceAgeSec == 0) revert MissingPriceAge();
215
216 Slot storage s = _slots[account];
217 for (uint256 i; i < s.protocols.length; i++)
isAllowedProtocol[account][s.protocols[i]] = false;
218 for (uint256 i; i < s.assets.length; i++) isAllowedAs-
set[account][s.assets[i]] = false;

```

```

219
220     for (uint256 i; i < protocols.length; i++) isAllowedPro-
        tocol[account][protocols[i]] = true;
221     for (uint256 i; i < assets.length; i++)      isAllowedAs-
        set[account][assets[i]]          = true;
222
223     s.protocols          = protocols;
224     s.assets             = assets;
225     s.maxAmountPerTx     = maxAmountPerTx;
226     s.maxLtvBps          = maxLtvBps;
227     s.collateralOracle   = collateralOracle;
228     s.borrowOracle       = borrowOracle;
229     s.maxPriceAgeSec     = maxPriceAgeSec;
230 }
231
232 function evaluate(bytes calldata txData, Context calldata ctx)
    external view returns (bool) {
233     // Fail closed unless the stored config is current for
    // this registration epoch.
234     if (!_configCurrent(ctx.account, ctx.configEpoch)) return
        false;
235     // No supported borrow selector is payable – reject native
    ETH.
236     if (ctx.value != 0) return false;
237     if (!isAllowedProtocol[ctx.account][ctx.target]) return
        false;
238     Slot storage s = _slots[ctx.account];
239
240     if (ctx.selector == AAVE_BORROW) {
241         if (txData.length < LEN_AAVE) return false;
242         // referralCode (4th arg) stays decoded-but-uncon-
        strained: attribution only, no fund effect.
243         (address asset, uint256 amount, uint256 rateMode, ,
        address onBehalfOf) =
244         abi.decode(txData[4:], (address, uint256, uint256,
        uint16, address));
245         // Aave: enforce variable-rate borrowing only;
        stable-rate debt is rejected.
246         if (rateMode != AAVE_VARIABLE_RATE)          return false;
247         if (!isAllowedAsset[ctx.account][asset]) return false;
248         if (amount > s.maxAmountPerTx)                return false;
249         if (onBehalfOf != ctx.account)                return false;
250         return _ltvCheck(s, asset, amount, ctx.account);
251     }
252
253     if (ctx.selector == MORPHO_BORROW) {
254         if (txData.length < LEN_MORPHO) return false;

```

```

255 (address asset, uint256 amount, address onBehalf,
    address receiver) =
256   abi.decode(txData[4:], (address, uint256, address,
    address));
257   if (!isAllowedAsset[ctx.account][asset]) return false;
258   if (amount > s.maxAmountPerTx) return false;
259   if (onBehalf != ctx.account) return false;
260   if (receiver != ctx.account) return false;
261   return _ltvCheck(s, asset, amount, ctx.account);
262 }
263
264 if (ctx.selector == COMPOUND_BORROW) {
265   if (txData.length < LEN_COMPOUND) return false;
266   uint256 amount = abi.decode(txData[4:], (uint256));
267   // The Compound borrow target is the cToken, but
   `amount` and the allowlist/oracle are
268   // denominated in the UNDERLYING asset. Resolve it
   via cToken.underlying() and key both
269   // the allowlist and the LTV check on the underlying.
270   // Targets that don't expose underlying() (e.g. cETH)
   resolve nothing and are denied – fail-closed by design.
271   address underlying;
272   try ICERC20(ctx.target).underlying() returns (address
   u) { underlying = u; }
273   catch { return false; }
274   if (!isAllowedAsset[ctx.account][underlying]) return
   false;
275   if (amount > s.maxAmountPerTx) return
   false;
276   return _ltvCheck(s, underlying, amount, ctx.account);
277 }
278
279 return false;
280 }
281
282 function discriminator() external pure returns (bytes32) {
283   return keccak256("BorrowPermission");
284 }
285
286 function _ltvCheck(Slot storage s, address asset, uint256
   amount, address account)
287   internal
288   view
289   returns (bool)
290 {
291   // Defense-in-depth: configure() rejects a single-oracle
   config, so reaching here with one

```

```

292 // oracle unset means BOTH are unset – the amount-cap-only
    mode, where no LTV ceiling applies.
293 if (s.collateralOracle == address(0) || s.borrowOracle ==
    address(0)) return true;
294
295 // On L2s, check sequencer-uptime first.
296 (uint256 colValue, uint8 colDec, uint256 colUpdatedAt) =
    IOracle(s.collateralOracle).getPrice(account, address(0));
297 (uint256 borPrice, uint8 borDec, uint256 borUpdatedAt) =
    IOracle(s.borrowOracle).getPrice(asset, address(0));
298 if (s.maxPriceAgeSec > 0) {
299     if (colUpdatedAt == 0 || block.timestamp - colUpdatedAt
        > s.maxPriceAgeSec) return false;
300     if (borUpdatedAt == 0 || block.timestamp - borUpdatedAt
        > s.maxPriceAgeSec) return false;
301 }
302
303 if (colDec > 77 || borDec > 77) return false;
304 if (colValue == 0) return false;
305 if (borPrice == 0) return false;
306
307 // Enforce trueLTV <= maxLtvBps, i.e.
308 //     amount * borPrice / 10^borDec <= (maxLtvBps /
    10_000) * colValue / 10^colDec
309 // rearranged into the largest borrow amount the ceiling
    permits:
310 //     maxAmountAllowed = colValue * maxLtvBps * 10^borDec
    / (10_000 * 10^colDec * borPrice)
311 // and compared against the integer `amount` directly.
    Two properties matter:
312 // (1) Fail-CLOSED. Every step floors, all in the
    borrower's disfavour, so maxAmountAllowed
313 //     never exceeds the true ceiling and an over-LTV
    borrow can never slip through. This
314 //     also closes the prior bug where pre-flooring the
    borrow value to whole numeraire
315 //     units (borrowScaled) collapsed any sub-1-unit
    borrow to 0 and passed ANY ceiling.
316 // (2) No collateral truncation. The LTV fraction is
    applied to the full-precision colValue
317 //     mantissa FIRST; only then is the decimal scale
    collapsed by dividing by 10^colDec.
318 //     Folding 10^colDec in before taking the fraction
    would floor away fractional
319 //     collateral and over-block borrows that are genuinely
    within the ceiling.
320

```

```

    // colValue != 0, borPrice != 0 and decimals <= 77 are
    all guarded above, so every divisor is
321 // non-zero and 10^colDec / 10^borDec cannot overflow;
    mulDiv carries each product at full
322 // precision in a 512-bit intermediate.
323     uint256 scaledBound      = Math.mulDiv(colValue, s.maxLtvBps,
    10_000);
324     uint256 maxAmountAllowed = Math.mulDiv(scaledBound, 10 **
    uint256(borDec), 10 ** uint256(colDec)) / borPrice;
325     return amount <= maxAmountAllowed;
326 }
327
328 // IPermissionIntrospection
329
330     function permissionId() external pure override returns
    (bytes32) {
331         return keccak256("sail.permission.BorrowPermission.v1");
332     }
333
334     function permissionVersion() external pure override returns
    (bytes32) {
335         return keccak256("v1");
336     }
337
338     function metadataURI() external pure override returns (string
    memory) {
339         return "";
340     }
341
342     function capabilityIds() external pure override returns
    (bytes32[] memory ids) {
343         ids = new bytes32[](1);
344         ids[0] = SailCapabilities.BOUNDED_BORROW;
345     }
346 }

```

```

1 // SPDX-License-Identifier: GPL-2.0-or-later
2 pragma solidity 0.8.26;
3
4 import {Context} from "../interfaces/IPermission.sol";
5 import {IOracle} from "../interfaces/IOracle.sol";
6 import {IPermissionIntrospection} from "../interfaces/IPermissionIntrospection.sol";
7 import {SailCapabilities} from "../interfaces/SailCapabilities.sol";

```



```

8  import {ConfigurablePermission} from "../ConfigurablePermission.sol";
9  import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
10
11  /// @dev Minimal Compound V2 cToken surface: the underlying ERC-20
12  ///    a cToken market wraps.
13  ///    Used to resolve the cToken (the borrow call target) to
14  ///    its underlying borrow asset.
15  interface ICerc20 {
16      function underlying() external view returns (address);
17  }
18  /// @title BorrowPermission – bounded borrow with optional LTV
19  ///    ceiling
20  /// @notice REFERENCE LAUNCH TEMPLATE – part of the hardened
21  ///    reference set, NOT part of the
22  ///    trusted core. This is one of the seven hardened launch
23  ///    templates.
24  ///    It is documented with the honest boundaries below
25  ///    ("what this cannot protect against"). It sits OUTSIDE
26  ///    the trusted core
27  ///    (SailKernel, SailGovernance, MandateFactory,
28  ///    StandardFeePolicy, SafeModuleEnabler):
29  ///    a bug here cannot reach the kernel or accounts that
30  ///    have not registered it. The
31  ///    kernel evaluates any permission safely under staticcall
32  ///    + a gas cap + fail-closed
33  ///    semantics, but it does NOT verify that this permission's
34  ///    logic correctly enforces
35  ///    what its NatSpec claims, so registrants remain
36  ///    responsible for reviewing it. The
37  ///    loud "UNAUDITED – EXPERIMENTAL" banner is reserved
38  ///    for the future experimental template
39  ///    set (currently empty), not this hardened launch set.
40  ///    See docs/SECURITY.md for the
41  ///    reference-template documentation.
42  ///
43  ///    WHAT IT IS. A reference borrow permission. One deployment
44  ///    serves any number of accounts;
45  ///    each stores its own protocol and asset allowlists, a
46  ///    per-tx amount cap, an LTV ceiling,
47  ///    and a pair of price oracles. Supports Aave V3, Morpho,
48  ///    and Compound V2 borrow selectors.
49  ///
50  ///    WHAT IT ENFORCES. For every borrow: the protocol (call
51  ///    target) and asset are

```



```

    ///          allowlisted; the amount is within the per-tx cap; the
    position is credited to the
37    ///          account itself (onBehalfOf / receiver == account);
    and, when oracles are configured, the
38    ///          resulting loan-to-value is within maxLtvBps.
39    ///
40    ///          ORACLE MODES. Oracles are configured in matched pairs:
41    ///          - ZERO oracles 'amount-cap-only borrowing. NO LTV
    ceiling is applied; only the
42    ///          per-tx cap and the allowlists
    bound the borrow.
43    ///          - BOTH oracles 'the LTV ceiling is enforced against
    the collateral and borrow feeds.
44    ///          Exactly ONE oracle is rejected at configure()
    (OracleConfigInconsistent): loan-to-value
45    ///          is a ratio of borrow value to collateral value, and a
    single feed can price only one
46    ///          side, so a one-oracle config cannot compute a ratio
    and is a configuration error.
47    ///
48    ///          HONEST BOUNDARY – what it does NOT do. With zero
    oracles, ONLY the size cap applies –
49    ///          there is no LTV ceiling, despite maxLtvBps being stored.
    When oracles are set, the LTV
50    ///          ceiling is only as good as the configured oracles'
    honesty and freshness; it does NOT
51    ///          protect against a manipulated or compromised feed.
    The collateral oracle is trusted to
52    ///          report the account's aggregate collateral value (it
    is queried by account address). LTV
53    ///          is checked at borrow time only – it does NOT monitor
    ongoing position health after the
54    ///          borrow, and the cap is per-transaction, not cumulative.
    Likewise maxLtvBps bounds each
55    ///          borrow step's size against the collateral valuation
    at the moment of that call; it does
56    ///          NOT bound the cumulative LTV of a position built across
    multiple borrows (e.g. a leverage
57    ///          loop). For cumulative-position safety, rely on the
    lending protocol's own health factor
58    ///          and/or a separate position-monitoring permission.
59    ///
60    ///          LTV CEILING (fail-closed, amount-based). When both
    oracles are configured the ceiling is
61    ///          enforced as an amount-based bound: the maximum borrow
    amount is derived by applying
62

```

```

    ///          maxLtvBps to the full-precision collateral value and
    collapsing the decimal scale last,
63    ///          then the borrow is required to be at or under that
    amount. A borrow worth less than one
64    ///          numeraire unit no longer rounds its value to zero and
    slips an LTV ceiling – the prior
65    ///          fail-open behaviour is closed. Missing/zero prices,
    out-of-range decimals, or
66    ///          a stale feed all deny. Because every step rounds in
    the borrower's disfavour, a borrow
67    ///          that is marginally WITHIN the true LTV ceiling may be
    rejected; this is the conservative,
68    ///          fail-closed direction (it never permits an over-LTV
    borrow). Recourse is to borrow
69    ///          slightly less.
70    ///
71    ///          ASSET RESOLUTION. The operator allowlists the UNDERLYING
    borrow asset. On the Compound
72    ///          path the call target is the cToken, but the amount
    and the allowlist/LTV are
73    ///          underlying-denominated, so the template resolves
    cToken.underlying() and keys both on the
74    ///          underlying. Targets with no underlying() (e.g. cETH)
    resolve nothing and are denied –
75    ///          fail-closed by design.
76    ///
77    ///          INTEREST-RATE MODE (Aave). Aave borrows are constrained
    to VARIABLE rate mode; a
78    ///          stable-rate borrow is rejected. This is the reference
    template's opinionated default –
79    ///          stable rate imposes an ongoing cost choice on the user
    that the manager should not make
80    ///          on the template's behalf – NOT a claim that variable
    is universally safer (in a rising
81    ///          rate environment a strategy may legitimately prefer a
    locked stable rate). A strategy
82    ///          that genuinely wants stable-rate debt should use a
    dedicated permission, not this one.
83    ///          Trade-off: if a market's variable-rate borrowing is
    disabled/paused while stable is
84    ///          available, this template cannot borrow there at all –
    a bounded availability limitation,
85    ///          not a loss of funds (the block is fail-closed). Morpho
    and Compound borrows carry no
86    ///          Aave-style rate-mode argument and are unaffected.
87    ///
88

```

```

    ///          REFERRAL CODE (Aave). The template does NOT constrain
    the Aave referral code; it is
89    ///          off-chain attribution on the account's own position
    and has no effect on funds,
90    ///          principal, or the resulting position. Whether to
    participate in a venue referral program
91    ///          is the manager's choice.
92    ///
93    ///          GAS BUDGET (operator note). This template's own evaluate
    cost is light, but the whole
94    ///          evaluation runs under the kernel's 150k PERMIS-
    SION_GAS_CAP. A heavy operator-supplied
95    ///          oracle adapter can push the evaluation over that cap,
    which fails closed (deny). Operators
96    ///          must budget their adapter's gas.
97    ///
98    ///          CONFIG FRESHNESS (fail-closed). Evaluation denies
    unless this account is configured AND
99    ///          its stored config epoch equals the kernel's current
    registration epoch for this
100    ///          (account, permission). A configuration left over from
    a prior registration – e.g. after a
101    ///          revoke / re-register cycle – is never honoured.
102    ///
103    /// @dev      Config blob:
104    ///          abi.encode(
105    ///              address[] protocols,
106    ///              address[] assets,
107    ///              uint256    maxAmountPerTx,
108    ///              uint256    maxLtvBps,
109    ///              address    collateralOracle,
110    ///              address    borrowOracle,
111    ///              uint256    maxPriceAgeSec
112    ///          )
113    contract BorrowPermission is ConfigurablePermission, IPermission-
    Introspection {
114        bytes4 private constant AAVE_BORROW      = bytes4(keccak256("bor-
    row(address,uint256,uint256,uint16,address)"));
115    /// @dev `borrow(address,uint256,address,address)` – the Morpho
    Optimizer / Morpho-Aave style
116    ///          borrow ABI, NOT Morpho Blue (whose `borrow((Market-
    Params),uint256,uint256,address,address)`
117    ///          has a different selector and will simply not match
    here, i.e. fail closed). Operators
118    ///          targeting Morpho Blue need a Blue-specific permission,
    not this template.
119

```

```

    bytes4 private constant MORPHO_BORROW = bytes4(keccak256("bor-
row(address,uint256,address,address)"));
120  bytes4 private constant COMPOUND_BORROW = bytes4(kec-
cak256("borrow(uint256)"));
121
122  /// @dev Upper bound on each configured allowlist, mirroring
the sibling templates.
123  uint256 private constant MAX_ALLOWLIST_LENGTH = 50;
124
125  /// @dev Aave interest-rate mode for variable-rate debt
(stable-rate is 1). The Aave borrow
126  ///      branch constrains the decoded rate mode to this value
– see the INTEREST-RATE MODE note.
127  uint256 private constant AAVE_VARIABLE_RATE = 2;
128
129  uint256 private constant LEN_AAVE      = 164;
130  uint256 private constant LEN_MORPHO    = 132;
131  uint256 private constant LEN_COMPOUND = 36;
132
133  struct Slot {
134  address[] protocols;
135  address[] assets;
136  uint256   maxAmountPerTx;
137  uint256   maxLtvBps;
138  address   collateralOracle;
139  address   borrowOracle;
140  uint256   maxPriceAgeSec;
141  }
142
143  mapping(address account => Slot) private _slots;
144  mapping(address account => mapping(address => bool)) public
isAllowedProtocol;
145  mapping(address account => mapping(address => bool)) public
isAllowedAsset;
146
147  /// @notice Tooling-layer attribution for the template author.
The kernel never reads this.
148  address public immutable author;
149
150  error LtvBpsTooLarge(uint256 bps);
151  /// @notice Thrown when exactly one oracle is configured. LTV
is a ratio of borrow value to
152  ///      collateral value; a single feed prices only one
side and cannot form the ratio.
153  ///      Configure either zero oracles (amount-cap-only)
or both.
154  error OracleConfigInconsistent();

```

```

155  /// @notice Thrown when a configured allowlist exceeds
    MAX_ALLOWLIST_LENGTH.
156  error AllowlistTooLong();
157  /// @notice Thrown when the protocols or assets allowlist is
    empty (would deny all borrows).
158  error EmptyAllowlist();
159
160  constructor(address _kernel, address _author)
161  ConfigurablePermission(_kernel, "BorrowPermission", "2")
162  {
163      author = _author;
164  }
165
166  function getConfig(address account)
167  external
168  view
169  returns (
170      address[] memory protocols,
171      address[] memory assets,
172      uint256 maxAmountPerTx,
173      uint256 maxLtvBps,
174      address collateralOracle,
175      address borrowOracle,
176      uint256 maxPriceAgeSec
177  )
178  {
179      Slot storage s = _slots[account];
180      return (s.protocols, s.assets, s.maxAmount-
        PerTx, s.maxLtvBps, s.collateralOracle, s.borrowOracle, s.max-
        PriceAgeSec);
181  }
182
183  function _applyConfig(address account, bytes calldata params)
    internal override {
184      (
185          address[] memory protocols,
186          address[] memory assets,
187          uint256 maxAmountPerTx,
188          uint256 maxLtvBps,
189          address collateralOracle,
190          address borrowOracle,
191          uint256 maxPriceAgeSec
192      ) = abi.decode(params, (address[], address[], uint256,
        uint256, address, address, uint256));
193
194      // Config validation (reference-grade, matching the sibling
        templates): bound both allowlists,

```

```

195 // reject empty arrays, and reject zero-address entries.
    maxAmountPerTx == 0 stays allowed
196 // (fail-closed: it simply denies every borrow). A
    zero-address asset combined with a
197 // non-reverting target whose underlying() returns
    address(0) would otherwise pass the asset
198 // check, so this also closes that defense-in-depth gap.
199 if (protocols.length > MAX_ALLOWLIST_LENGTH || assets.length
    > MAX_ALLOWLIST_LENGTH) revert AllowlistTooLong();
200 if (protocols.length == 0 || assets.length == 0) revert
    EmptyAllowlist();
201 for (uint256 i; i < protocols.length; i++) if (protocols[i]
    == address(0)) revert ZeroAddress();
202 for (uint256 i; i < assets.length; i++) if (assets[i] ==
    address(0)) revert ZeroAddress();
203
204 if (maxLtvBps > 10_000) revert LtvBpsTooLarge(maxLtvBps);
205 // Oracles must come as a matched pair. LTV is a ratio of
    borrow value to collateral value;
206 // a single feed can price only one side, so exactly one
    oracle cannot form the ratio and is
207 // a meaningless config. Require zero (amount-cap-only)
    or both – rejecting the in-between at
208 // config time, where it is debuggable, rather than silently
    disabling the ceiling later.
209 bool colSet = collateralOracle != address(0);
210 bool borSet = borrowOracle != address(0);
211 if (colSet != borSet) revert OracleConfigInconsistent();
212 // When the oracles are set, a freshness bound is mandatory;
    0 would silently accept
213 // arbitrarily stale prices and re-open the gap the oracle
    is meant to close.
214 if (colSet && maxPriceAgeSec == 0) revert MissingPriceAge();
215
216 Slot storage s = _slots[account];
217 for (uint256 i; i < s.protocols.length; i++)
    isAllowedProtocol[account][s.protocols[i]] = false;
218 for (uint256 i; i < s.assets.length; i++) isAllowedAs-
    set[account][s.assets[i]] = false;
219
220 for (uint256 i; i < protocols.length; i++) isAllowedPro-
    tocol[account][protocols[i]] = true;
221 for (uint256 i; i < assets.length; i++) isAllowedAs-
    set[account][assets[i]] = true;
222
223 s.protocols = protocols;
224 s.assets = assets;

```



```

225     s.maxAmountPerTx    = maxAmountPerTx;
226     s.maxLtvBps         = maxLtvBps;
227     s.collateralOracle  = collateralOracle;
228     s.borrowOracle      = borrowOracle;
229     s.maxPriceAgeSec    = maxPriceAgeSec;
230 }
231
232 function evaluate(bytes calldata txData, Context calldata ctx)
external view returns (bool) {
233 // Fail closed unless the stored config is current for
this registration epoch.
234 if (!_configCurrent(ctx.account, ctx.configEpoch)) return
false;
235 // No supported borrow selector is payable – reject native
ETH.
236 if (ctx.value != 0) return false;
237 if (!isAllowedProtocol[ctx.account][ctx.target]) return
false;
238 Slot storage s = _slots[ctx.account];
239
240 if (ctx.selector == AAVE_BORROW) {
241 if (txData.length < LEN_AAVE) return false;
242 // referralCode (4th arg) stays decoded-but-uncon-
strained: attribution only, no fund effect.
243 (address asset, uint256 amount, uint256 rateMode, ,
address onBehalfOf) =
244 abi.decode(txData[4:], (address, uint256, uint256,
uint16, address));
245 // Aave: enforce variable-rate borrowing only;
stable-rate debt is rejected.
246 if (rateMode != AAVE_VARIABLE_RATE) return false;
247 if (!isAllowedAsset[ctx.account][asset]) return false;
248 if (amount > s.maxAmountPerTx) return false;
249 if (onBehalfOf != ctx.account) return false;
250 return _ltvCheck(s, asset, amount, ctx.account);
251 }
252
253 if (ctx.selector == MORPHO_BORROW) {
254 if (txData.length < LEN_MORPHO) return false;
255 (address asset, uint256 amount, address onBehalf,
address receiver) =
256 abi.decode(txData[4:], (address, uint256, address,
address));
257 if (!isAllowedAsset[ctx.account][asset]) return false;
258 if (amount > s.maxAmountPerTx) return false;
259 if (onBehalf != ctx.account) return false;
260 if (receiver != ctx.account) return false;

```



```

261     return _ltvCheck(s, asset, amount, ctx.account);
262 }
263
264 if (ctx.selector == COMPOUND_BORROW) {
265     if (txData.length < LEN_COMPOUND) return false;
266     uint256 amount = abi.decode(txData[4:], (uint256));
267     // The Compound borrow target is the cToken, but
268     // `amount` and the allowlist/oracle are
269     // // denominated in the UNDERLYING asset. Resolve it
270     // // via cToken.underlying() and key both
271     // // the allowlist and the LTV check on the underlying.
272     // // Targets that don't expose underlying() (e.g. cETH)
273     // // resolve nothing and are denied – fail-closed by design.
274     address underlying;
275     try ICERC20(ctx.target).underlying() returns (address
276     u) { underlying = u; }
277     catch { return false; }
278     if (!isAllowedAsset[ctx.account][underlying]) return
279     false;
280     if (amount > s.maxAmountPerTx) return
281     false;
282     return _ltvCheck(s, underlying, amount, ctx.account);
283 }
284
285 return false;
286 }
287
288 function discriminator() external pure returns (bytes32) {
289     return keccak256("BorrowPermission");
290 }
291
292 function _ltvCheck(Slot storage s, address asset, uint256
293 amount, address account)
294 internal
295 view
296 returns (bool)
297 {
298     // Defense-in-depth: configure() rejects a single-oracle
299     // config, so reaching here with one
300     // oracle unset means BOTH are unset – the amount-cap-only
301     // mode, where no LTV ceiling applies.
302     if (s.collateralOracle == address(0) || s.borrowOracle ==
303     address(0)) return true;
304
305     // On L2s, check sequencer-uptime first.
306     (uint256 colValue, uint8 colDec, uint256 colUpdatedAt) =
307     IOracle(s.collateralOracle).getPrice(account, address(0));

```



```

297 (uint256 borPrice, uint8 borDec, uint256 borUpdatedAt) =
    IOracle(s.borrowOracle).getPrice(asset, address(0));
298 if (s.maxPriceAgeSec > 0) {
299     if (colUpdatedAt == 0 || block.timestamp - colUpdatedAt
    > s.maxPriceAgeSec) return false;
300     if (borUpdatedAt == 0 || block.timestamp - borUpdatedAt
    > s.maxPriceAgeSec) return false;
301 }
302
303 if (colDec > 77 || borDec > 77) return false;
304 if (colValue == 0) return false;
305 if (borPrice == 0) return false;
306
307 // Enforce trueLTV <= maxLtvBps, i.e.
308 //     amount * borPrice / 10^borDec <= (maxLtvBps /
    10_000) * colValue / 10^colDec
309 // rearranged into the largest borrow amount the ceiling
    permits:
310 //     maxAmountAllowed = colValue * maxLtvBps * 10^borDec
    / (10_000 * 10^colDec * borPrice)
311 // and compared against the integer `amount` directly.
    Two properties matter:
312 // (1) Fail-CLOSED. Every step floors, all in the
    borrower's disfavour, so maxAmountAllowed
313 //     never exceeds the true ceiling and an over-LTV
    borrow can never slip through. This
314 //     also closes the prior bug where pre-flooring the
    borrow value to whole numeraire
315 //     units (borrowScaled) collapsed any sub-1-unit
    borrow to 0 and passed ANY ceiling.
316 // (2) No collateral truncation. The LTV fraction is
    applied to the full-precision colValue
317 //     mantissa FIRST; only then is the decimal scale
    collapsed by dividing by 10^colDec.
318 //     Folding 10^colDec in before taking the fraction
    would floor away fractional
319 //     collateral and over-block borrows that are genuinely
    within the ceiling.
320 // colValue != 0, borPrice != 0 and decimals <= 77 are
    all guarded above, so every divisor is
321 // non-zero and 10^colDec / 10^borDec cannot overflow;
    mulDiv carries each product at full
322 // precision in a 512-bit intermediate.
323 uint256 scaledBound = Math.mulDiv(colValue, s.maxLtvBps,
    10_000);
324 uint256 maxAmountAllowed = Math.mulDiv(scaledBound, 10 **
    uint256(borDec), 10 ** uint256(colDec)) / borPrice;

```

```
325     return amount <= maxAmountAllowed;
326 }
327
328 // IPermissionIntrospection
329
330     function permissionId() external pure override returns
331     (bytes32) {
332         return keccak256("sail.permission.BorrowPermission.v1");
333     }
334
335     function permissionVersion() external pure override returns
336     (bytes32) {
337         return keccak256("v1");
338     }
339
340     function metadataURI() external pure override returns (string
341     memory) {
342         return "";
343     }
344
345     function capabilityIds() external pure override returns
346     (bytes32[] memory ids) {
347         ids = new bytes32[](1);
348         ids[0] = SailCapabilities.BOUNDED_BORROW;
349     }
350 }
```

Severity

Impact Explanation: [Low] This is a policy under-enforcement/design weakness: a single raw-unit per-tx cap across heterogeneous assets can permit larger-than-intended low-decimal borrows per step. Funds remain within the account; venue HF/liquidity (and, when enabled, the LTV ceiling) still bound safety. No direct theft, freeze, or external victim.

Likelihood Explanation: [Medium] The configuration (multi-asset allowlists with a single shared raw cap, and for Scenarios 1/3, disabling oracles) is supported and plausible in normal operation. While it reflects operator responsibility, it is not rare or exceptional; thus, the scenario is uncommon but realistic rather than rare.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (Aave/Morpho, oracles disabled): The permissionSigner configures BorrowPermission with assets[] including WETH (18) and USDC (6) and sets maxAmountPerTx = 1e18. Oracles are unset (amount-cap-only). The manager submits an Aave/Morpho borrow of USDC for 1,000,000e6. evaluate() allows it because 1e12 < 1e18 and _ltvCheck returns true when both oracles are unset; Aave/Morpho HF/liquidity still gate the final size. Result: a much larger-than-intended single-step USDC borrow is authorized by the template.

Preconditions / Assumptions

- (a). BorrowPermission is registered for the account and its config epoch is current
- (b). assets[] contains mixed-decimal tokens (e.g., WETH 18-decimals and USDC 6-decimals), and both are allowlisted
- (c). Aave/Morpho protocol addresses are allowlisted
- (d). maxAmountPerTx is set to a large raw value aligned with an 18-decimal asset (e.g., 1e18)
- (e). Both collateralOracle and borrowOracle are unset



(amount-cap-only mode)

(f). The account has sufficient collateral and the venue's HF/liquidity allow the borrow

(g). Manager acts within the registered permission; onBehalfOf/receiver equals the account

Scenario 2.

Scenario 3 (Compound V2, oracles disabled): The permissionSigner allowlists cUSDC (protocol) and USDC (asset) with maxAmountPerTx = 1e18, oracles unset. The manager calls cUSDC.borrow(500,000e6). The template resolves underlying() to USDC and compares the raw amount to 1e18; it passes, and _ltvCheck returns true. Venue HF/liquidity still apply. Result: a large low-decimal borrow per step is permitted by the template.

Preconditions / Assumptions

(a). BorrowPermission is registered for the account and its config epoch is current

(b). assets[] includes USDC (6-decimals) and the Compound V2 cUSDC market is allowlisted as a protocol

(c). maxAmountPerTx is set to a large raw value aligned with an 18-decimal asset (e.g., 1e18)

(d). Both collateralOracle and borrowOracle are unset (amount-cap-only mode)

(e). The account has sufficient collateral and the venue's HF/liquidity allow the borrow

(f). Manager acts within the registered permission; cUSDC.underlying() resolves USDC; funds are credited to the account

Scenario 3.

Scenario 2 (oracles enabled): The permissionSigner enables both oracles and sets maxLtvBps and maxAmountPerTx = 1e18 while assets[] includes WETH (18) and USDC (6). The manager borrows USDC for 800,000e6. The LTV-derived maxAmountAllowed is respected and fresh; amount <= maxAmountPerTx also passes due to the large shared raw cap. Result: the LTV ceiling remains enforced, but the independent per-step raw cap is looser for low-decimals, undermining the operator's intent for a uniform per-step bound.

Preconditions / Assumptions

(a). BorrowPermission is registered for the account and its config epoch is current

(b). assets[] contains mixed-decimal tokens (e.g., WETH 18 and USDC 6), and both are allowlisted

(c). Both collateral and borrow oracles are configured and



fresh, with valid decimals and non-zero prices
(d). `maxLtvBps` is set to a reasonable ceiling
(e). `maxAmountPerTx` is set to a large raw value aligned with an 18-decimal asset (e.g., `1e18`)
(f). The account's collateral valuation supports the borrow under the LTV ceiling
(g). Manager acts within the registered permission; `onBehalfOf/receiver` equals the account

Octane Security

Warning #10

Predictable nonce-epoch bump in SailKernel.setManager enables post-rotation reattachment of pre-signed permissions causing immediate reuse of pre-rotation configs without fresh consent

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Acknowledged

The actor is the permissionSigner – the trusted role that authorizes permissions in the first place – pre-signing a next-epoch register/replace, and any reattached permission stays strictly within the bounds that signer already authorized. No signature is forged and no value moves outside the signed envelope. The proposed remediation binds the manager into the registration/replacement signing scheme, a change to the trusted core's EIP-712 typehashes with cross-system signing impact; we are declining that pre-launch given the low, in-envelope severity and mitigating it operationally (do not pre-sign future-epoch registration messages).

Because setManager adds a fixed `NONCE_EPOCH_INCREMENT` (2^{128}) to signerNonces and clears only currently registered permissions, a permissionSigner's Register/Replace signature pre-signed at `nonce = current + 2^{128}` becomes valid immediately after rotation. If the permission was configured while unregistered, its `configuredEpoch` matches the kernel's unchanged `registrationEpoch` (often 0), so upon re-registration the stored config becomes active without a fresh configure/sign step at rotation time.

SailKernel.setManager clears the registry and bumps signerNonces, batchNonces, and managerNonces by a fixed `NONCE_EPOCH_INCREMENT` (2^{128}). It also bumps `registrationEpoch` only for permissions that were actually registered at the time of rotation. `ConfigurablePermission` supports `configure-before-register` and stamps `configuredEpoch` with the current `kernel.registrationEpoch(account, permission)`, which for an unregistered permission is typically 0 and remains unchanged across rotation.

`registerPermission/replacePermission` require an EIP-712 signature from permissionSigner with an exact-equality nonce check against `signerNonces[account]`. If, prior to rotation, the permissionSigner pre-signed a `RegisterPermission/ReplacePermissions` message at `nonce = current signerNonces + 2^{128}` (with a long deadline), that signature is invalid pre-rotation but becomes valid immediately after setManager increments `signerNonces` by $+2^{128}$. Broadcasting it post-rotation (by the new manager or any relayer) re-registers the permission(s) without any fresh permissionSigner action at rotation time.

Upon re-registration, the kernel passes `ctx.configEpoch = registrationEpoch(account, permission)` into the template. For a permission never registered before, this is 0, matching `configuredEpoch (0)` stored during pre-rotation configure, so `_configCurrent()` passes and the old config becomes active. This subverts the practical effect of the nonce-epoch rotation ("no mandate silently transfers") in cases where operators pre-sign future-epoch register/replace messages. No signature is forged and value still moves within the previously signed permission envelope.

file: contracts/core/SailKernel.sol

```
150  /// @notice EIP-712 type hash for single-permission registra-
    tion.
151  - ///          Type string: "RegisterPermission(address
    account,address permission,uint256 nonce,uint256 deadline)"
152    bytes32 public constant REGISTER_PERMISSION_TYPEHASH =
    keccak256(
153  - "RegisterPermission(address account,address permis-
    sion,uint256 nonce,uint256 deadline)"
154  );
...

162  /// @notice EIP-712 type hash for atomic permission replacement.
163  - ///          Type string: "ReplacePermission(address account,ad-
    dress oldPermission,address newPermission,uint256 nonce,uint256
    deadline)"
164    bytes32 public constant REPLACE_PERMISSION_TYPEHASH =
    keccak256(
165  - "ReplacePermission(address account,address oldPermis-
    sion,address newPermission,uint256 nonce,uint256 deadline)"
166  );
167
168  /// @notice EIP-712 type hash for atomic batch permission
    replacement.
169  - ///          Type string: "ReplacePermissions(address ac-
    count,address[] oldPermissions,address[] newPermissions,uint256
    nonce,uint256 deadline)"
170    bytes32 public constant REPLACE_PERMISSIONS_TYPEHASH =
    keccak256(
171  - "ReplacePermissions(address account,address[] oldPermis-
    sions,address[] newPermissions,uint256 nonce,uint256 deadline)"
172  );
...
```

```

192  /// @notice EIP-712 type hash for batch permission registration.
193 - ///      Type string: "RegisterPermissions(address
      account,address[] permissions,uint256 nonce,uint256 deadline)"
194  ///      The `address[]` field is encoded as keccak256 of
      the ABI-packed padded addresses
195  ///      per EIP-712 §4 (see `_hashAddressArray`).
196  bytes32 public constant REGISTER_PERMISSIONS_TYPEHASH =
      keccak256(
197 - "RegisterPermissions(address account,address[] permis-
      sions,uint256 nonce,uint256 deadline)"
198  );

...

1006 _verifySignerSig(
1007     account,
1008 - keccak256(abi.encode(REGISTER_PERMISSION_TYPEHASH,
      account, permission, nonce, deadline)),
1009     sig
1010 );

...

1074 _verifySignerSig(
1075     account,
1076 - keccak256(abi.encode(REPLACE_PERMISSION_TYPEHASH,
      account, oldPermission, newPermission, nonce, deadline)),
1077     sig
1078 );

...

1133 REPLACE_PERMISSIONS_TYPEHASH,
1134     account,

1135 _hashAddressArray(oldPermissions),
1136 _hashAddressArray(newPermissions),

...

1310 REGISTER_PERMISSIONS_TYPEHASH,
1311     account,

1312 _hashAddressArray(permissions),
1313     nonce,

```



```

150  /// @notice EIP-712 type hash for single-permission registra-
    tion.
151 +///          Type string: "RegisterPermission(address
        account,address manager,address permission,uint256 nonce,uint256
        deadline)"
152  bytes32 public constant REGISTER_PERMISSION_TYPEHASH =
        keccak256(
153 + "RegisterPermission(address account,address manager,address
        permission,uint256 nonce,uint256 deadline)"
154  );

...

162  /// @notice EIP-712 type hash for atomic permission replacement.
163 +///          Type string: "ReplacePermission(address account,ad-
        dress manager,address oldPermission,address newPermission,uint256
        nonce,uint256 deadline)"
164  bytes32 public constant REPLACE_PERMISSION_TYPEHASH =
        keccak256(
165 + "ReplacePermission(address account,address manager,ad-
        dress oldPermission,address newPermission,uint256 nonce,uint256
        deadline)"
166  );
167
168  /// @notice EIP-712 type hash for atomic batch permission
        replacement.
169 +///          Type string: "ReplacePermissions(address
        account,address manager,address[] oldPermissions,address[] new-
        Permissions,uint256 nonce,uint256 deadline)"
170  bytes32 public constant REPLACE_PERMISSIONS_TYPEHASH =
        keccak256(
171 + "ReplacePermissions(address account,address man-
        ager,address[] oldPermissions,address[] newPermissions,uint256
        nonce,uint256 deadline)"
172  );

...

192  /// @notice EIP-712 type hash for batch permission registration.
193 +///          Type string: "RegisterPermissions(address ac-
        count,address manager,address[] permissions,uint256 nonce,uint256
        deadline)"
194  ///          The `address[]` field is encoded as keccak256 of
        the ABI-packed padded addresses
195  ///          per EIP-712 §4 (see `_hashAddressArray`).

```

```

196     bytes32 public constant REGISTER_PERMISSIONS_TYPEHASH =
        keccak256(
197 + "RegisterPermissions(address account,address manager,address[] permissions,uint256 nonce,uint256 deadline)"
198     );

    ...

1006 _verifySignerSig(
1007     account,
1008+ keccak256(abi.encode(REGISTER_PERMISSION_TYPEHASH,
        account, configs[account].manager, permission, nonce, deadline)),
1009     sig
1010 );

    ...

1074 _verifySignerSig(
1075     account,
1076+ keccak256(abi.encode(REPLACE_PERMISSION_TYPEHASH,
        account, configs[account].manager, oldPermission, newPermission,
        nonce, deadline)),
1077     sig
1078 );

    ...

1133 REPLACE_PERMISSIONS_TYPEHASH,
1134     account,
1135+ configs[account].manager,
1136 _hashAddressArray(oldPermissions),
1137 _hashAddressArray(newPermissions),

    ...

1311 REGISTER_PERMISSIONS_TYPEHASH,
1312     account,
1313+ configs[account].manager,
1314 _hashAddressArray(permissions),
1315     nonce,

```

Severity

Impact Explanation: [Low] Funds can move immediately post-rotation under previously configured permissions, but always within the signed permission envelope (addresses and behaviors previously authorized by the permission-Signer). No cross-envelope theft or core invariant break occurs.

Likelihood Explanation: [Low] Exploitation relies on uncommon operator practices: the permissionSigner must have pre-signed register/replace at exactly next-epoch nonce ($\text{current} + 2^{128}$) with a long deadline and made that signature available; these are rare, operator-dependent preconditions outside an attacker's full control.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 – Pre-signed next-epoch RegisterPermission reattaches a previously configured-but-unregistered template post-rotation: The owner calls setManager; signer-Nonces increase by 2^{128} while registrationEpoch for the unregistered template remains 0. A relayer submits a RegisterPermission(A, T, nonce = old + 2^{128}) that was pre-signed by the permissionSigner before rotation. The kernel verifies and registers T, and dispatch under T succeeds because $\text{configuredEpoch} == \text{ctx.configEpoch} == 0$.

Preconditions / Assumptions

- (a). Account A is registered; permissionSigner (PS) controls signer ops; manager will be rotated.
- (b). Template T was configured while unregistered; $\text{configuredEpoch}(A,T) == \text{registrationEpoch}(A,T) == 0$.
- (c). PS pre-signed RegisterPermission(A, T) at nonce = current signerNonces + 2^{128} with a long deadline and the signature is available to the submitter.
- (d). Owner rotates manager via setManager (signerNonces += 2^{128} ; only currently registered permissions cleared; T remains unregistered with epoch 0).

Scenario 2.

Scenario 2 – Pre-signed next-epoch ReplacePermissions swaps a set immediately after rotation: The owner rotates the manager. A relayer submits a pre-signed ReplacePermissions(A, oldPerms[], newPerms[], nonce = old + 2^{128}). The kernel atomically reattaches newPerms that were configured while unregistered; their configuredEpoch (0) matches ctx.configEpoch (0), so they are active without fresh configure.

Preconditions / Assumptions

- (a). Account A registered; PS controls signer ops; manager will be rotated.
- (b). newPerms[] were configured while unregistered; each has configuredEpoch == registrationEpoch == 0.
- (c). PS pre-signed ReplacePermissions(A, oldPerms[], newPerms[]) at nonce = current signerNonces + 2^{128} with a long deadline and the signature is available.
- (d). Owner rotates manager via setManager (signerNonces += 2^{128} ; only currently registered permissions cleared; unregistered epochs unchanged).

Scenario 3.

Scenario 3 – ApproveAndCallBatchPermission reattachment with requireRecipientIsAccount=false: Prior to rotation, the template P was configured (requireRecipientIsAccount=false) while unregistered and a next-epoch RegisterPermission was pre-signed. After setManager, the relayer registers P using that signature. The new manager can immediately execute approve/consume/reset brackets at allowlisted pairs, with outputs unconstrained by recipient (as allowed by config), without fresh post-rotation consent.

Preconditions / Assumptions

- (a). Account A registered; PS controls signer ops; manager will be rotated.
- (b). ApproveAndCallBatchPermission P configured pre-rotation while unregistered with requireRecipientIsAccount=false; configuredEpoch == registrationEpoch == 0.
- (c). PS pre-signed RegisterPermission(A, P) at nonce = current signerNonces + 2^{128} with a long deadline and the signature is available.
- (d). Owner rotates manager via setManager (signerNonces += 2^{128} ; unregistered epochs unchanged).

Warning #11

Missing Safe fallback-relay guard in SailKernel.recordDeposit/recordWithdrawal causes unauthorized mutation of informational counters

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Out of scope

`recordDeposit/recordWithdrawal` write only the `cumulativeDeposits/cumulativeWithdrawals` counters, which feed no on-chain logic – no NAV, fee, or LTV computation reads them; they are advisory values for off-chain tooling. The precondition is again a `permissionSigner Safe` whose `fallbackHandler` is set to the kernel (a self-inflicted, out-of-model misconfiguration). The worst case is cosmetic inflation of an advisory counter with no on-chain consequence. Documented as an operator requirement; no trusted-core guard added for an off-chain-only counter.

`recordDeposit` and `recordWithdrawal` in `SailKernel` authorize solely by `msg.sender == permissionSigner` and lack an exact-calldata-length guard. If the `permissionSigner` is a `Gnosis Safe` whose fallback handler is set to the `SailKernel`, any external caller can send these function selectors to the `Safe`; the `Safe` fallback forwards to the kernel with `msg.sender = the Safe` and trailing 20 bytes, which `Solidity` ignores for fixed-arg functions. The kernel then updates `cumulativeDeposits/cumulativeWithdrawals` without any owner/signer authorization.

`SailKernel`'s `recordDeposit(account, amount)` and `recordWithdrawal(account, amount)` check only that `msg.sender` equals `configs[account].permissionSigner` and do not verify exact calldata length. `Gnosis Safe`'s `FallbackManager` forwards unknown calls to its configured fallback handler, appending 20 bytes of the original caller's address and setting `msg.sender` to the `Safe`. `Solidity`'s ABI decoding for fixed-argument functions ignores trailing bytes, so these forwarded calls cleanly match the function selector and decode arguments. If a `permissionSigner` is a `Safe` whose fallback handler is set to the `SailKernel`, an arbitrary external user can send the `recordDeposit/recordWithdrawal` calldata to the `Safe`; the `Safe` forwards it to the kernel as `msg.sender = Safe`, satisfying the authorization check and mutating `cumulativeDeposits/cumulativeWithdrawals`. Other `Safe`-authorized fixed-arg entrypoints (`setManager`, `collectFees`) include exact-length guards and are protected, but these two are not. No funds move today; the impact is unauthorized, integrity-only inflation of informational counters used by off-chain tooling and potential future logic.

file: contracts/core/SailKernel.sol



```

134  ///      fallback-relay rationale.
135  uint256 private constant COLLECT_FEES_CALLDATA_LEN = 132;

136
137  //
  -----

...

1884  /// @param amount Deposit amount to record (in the account's
      base currency units).
1885  function recordDeposit(address account, uint256 amount)
      external nonReentrant {

1886  _requireRegistered(account);
1887  if (msg.sender != configs[account].permissionSigner) revert
      NotPermissionSigner();

...

1894  /// @param amount Withdrawal amount to record (in the
      account's base currency units).
1895  function recordWithdrawal(address account, uint256 amount)
      external nonReentrant {

1896  _requireRegistered(account);
1897  if (msg.sender != configs[account].permissionSigner) revert
      NotPermissionSigner();

```

```

134  ///      fallback-relay rationale.
135  uint256 private constant COLLECT_FEES_CALLDATA_LEN = 132;
136 + uint256 private constant RECORD_PRINCIPAL_CALLDATA_LEN = 68;
      // selector (4) + 2 static args (64)
137
138  //
  -----

...

1885  /// @param amount Deposit amount to record (in the account's
      base currency units).
1886  function recordDeposit(address account, uint256 amount)
      external nonReentrant {

```

```
1887+ if (msg.data.length != RECORD_PRINCIPAL_CALldata_LEN)
    revert UnexpectedCalldataLength();
1888 _requireRegistered(account);
1889 if (msg.sender != configs[account].permissionSigner) revert
    NotPermissionSigner();

...

1896 /// @param amount Withdrawal amount to record (in the
    account's base currency units).
1897 function recordWithdrawal(address account, uint256 amount)
    external nonReentrant {
1898+ if (msg.data.length != RECORD_PRINCIPAL_CALldata_LEN)
    revert UnexpectedCalldataLength();
1899 _requireRegistered(account);
1900 if (msg.sender != configs[account].permissionSigner) revert
    NotPermissionSigner();
```

Severity

Impact Explanation: [Low] The effect is unauthorized updates to informational counters (cumulativeDeposits/cumulativeWithdrawals). No funds move, no fee over-collection occurs, and core functionality remains intact.

Likelihood Explanation: [Low] Exploitation requires an uncommon, operator-controlled Safe configuration (fallback handler set to the SailKernel) that the attacker cannot induce and which typically breaks normal ERC-1271-based signer flows, making it rare in practice.

Exploitation

Exploitation Scenarios:

Scenario 1.

Single account: The account is registered and its permissionSigner is a Gnosis Safe whose fallback handler is set to the SailKernel. An attacker sends recordDeposit(account, X) or recordWithdrawal(account, X) calldata to the Safe. The Safe fallback forwards to the kernel with msg.sender = the Safe and 20 extra bytes; the kernel accepts and increments the respective cumulative counter without any owner authorization.

Preconditions / Assumptions

- (a). registered[account] == true
- (b). configs[account].permissionSigner is a Gnosis Safe S
- (c). Safe S.fallbackHandler == SailKernel (non-default operator configuration)

Scenario 2.

Multiple accounts share a misconfigured signer Safe: Several registered accounts use the same permissionSigner Safe, whose fallback handler is set to the SailKernel. An attacker sends recordDeposit/recordWithdrawal calls for each account to the Safe, and the kernel increments counters for all affected accounts.

Preconditions / Assumptions

- (a). For each A_i , registered[A_i] == true
- (b). For each A_i , configs[A_i].permissionSigner == the same



Gnosis Safe S

(c). Safe S.fallbackHandler == SailKernel (non-default operator configuration)

Scenario 3.

permissionSigner equals the account Safe: The registered account configures itself (the Safe) as its permission-Signer and sets its fallback handler to the SailKernel.

An attacker calls recordDeposit/recordWithdrawal on the Safe; the fallback forwards to the kernel as msg.sender = the Safe, causing unauthorized counter updates.

Preconditions / Assumptions

(a). registered[account] == true

(b). configs[account].permissionSigner == account (the Safe itself)

(c). account.fallbackHandler == SailKernel (non-default operator configuration)

Warning #12

Missing pause guard in SailKernel.registerAccount/createAccount allows policy attachment during pause

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Acknowledged

Onboarding is callable during an emergency pause, but no funds can move while paused – `dispatch`, `dispatchBatch`, and `collectFees` are all pause-gated, and `setFeePolicy` already blocks non-zero policy updates during pause. The only effect is that a new account can be registered (and a policy attached) during the pause window, usable only after the pause lifts. This is a containment nuance, not a fund-risk or invariant gap; the pause's scope (it gates fund movement, not onboarding) is documented rather than expanded.

SailKernel's `registerAccount` and `createAccount` are not gated by `whenNotPaused`, allowing `_registerAccount` to attach a non-zero allowlisted `feePolicy` while the protocol is paused, despite `setFeePolicy` forbidding non-zero updates during pause. No funds move during pause, but this weakens pause-time containment for new accounts.

While the protocol is paused (`governance.isPaused() == true`), `SailKernel.registerAccount` and `SailKernel.createAccount` remain callable and both invoke `_registerAccount` without a pause check. `_registerAccount` sets `registered[account] = true`, `sessionActive = true`, and configures `feePolicy/feeAsset` if `feePolicy` is allowlisted. In contrast, `setFeePolicy` explicitly blocks non-zero updates while paused. Fund-moving and permission-adding operations (`dispatch`, `dispatchBatch`, `collectFees`, `registerPermission(s)`, `replacePermission(s)`) are properly gated by `whenNotPaused`, so no funds can move during the pause. However, the pause does not fully contain onboarding/policy-attachment surfaces for new accounts: a non-zero `feePolicy` can be attached during pause via onboarding, and immediately used after unpause (e.g., to collect fees or `dispatch` once permissions are added). For `registerAccount`, Safe owners explicitly authorize the `feePolicy` in the signed digest; for `createAccount`, the policy is attached at initial registration. This is an operational containment gap rather than a direct theft vector.

file: contracts/core/SailKernel.sol

```
909     if (permissionSigner == address(0) || manager == address(0))  
        revert ZeroAddress();
```



```
910     if (feePolicy != address(0) && !governance.trustedFeePolicy(feePolicy)) revert UntrustedFeePolicy(feePolicy);

911     registered[account] = true;
912     configs[account] = AccountConfig({
```

```
909     if (permissionSigner == address(0) || manager == address(0))
        revert ZeroAddress();
910     if (feePolicy != address(0) && !governance.trustedFeePolicy(feePolicy)) revert UntrustedFeePolicy(feePolicy);
911 + if (governance.isPaused() && feePolicy != address(0))
        revert ProtocolPaused();
912     registered[account] = true;
913     configs[account] = AccountConfig({
```

Severity

Impact Explanation: [Low] No direct principal loss, no invariant break, and no funds move during pause. The impact is a containment/policy-enforcement gap: new accounts can attach a non-zero feePolicy during pause, enabling immediate post-unpause activity.

Likelihood Explanation: [Low] Requires the rare/exceptional condition of an emergency pause and owner/operator onboarding activity within a limited (up to 72h) window.

Exploitation

Exploitation Scenarios:

Scenario 1.

During a governance-imposed pause, an operator onboards new accounts via `registerAccount` (with owner signatures over a digest that includes `feePolicy/feeAsset`) or via `createAccount`, both of which call `_registerAccount` and attach a non-zero allowlisted `feePolicy` while paused. No funds can move during the pause, but immediately after unpause the operator can collect fees or begin dispatching (after registering permissions), reducing the effectiveness of the pause as a full freeze on new policy attachments.

Preconditions / Assumptions

- (a). The protocol is in an emergency pause (`governance.isPaused() == true`).
- (b). The `feePolicy` used is already in `governance.trustedFeePolicy`.
- (c). For `registerAccount`: Safe owners sign a `Safe.execTransaction` calling `registerAccount` and sign the EIP-712 digest that includes `feePolicy` and `feeAsset`.
- (d). For `createAccount`: `safeFactory`, `safeSingleton`, and the module-setup helper are allowlisted and codehash-pinned; the Safe is initialized with the kernel module enabled.
- (e). No fund-moving operations are attempted during the pause (`dispatch/dispatchBatch/collectFees` are `whenNotPaused`).

Warning #13

Shared nonce between `configure` and `setAgentIdentity` in `ConfigurablePermission` causes recoverable config-update denial and temporary use of old bounds

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Acknowledged

`configure` and `setAgentIdentity` share a per-account nonce, so a pre-signed identity update can consume the nonce and cause a pending `configure` signature to fail verification – a recoverable delay in which the prior configuration remains in effect for, at most, one additional call within already-authorized bounds. No allowlist or recipient pin is bypassed and the issue is resolved by re-signing. Splitting the nonce namespaces would change the off-chain signing surface; given the impact is a recoverable delay within the signed envelope, we are accepting it and documenting that an operator should not co-sign a `configure` and an identity update at the same nonce.

`ConfigurablePermission` uses a single per-account nonce for both `configure()` and `setAgentIdentity()`. A valid `SetAgentIdentity` signature mined first consumes the shared nonce, causing a concurrently pre-signed `configure()` to fail verification. Identity updates are metadata-only and do not change `isConfigured/configuredEpoch`, so `evaluate()` behavior stays bound to the previous configuration until the operator re-signs `configure`. This enables a short window where operations proceed under old bounds, but no permissions or recipients are bypassed, and the issue is recoverable by re-signing.

In `ConfigurablePermission`, both `configure(account, params, deadline, sig)` and `setAgentIdentity(account, ref, deadline, sig)` read the same per-account counter `configNonces[account]`, bind the current `kernel.registrationEpoch(account, address(this))` into the EIP-712 struct, verify a signature from the account's `permissionSigner`, and then increment the same nonce. `setAgentIdentity` writes only `_agentIdentities[account]` and emits an event; it does not touch `isConfigured[account]` or `configuredEpoch[account]`. All included launch templates' `evaluate()` paths gate on `_configCurrent(account, ctx.configEpoch)`, i.e., they require `isConfigured` true and `configuredEpoch` equal to the kernel's current epoch. As a result, if a valid `SetAgentIdentity` signature (at the same nonce/epoch) is submitted before a pending `configure`, it advances the shared nonce so the pre-signed `configure` no longer verifies and reverts. Until the operator re-signs and re-applies `configure` using the new nonce, the prior configuration remains in effect. Identity updates cannot revive stale configurations (`configuredEpoch` is not updated by identity), and no allowlist or recipient pinning is bypassed. The window is further limited by epoch and deadline binding (any

revoke/replace/rotation bumps epoch; expired deadlines fail). The effect is a recoverable denial/delay of configuration changes with potential one-more-call under previously authorized bounds. Mitigations include sequencing signatures (do not sign two ops at the same nonce), using `configureDirect` when feasible, or using `replacePermission` for atomic tightening.

file: contracts/templates/ConfigurablePermission.sol

```
62     mapping(address account => uint256) public configNonces;
63     mapping(address account => bool)      public isConfigured;
...
176     if (!kernel.registered(account)) revert AccountNotRegistered(account);
177
178 - uint256 nonce = configNonces[account];
179 // Bind identity writes to the current epoch too, so a
180 // stale identity signature cannot be
181 // replayed across an epoch change. NOTE: identity writes
182 // do NOT stamp configuredEpoch -
...
189     if (!_verifySig(permSigner, digest, sig)) revert
190         InvalidSignature();
191
192 - configNonces[account] = nonce + 1;
193 _agentIdentities[account] = ref;
194 emit AgentIdentitySet(account, ref.agentId, ref.agentWallet);
...
205     if (msg.sender != permSigner) revert NotPermissionSigner(msg.sender, permSigner);
206
207 - configNonces[account]++;
208 _agentIdentities[account] = ref;
209 emit AgentIdentitySet(account, ref.agentId, ref.agentWallet);
```

```
62     mapping(address account => uint256) public configNonces;
```

```

63 + mapping(address account => uint256) public identityNonces;
64   mapping(address account => bool)      public isConfigured;

...

177   if (!kernel.registered(account)) revert AccountNotRegis-
    tered(account);
178
179 + uint256 nonce = identityNonces[account];
180 // Bind identity writes to the current epoch too, so a
    stale identity signature cannot be
181 // replayed across an epoch change. NOTE: identity writes
    do NOT stamp configuredEpoch –

...

190   if (!_verifySig(permSigner, digest, sig)) revert
    InvalidSignature();
191
192 + identityNonces[account] = nonce + 1;
193 _agentIdentities[account] = ref;
194   emit AgentIdentitySet(account, ref.agentId, ref.agentWal-
    let);

...

206   if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);
207
208 + identityNonces[account]++;
209 _agentIdentities[account] = ref;
210   emit AgentIdentitySet(account, ref.agentId, ref.agentWal-
    let);

```

Severity

Impact Explanation: [Low] Per-account, recoverable denial/delay of configuration changes; value remains within previously authorized allowlists/caps/recipient pinning. No cross-envelope fund movement or invariant break; resolved by re-signing configure or using direct/atomic flows.

Likelihood Explanation: [Low] Requires operator error (two different signatures at the same nonce), the identity signature to be available to a third party, valid deadlines, and no epoch change. The attacker gains only a brief timing advantage; no direct profit or signature minting ability.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1: A third party holding a valid `SetAgentIdentity` signature at the same nonce front-runs it before a pending `configure()` that tightens bounds. `setAgentIdentity` verifies and increments the nonce; the `configure` signature then fails verification. Until the operator re-signs `configure`, the old configuration remains live, allowing one or more calls under previously authorized allowlists/caps.

Preconditions / Assumptions

- (a). Account is registered and has an existing configuration (`isConfigured` true and `configuredEpoch` matches kernel epoch).
- (b). The `permissionSigner` produces two EIP-712 signatures at the same on-chain nonce and epoch: one for `configure()` (tightening bounds) and one for `setAgentIdentity()`.
- (c). A third party (e.g., relay/manager) holds the valid identity signature.
- (d). No intervening epoch change (no `revoke/replace/rotation`) and deadlines are not expired.

Scenario 2.

Scenario 2: During a `configure-then-replace` migration, an identity signature for the new permission is front-run

first, consuming the shared nonce. The intended `configure(newPerm)` fails, but the operator proceeds to replace `oldPerm`→`newPerm`. The new permission is registered but not configured; `evaluate()` fails closed until `configure` is re-signed and applied.

Preconditions / Assumptions

- (a). Operator intends to migrate by pre-configuring a new permission and then calling `replacePermission`.
- (b). For the new permission address, the `permissionSigner` mistakenly signs `configure(newPerm, ...)` and `setAgentIdentity(newPerm, ...)` at the same nonce/epoch.
- (c). A third party submits the identity signature first; `configure(newPerm)` then fails verification.
- (d). Operator proceeds to replace `oldPerm`→`newPerm` without verifying `configure` success; deadlines valid and no epoch change.

Scenario 3.

Scenario 3: After an identity front-run invalidates a tightening `configure` on `SwapPermission` or `BorrowPermission`, the manager promptly executes one call still permitted by the old per-transaction cap or bound before the operator re-applies `configure`.

Preconditions / Assumptions

- (a). Same as Scenario 1 (shared-nonce signatures and identity sig broadcast first).
- (b). Manager has a prepared transaction that remains permitted by the old config (within old allowlists/caps/oracle/recipient constraints).
- (c). Operator has not yet re-signed and applied the new `configure`.